

سید محمد



معاونت تحقیق و فن آوری

به نام خدا

مشور اخلاق پژوهش

بیاری از خداوند سبحان و اعتقاد به این که عالم محضر خداست و بهواره ناظر بر اعمال انسان و به منظور پاس داشت مقام بلند دانش و تحقیق و نظر به اهمیت جایگاه دانشگاه در اعتلای فرهنگ و تمدن بشری، مادران و دانشجویان و اعضا هیأت علمی واحد های دانشگاه آزاد اسلامی متعهد می گردیم اصول زیر را در انجام فعالیت های تحقیقی مد نظر قرار داده و از آن تخطی نکنیم:

- ۱- اصل حقیقت جویی: تلاش در راستای پی جویی حقیقت و وفاداری به آن و دوری از هرگونه پنهان سازی حقیقت.
- ۲- اصل رعایت حقوق: التزام به رعایت کامل حقوق تحقیگران و پژوهیدگان (انسان، حیوان و نبات) و سایر صاحبان حق.
- ۳- اصل مالکیت مادی و معنوی: تعهد به رعایت کامل حقوق مادی و معنوی دانشگاه و کلیه بهکاران تحقیق.
- ۴- اصل منفعت ملی: تعهد به رعایت مصالح ملی و در نظر داشتن پیشبرد و توسعه کشور در کلیه مراحل تحقیق.
- ۵- اصل رعایت انصاف و امانت: تعهد به اجتناب از هرگونه جانب داری غیر علمی و حفاظت از اموال، تجهیزات و منابع در اختیار.
- ۶- اصل رازداری: تعهد به صیانت از اسرار و اطلاعات محرمانه افراد، سازمان ها و کلیه افراد و نهادهای مرتبط با تحقیق.
- ۷- اصل احترام: تعهد به رعایت حریم ها و حرمت ها در انجام تحقیقات و رعایت جانب نقد و خودداری از هرگونه حرمت شکنی.
- ۸- اصل ترویج: تعهد به رواج دانش و اشاعه نتایج تحقیقات و انتقال آن به بهکاران علمی و دانشجویان به غیر از مواردی که منع قانونی دارد.
- ۹- اصل برائت: التزام به برائت جویی از هرگونه رفتار غیر حرفه ای و اعلام موضع نسبت به کسانی که حوزه علم و تحقیق را به شبهه های غیر علمی می آلودند.



تعهدنامه اصالت پایان نامه تحصیلی

اینجانب محمد نوخیز دانش آموخته مقطع کارشناسی ارشد ناپیوسته در رشته کامپیوتر گرایش شبکه‌های کامپیوتری که در تاریخ ۱۴۰۴/ ۱۱ / ۲۵ از پایان نامه خود تحت عنوان: بهبود عملکرد رابط‌های برنامه‌نویسی کاربردی در محیط رایانش ابری با استفاده از الگوریتم یادگیری تقویتی و شبکه عصبی با حافظه کوتاه مدت در بستر میکروسرویس‌ها با کسب نمره و درجه دفاع نموده‌ام بدین وسیله متعهد می‌شوم:

- (۱) این پایان نامه / رساله حاصل تحقیق و تحقیق انجام شده توسط اینجانب بوده و در مواردی که از دستاوردهای علمی و تحقیقی دیگران (اعم از پایان نامه، کتاب، مقاله و.....) استفاده نموده‌ام، مطابق ضوابط و رویه موجود، نام منبع مورد استفاده و سایر مشخصات آن را در فهرست مربوطه ذکر و درج کرده‌ام.
- (۲) این پایان نامه / رساله قبلاً برای دریافت هیچ مدرک تحصیلی (هم سطح، پایین تر یا بالاتر) در سایر دانشگاه ها و مؤسسات آموزش عالی ارائه نشده است.
- (۳) چنانچه بعد از فراغت از تحصیل، قصد استفاده و هرگونه بهره برداری اعم از چاپ، ثبت اختراع و..... از این پایان نامه داشته باشم، از حوزه معاونت تحقیقی واحد مجوز های مربوطه را اخذ نمایم.
- (۴) چنانچه در هر مقطعی زمانی خلاف موارد فوق ثابت شود، عواقب ناشی از آن را می‌پذیرم و واحد دانشگاهی مجاز است با اینجانب مطابق ضوابط و مقررات رفتار نموده و در صورت ابطال مدرک تحصیلی‌ام هیچگونه ادعایی نخواهم داشت.

نام و نام خانوادگی:

تاریخ و امضاء



دانشگاه آزاد اسلامی

واحد علوم و تحقیقات

دانشکده مهندسی برق و کامپیوتر، گروه کامپیوتر

پایان نامه برای دریافت درجه کارشناسی ارشد در رشته مهندسی کامپیوتر (M.Sc)

گرایش شبکه‌های کامپیوتری

عنوان:

بهبود عملکرد رابط‌های برنامه‌نویسی کاربردی در محیط رایانش ابری با استفاده از الگوریتم یادگیری تقویتی و شبکه عصبی با حافظه کوتاه‌مدت در بستر میکروسرویس‌ها

استاد راهنما:

دکتر محمد رضا علی‌زاده

استاد مشاور:

دکتر فرساد زمانی بروجنی

نگارش:

محمد نوخیز

بهار ۱۴۰۵

سپاسگزاری از

تقديم به

فهرست مطالب

عنوان	شماره صفحه
چکیده	۱
فصل اول کلیات تحقیق	
۱-۱- مقدمه	۳
۱-۲- بیان مسئله	۴
۱-۳- جنبه نوآوری تحقیق	۵
۱-۴- اهداف تحقیق	۷
۱-۵- فرضیه‌های تحقیق	۸
۱-۶- اهمیت و ضرورت انجام تحقیق	۸
۱-۷- متغیرهای تحقیق	۹
۱-۸- روش کار و روش‌شناسی تحقیق	۱۰
۱-۹- مروری بر ساختار پایان‌نامه	۱۱
فصل دوم مروری بر ادبیات تحقیق و پیشینه تحقیق ۱۲	
۲-۱- مقدمه	۱۳
۲-۲- ادبیات تحقیق	۱۳
۲-۲-۱- رایانش ابری	۱۳
۲-۲-۲- معماری میکروسرویس	۱۴
۲-۲-۳- رابطه‌های برنامه نویسی کاربردی (APIs)	۱۵
۲-۲-۴- یادگیری ماشین	۱۶
۲-۲-۵- یادگیری تقویتی	۱۷
۲-۲-۶- الگوریتم‌های رایج یادگیری ماشین	۱۸
۲-۲-۷- شبکه‌های عصبی بازگشتی با حافظه کوتاه مدت (LSTM)	۲۰
۲-۲-۸- توازن بار	۲۲
۲-۲-۹- کوپرنیتیس	۲۲
۲-۲-۱۰- داکر	۲۲
۲-۲-۱۱- پرومیتئوس و گرافانا	۲۲
۲-۲-۱۲- کیفیت سرویس (QoS)	۲۳
۲-۳- پیشینه تحقیق	۲۳
۲-۳-۱- تحلیل مقالات	۳۲
۲-۳-۲- شکاف تحقیقاتی	۳۷
۲-۴- جمع‌بندی	۳۸
فصل سوم روش اجرای تحقیق	
۳-۱- مقدمه	۴۱
۳-۲- طراحی روش پیشنهادی	۴۱

۴۱	۳-۲-۱- ساختار کلی مدل پیشنهادی
۴۳	۳-۲-۲- ماژول پیش‌بینی بار کاری
۴۵	۳-۲-۳- ماژول تصمیم‌گیری تخصیص منابع
۴۷	۳-۲-۴- ماژول مانیتورینگ و بازخورد
۴۶	۳-۲-۵- تعامل بین ماژول‌ها و سیستم ارکستریشن Kubernetes
۴۷	۳-۲-۶- مزایای مدل پیشنهادی
۴۸	۳-۳- محیط پیاده‌سازی و ابزارهای مورد استفاده
۴۸	۳-۳-۱- محیط سخت‌افزاری
۴۸	۳-۳-۲- محیط نرم‌افزاری
۴۹	۳-۳-۴- فرآیند پیاده‌سازی و اجرا
۴۹	۳-۴- الگوریتم پیشنهادی
۴۹	۳-۴-۱- توضیح الگوریتم
۵۶	۳-۴-۲- شبه کد الگوریتم
۵۷	۳-۵- جمع‌بندی

فصل چهارم تجزیه و تحلیل داده‌ها

۵۹	۴-۱- مقدمه
۶۰	۴-۲- ارزیابی
۶۰	۴-۲-۱- مقالات مورد ارزیابی
۶۱	۴-۲-۲- پارامترهای ارزیابی
۶۳	۴-۲-۳- نتایج ارزیابی
۶۹	۴-۳- جمع‌بندی

فصل پنجم نتیجه‌گیری و پیشنهادات

۷۱	۵-۱- مقدمه
۷۱	۵-۲- پاسخ به سوالات تحقیق
۷۲	۵-۳- محدودیت‌ها
۷۲	۵-۴- پیشنهادات برای تحقیقات آینده

۷۵	منابع
----	-------

۷۹	پیوست‌ها
----	----------

۷۷	چکیده انگلیسی
----	---------------

فهرست جدول‌ها

عنوان	شماره صفحه
جدول (۱-۱) متغیرهای تحقیق	۱۰
جدول (۱-۲) مروری بر تحقیقات گذشته	۲۹
جدول (۲-۲) خلاصه مقالات مربوط به مدل‌های پیش‌بینی محور	۳۳
جدول (۳-۲) خلاصه مقالات مدل‌های یادگیری تقویتی	۳۳
جدول (۴-۲) خلاصه مقالات مربوط به مدل‌های ترکیبی	۳۴
جدول (۵-۲) خلاصه مقالات مربوط به روش‌های فراابتکاری و بهینه‌سازی چندهدفه	۳۴
جدول (۶-۲) خلاصه مقالات مربوط به چارچوب‌های ارکستریشن و کنترل هوشمند	۳۵
جدول (۷-۲) خلاصه مقالات مدل‌های ساختارمحور و وابستگی محور	۳۵
جدول (۸-۲) تحلیل مقایسه‌ای مقالات	۳۶
جدول (۱-۳) توضیح نمادهای مورد استفاده	۴۷
جدول (۲-۳) مشخصات سخت افزاری	۴۹
جدول (۳-۳) مشخصات نرم افزاری	۵۰
جدول (۴-۳) توضیح نمادهای مورد استفاده	۵۶
جدول (۱-۴) مقایسه‌ای خلاصه ۴ مقاله منتخب	۶۲

فهرست شکل‌ها

عنوان	شماره صفحه
شکل (۱-۲) طبقه‌بندی پژوهش‌های مرتبط با بهبود عملکرد API مبتنی بر یادگیری ماشین	۳۰
شکل (۲-۲) چارچوب کلی تصمیم‌گیری در روش‌های هوشمند مدیریت منابع API	۳۷
شکل (۳-۲) نمایش شکاف‌های پژوهشی در روش‌های موجود بهبود عملکرد API	۳۸
شکل (۱-۳) معماری روش پیشنهادی	۴۷
شکل (۲-۳) فلوچارت سه ماژول اصلی	۴۷
شکل (۳-۳) الگوریتم RF در روش پیشنهادی	۵۷
شکل (۴-۳) شبه کد روش پیشنهادی	۵۸
شکل (۵-۳) حلقه کلی الگوریتم پیشنهادی	۵۸
شکل (۱-۴) تمامی روش‌ها	۶۵
شکل (۲-۴) تأخیر	۶۶
شکل (۳-۴) بهره‌وری مصرف منابع	۶۶
شکل (۴-۴) نرخ موفقیت پاسخ	۶۷
شکل (۵-۴) مصرف انرژی	۶۷
شکل (۶-۴) هزینه	۶۸
شکل (۷-۴) مقیاس‌پذیری	۶۸

چکیده

در این پژوهش، باهدف بهینه‌سازی عملکرد رابط‌های برنامه‌نویسی کاربردی (API) در معماری‌های میکروسرویسی رایانش ابری، یک مدل هوشمند ترکیبی مبتنی بر شبکه عصبی بازگشتی (LSTM) و یادگیری تقویتی (RL) طراحی و پیاده‌سازی گردید. این مدل با تحلیل داده‌های زمانی ترافیک و بار کاری، الگوهای مصرف منابع را پیش‌بینی نموده و از طریق سیاست یادگیری تقویتی، تخصیص پویا و پیش‌دستانه منابع پردازشی شامل CPU، RAM و GPU را در سطح خوشه‌ی Kubernetes انجام می‌دهد. نتایج آزمایش‌ها در یک محیط عملیاتی واقعی نشان داد که روش پیشنهادی موجب کاهش میانگین تأخیر پاسخ API ها ۳۲-۳۸٪، کاهش ۳۰-۳۵٪ تخطی SLA و افزایش نرخ موفقیت پاسخ‌ها، کاهش هزینه‌های عملیاتی بین ۳۵-۵۰٪، کاهش مصرف انرژی تا ۳۰-۴۵٪ و افزایش مقیاس‌پذیری ۳۵-۴۰٪ می‌شود. در مجموع، ترکیب یادگیری تقویتی و شبکه‌های عصبی LSTM در بستر میکروسرویس‌ها رویکردی مؤثر، سازگار و تعمیم‌پذیر برای بهبود کیفیت سرویس (QoS) و مدیریت هوشمند منابع در محیط‌های پویا، توزیع‌شده و مقیاس‌پذیر ابری به شمار می‌آید.

کلمات کلیدی: رایانش ابری، میکروسرویس، بهینه‌سازی API، یادگیری ماشین، LSTM، یادگیری تقویتی، Kubernetes، کیفیت سرویس، مدیریت منابع.

فصل اول

کلیات تحقیق

در سال‌های اخیر، فناوری رایانش ابری^۱ به‌عنوان یکی از مهم‌ترین دستاوردهای مهم فناوری اطلاعات، انقلابی در نحوه طراحی، توسعه و ارائه خدمات نرم‌افزاری و زیرساختی ایجاد کرده. این فناوری با فراهم کردن بستری پویا، مقیاس‌پذیر و بر پایه تقاضا، امکان استفاده از منابع محاسباتی برای سازمان‌ها فراهم آورده است. مفهوم رایانش ابری، بر اساس اشتراک منابع و ارائه خدمات از طریق اینترنت است؛ طوری که کاربران می‌توانند در هر زمان و از هر مکان، به منابعی مانند فضای ذخیره‌سازی، پردازنده، پایگاه داده و نرم‌افزار دسترسی داشته باشند. این ویژگی، تحول عظیمی در مدل‌های تجاری و فناوری ایجاد کرده رایانش ابری علاوه بر کاهش هزینه‌های زیرساختی، زمینه‌ساز بهبود سازمان‌ها در پاسخ به نیازهای پویا و متغیر بازار شده است. در چنین بستری، بهره‌مندی از معماری میکروسرویس‌ها^۲ به‌عنوان یکی از الگوهای کلیدی جایگاه مهمی پیدا کرده است. زیرا این معماری با تقسیم سیستم‌های پیچیده به مجموعه‌ای از سرویس‌های کوچک، مستقل، انعطاف‌پذیری و مقیاس‌پذیری را افزایش داده است. هر سرویس در این معماری مسئولیت مشخصی دارد و از طریق رابط‌های برنامه‌نویسی کاربردی^۳ (API) با سایر سرویس‌ها ارتباط برقرار می‌کند. با توجه به مزایای رایانش ابری و معماری میکروسرویس‌ها عملکرد API‌ها همواره یکی از چالش‌های مهم در کارایی سیستم‌های ابری شناخته می‌شود. از جمله افزایش حجم درخواست‌ها، تغییرات پویا در الگوی تقاضای کاربران، نوسانات شبکه و سربار ارتباطی میان سرویس‌ها می‌تواند موجب افت محسوس در کیفیت خدمات شود. این مشکلات هم بر تجربه کاربران تأثیر می‌گذارند، هم می‌توانند موجب افزایش هزینه‌های عملیاتی، نارضایتی مشتریان و کاهش اعتماد به سرویس‌های ابری شوند؛ لذا، مدیریت هوشمند عملکرد API‌ها و بهینه‌سازی منابع محاسباتی در محیط‌های ابری، به یکی از موضوعات مهم و حیاتی در پژوهش‌های علمی و صنعتی بدل شده است.

در تحقیقات اخیر، محققان برای پاسخ به این چالش‌ها، از ابزارهای مختلفی استفاده کردند. یکی از مهم‌ترین آن‌ها، استفاده از هوش مصنوعی و یادگیری ماشین^۴ برای پیش‌بینی و بهینه‌سازی سیستم‌های ابری است. در بین روش‌های یادگیری ماشین هم دو گروه از الگوریتم‌ها در این زمینه اهمیت بالایی پیدا کردند: یادگیری تقویتی^۵ (RL) و شبکه‌های عصبی بازگشتی با حافظه کوتاه‌مدت^۶ (LSTM).

الگوریتم‌های یادگیری تقویتی با الهام از فرآیند یادگیری انسانی، از طریق تعامل مداوم عامل^۷ با محیط و دریافت پاداش یا تنبیه بر اساس نتایج تصمیمات خود، به تدریج سیاست بهینه‌ای برای انتخاب عمل در موقعیت‌های مختلف می‌آموزند. از طرف دیگر، شبکه‌های LSTM به دلیل توانایی بالا در مدل‌سازی وابستگی‌های زمانی و تحلیل داده‌های متوالی، ابزار قدرتمندی برای پیش‌بینی روندهای بار کاری API‌ها محسوب می‌شوند. ترکیب این دو، یعنی پیش‌بینی بار ترافیکی با LSTM و تصمیم‌گیری بهینه با RL، مدلی هوشمند برای بهبود عملکرد API‌ها فراهم

¹ Cloud Computing

² Microservices Architecture

³ Application Programming Interface (API)

⁴ Machine Learning

⁵ Reinforcement Learning

⁶ Long Short-Term Memory

⁷ Agent

می‌کند که قادر است باوجود تغییرات پویا، واکنشی سریع، دقیق و کارآمد داشته باشد. در محیط‌های میکروسرویسی هر سرویس معمولاً در قالب یک کانتینر به‌صورت مستقل اجرا می‌شود و مجموعه‌ای از کانتینرها توسط ابزارهای ارکستریشن^۱ مانند Kubernetes مدیریت می‌شوند. Kubernetes مسئول توزیع خودکار بار، مقیاس‌پذیری پویا و نظارت بر سلامت سرویس‌ها است. اما سازوکارهای پیش‌فرض Kubernetes برای مقیاس‌پذیری معمولاً بر اساس آستانه‌های ایستا طراحی شده‌اند که توانایی تطبیق سریع با الگوهای پیچیده‌ی ترافیکی را ندارند. این موضوع باعث می‌شود که سیستم در شرایط پرتراфик یا کم‌بار، تصمیمات بهینه‌ای نگیرد؛ لذا ترکیب الگوریتم‌های یادگیری تقویتی و شبکه‌های LSTM با سازوکار Kubernetes می‌تواند باعث مدیریت پویا و هوشمند منابع گردد. شبکه LSTM با پیش‌بینی بار آینده، اطلاعات لازم را به عامل RL ارائه می‌دهد، و عامل RL بر اساس این داده‌ها، تصمیماتی درباره افزایش یا کاهش تعداد کانتینرهای API می‌گیرد. هدف این پژوهش، طراحی و ارزیابی یک مدل ترکیبی مبتنی بر الگوریتم یادگیری تقویتی و شبکه عصبی LSTM در بستر میکروسرویس‌ها است. این مدل با تکیه بر قابلیت یادگیری تطبیقی RL و توان پیش‌بینی LSTM، تلاش می‌کند تا ضمن بهبود سرعت پاسخ‌دهی، پایداری و مقیاس‌پذیری API‌ها، بهره‌وری منابع محاسباتی را نیز افزایش دهد. در نهایت، خواستار ارائه یک مدل جامع برای بهبود عملکرد API در محیط رایانش ابری هستیم آن‌هم از طریق ادغام هوشمند الگوریتم‌های یادگیری تقویتی و شبکه‌های LSTM.

۱-۲- بیان مسئله

رایانش ابری به‌عنوان یکی از تحولات بنیادین فناوری اطلاعات در دهه‌های اخیر، نحوه دسترسی و بهره‌برداری از منابع محاسباتی را متحول کرده است. طبق تعریف مؤسسه ملی استانداردها و فناوری ایالات متحده (NIST)^۲، رایانش ابری، مدلی برای دسترسی آسان، فراگیر و بر پایه تقاضا به منابع محاسباتی قابل‌بیکربندی مانند شبکه‌ها، سرورها، فضای ذخیره‌سازی، برنامه‌ها و سرویس‌ها از طریق اینترنت است که با حداقل نیاز به مدیریت مستقیم توسط کاربر انجام می‌پذیرد. این فناوری، با ارائه مدل‌های سرویس‌دهی متنوع نظیر زیرساخت به‌عنوان سرویس (IaaS)، پلتفرم به‌عنوان سرویس (PaaS) و نرم‌افزار به‌عنوان سرویس (SaaS)، سازمان‌ها را قادر ساخته تا بدون سرمایه‌گذاری‌های سنگین زیرساختی، سرویس‌های سریع، مقیاس‌پذیر و مقرون‌به‌صرفه ارائه دهند.

سازمان‌ها و کسب‌وکارها با مهاجرت به سمت ابر، نه‌تنها در پی کاهش هزینه‌های سخت‌افزاری و نگهداری هستند، بلکه به دنبال بهره‌گیری از مزایایی چون مقیاس‌پذیری، دردسترس بودن بالا و عملکرد سریع‌تر نیز هستند. در این میان، رابط‌های برنامه‌نویسی کاربردی (API) به‌عنوان مؤلفه‌هایی حیاتی در اتصال میان سرویس‌ها در مدل یک سیستم میکروسرویس محور، نقش کلیدی در عملکرد سیستم‌های ابری ایفا می‌کنند. با رشد معماری‌های ماژولار و توزیع‌شده، حجم تعاملات میان سرویس‌ها از طریق API‌ها افزایش چشمگیری یافته و همین امر فشار قابل‌توجهی بر منابع ابری وارد کرده است. یکی از مهم‌ترین چالش‌ها در این زمینه، حفظ کیفیت سرویس (QoS) در زمان افزایش بار و درخواست‌های هم‌زمان است. اغلب API‌ها در شرایط پرتراфик دچار افت کارایی، تأخیر در پاسخ‌دهی، و حتی ازدست‌رفتن درخواست‌ها می‌شوند. این موضوع، نه‌تنها تجربه کاربری را تضعیف می‌کند، بلکه می‌تواند

¹ Orchestration

² National Institute of Standards and Technology (NIST)

امنیت، پایداری و شهرت کسب‌وکارها را نیز تحت‌تأثیر قرار دهد، با گسترش روزافزون فناوری‌های دیجیتال و مهاجرت کسب‌وکارها به سمت زیرساخت‌های ابری، رایانش ابری یکی از ستون‌های اصلی معماری‌های مدرن IT تبدیل شده است، بهبود عملکرد API‌ها به یک نیاز حیاتی در چارچوب سیستم میکروسرویس محور ابری بدل شده است. در این زمینه، الگوریتم‌های یادگیری ماشین به‌ویژه در حوزه‌هایی چون پیش‌بینی بار ترافیکی، تخصیص منابع، تشخیص ناهنجاری و سیستم‌ها، بسیار مؤثر عمل کرده‌اند. مطالعات اخیر نشان می‌دهند که استفاده از مدل‌هایی مانند LSTM (برای پیش‌بینی سری‌های زمانی) و Random Forest (برای تحلیل چندعاملی) می‌تواند دقت تصمیم‌گیری سیستم‌های ابری را به طور چشمگیری افزایش دهد. [۱]

از سوی دیگر، Kubernetes به‌عنوان ابزار مدیریت اورکستریشن میکروسرویس‌ها، توانایی فوق‌العاده‌ای در مقیاس‌پذیری خودکار و مدیریت بار پویا دارد. اما چالش اساسی در اینجا، نحوه ادغام هوشمند یادگیری ماشین با سیستم‌های ارکستریشن مانند Kubernetes جهت پیش‌بینی بار و مقیاس‌پذیری خودکار پوسته‌های API است؛ امری که کمتر در تحقیقات کاربردی و پیاده‌سازی‌های صنعتی به‌صورت جامع بررسی شده است. [۲]

مطالعات پیشین مانند مقالاتی که منتشر شده در (Elsevier و IEEE)، هرچند به بررسی جنبه‌هایی از بهبود عملکرد API در محیط ابری پرداخته‌اند، اما عمدتاً تمرکزشان بر روی یکی از ابعاد مثل بهبود توان عملیاتی با ساخت‌افزارهای خاص یا پیش‌بینی عملکرد API با روش‌های آماری بوده است. درعین‌حال، تحلیل یکپارچه مواردی چون پیش‌بینی هوشمند بار، مدیریت پویا و مقیاس‌پذیری با در نظر گرفتن معیارهایی مانند مصرف CPU، حافظه، تأخیر پاسخ و نرخ موفقیت درخواست‌ها در سطح کلان، کمتر مورد توجه بوده است. [۳]

همچنین، کمتر پژوهشی به‌صورت جامع به مقایسه قبل و بعد از اعمال الگوریتم‌های ML در یک محیط شبیه‌سازی واقعی پرداخته است؛ محیطی که ترکیبی از Docker، Kubernetes، Prometheus، Grafana و ابزارهای تولید ترافیک مانند JMeter و Locust باشد. با توجه به این خلأ تحقیقاتی، در این پژوهش تلاش می‌شود تا با طراحی و پیاده‌سازی یک مدل هوشمند ترکیبی، عملکرد API‌ها در محیط رایانش ابری بهبود یابد. این مدل با استفاده از پیش‌بینی بار از طریق الگوریتم‌های یادگیری ماشین، و اعمال مقیاس‌پذیری پویا توسط Kubernetes، سعی در کاهش تأخیر، افزایش بهره‌وری منابع و بهبود تجربه کاربری خواهد داشت. این رویکرد نه تنها از لحاظ نظری نوآورانه است، بلکه از جنبه کاربردی نیز می‌تواند راه‌حلی مؤثر برای چالش‌های فعلی در مدیریت API‌های ابری برای صنایع بزرگ، از جمله فین‌تک، تجارت الکترونیک و ارائه‌دهندگان سرویس‌های ابری باشد. [۴]

در این پژوهش، هدف طراحی API جدید نیست، بلکه استفاده از الگوریتم‌های یادگیری ماشین جهت تحلیل رفتار و پیش‌بینی بار کاری API‌های موجود است. این الگوریتم‌ها به‌عنوان یک مؤلفه مستقل در کنار سایر سرویس‌ها اجرا شده و با ارائه تصمیم‌های تخصیص منابع به ارکستراسیون، عملکرد کلی API‌ها در زمان اجرا بهبود می‌یابد.

۱-۳- جنبه نوآوری تحقیق

۱. طراحی یک مدل هوشمند پیش‌بینانه برای مدیریت مصرف انرژی. در این پژوهش، یک مدل ترکیبی

مبتنی بر شبکه LSTM و منطق تصمیم‌گیری الهام‌گرفته از یادگیری تقویتی (RL-Inspired Decision)

Logic ارائه شده است. رویکرد این مدل به گونه‌ای هست که گره‌هایی هیچ بار کاری ندارند به صورت هوشمند غیرفعال شده و منابع پرمصرف تنها در زمان نیاز فعال می‌شوند.

۲. **تفکیک صریح وظایف پیش‌بینی و تصمیم‌گیری در معماری مدل پیشنهادی.** برخلاف بسیاری از مطالعات پیشین که پیش‌بینی و تصمیم‌گیری را در یک مدل واحد ادغام می‌کنند، در این پژوهش فرآیند پیش‌بینی بار (LSTM) و فرآیند تصمیم‌گیری تخصیص منابع (Random Forest و منطق مقیاس‌پذیری) به صورت ماژولار و مستقل طراحی شده‌اند. این تفکیک، انعطاف‌پذیری مدل، قابلیت توسعه و امکان جایگزینی الگوریتم‌ها را افزایش می‌دهد.

۳. **ارائه یک مکانیسم تصمیم‌گیری ترکیبی مبتنی بر پیش‌بینی چندمدلی^۱.** در مدل پیشنهادی، تصمیم نهایی مقیاس‌پذیری بر اساس ترکیب خروجی دو مدل پیش‌بینی‌کننده مستقل (LSTM و Random Forest) اتخاذ می‌شود. این رویکرد باعث کاهش تأخیر پیش‌بینی، افزایش پایداری تصمیمات و جلوگیری از نوسانات ناخواسته در فرآیند مقیاس‌پذیری می‌گردد که در بسیاری از پژوهش‌های پیشین کمتر مورد توجه قرار گرفته است.

۴. **ادغام الگوریتم‌های عملیاتی و غیرتهاجمی مدل هوشمند با چارچوب Kubernetes.** برای اجرای مقیاس‌پذیری خودکار در محیط‌های کانتینری واقعی و چند سرویسی از نوآوری‌های کلیدی این پژوهش است، به اطلاع می‌رساند ارائه روشی غیرتهاجمی برای اعمال تصمیمات هوشمند مقیاس‌پذیری در Kubernetes به گونه‌ای است که بدون نیاز به تغییر در کد API‌ها یا معماری سرویس‌ها، تصمیمات حاصل از مدل یادگیری ماشین از طریق Kubernetes API Server به فرآیند مقیاس‌پذیری خودکار منتقل می‌شوند.

۵. **ارزیابی دقیق عملکرد API Gateway ها (نظیر Tyk یا kong) با بهره‌گیری از ابزارهای مانیتورینگ بلادرنگ مانند Prometheus و Grafana** یک حلقه بازخورد داده‌محور برای یادگیری و تطبیق پویا مورد استفاده قرار می‌گیرد. نتایج واقعی اجرای تصمیمات مقیاس‌پذیری مجدداً به مدل بازگردانده می‌شوند. این سازوکار، امکان به‌روزرسانی تدریجی مدل‌ها و تطبیق آن‌ها با الگوهای جدید بار کاری را فراهم کرده و سیستم را به یک مدل نیمه خودآموز تبدیل می‌کند.

۶. **ارائه یک مدل ترکیبی هوشمند برای پیش‌بینی و مدیریت بلادرنگ بار API با استفاده از الگوریتم یادگیری تقویتی (RL) و شبکه عصبی LSTM، باهدف بهبود کیفیت سرویس (QoS) در معماری میکروسرویس محور برخلاف بسیاری از مطالعات که تنها بر یک یا دو شاخص مانند تأخیر پاسخ تمرکز دارند، در این پژوهش عملکرد مدل پیشنهادی به صورت هم‌زمان تأخیر، نرخ موفقیت درخواست‌ها و توان عملیاتی، و همچنین از منظر بهره‌وری منابع شامل مصرف CPU و حافظه مورد ارزیابی قرار گرفته است.**

۷. **طراحی یک مدل هوشمند پیش‌بینانه برای مدیریت هزینه و امکان پیاده‌سازی و بازتولید مدل در محیط‌های سبک و شبیه‌سازی‌شده.** با جلوگیری از تخصیص غیرضروری منابع، هزینه‌های عملیاتی

¹ Ensemble Prediction

را به طور قابل توجهی کاهش می دهد. استفاده همزمان از پیش بینی بار و تصمیم گیری RL موجب می شود منابع دقیقاً متناسب با نیاز واقعی سیستم مصرف شوند. یکی دیگر از جنبه های نوآورانه این تحقیق، طراحی مدل به گونه ای است که بدون نیاز به زیرساخت ابری واقعی، قابلیت پیاده سازی، آزمایش و بازتولید نتایج در یک محیط سبک مبتنی بر شبیه سازی داده ها را دارد؛ امری که بازتولیدپذیری پژوهش و امکان توسعه آن در محیط های واقعی Docker و Kubernetes را تسهیل می کند.

۱-۴- اهداف تحقیق

۱- اهداف علمی

اهداف علمی این تحقیق به دو صورت هدف اصلی و فرعی بیان می شود:

هدف اصلی:

- هدف اصلی این تحقیق، بهبود عملکرد و کارایی مؤلفه ی API به عنوان یکی از اجزای حیاتی در معماری سیستم های ابری و میکروسرویس محور است. در این پژوهش، تمرکز بر طراحی یک معماری جدید نیست، بلکه تلاش می شود با به کارگیری الگوریتم های هوشمند مبتنی بر یادگیری ماشین، کارایی این مؤلفه ارتقاء یابد. بدین منظور، یک مدل ترکیبی مبتنی بر شبکه LSTM برای پیش بینی بار ترافیکی API و مکانیزم تصمیم گیری هوشمند برای توزیع و تخصیص منابع ارائه می شود تا کیفیت سرویس و پایداری سیستم در شرایط بار متغیر بهبود یابد.

اهداف فرعی:

- کاهش تأخیر پاسخ^۱ و افزایش نرخ موفقیت درخواست های API در شرایط نوسان شدید بار کاری.
- بهبود بهره وری منابع محاسباتی از طریق تخصیص پویا و هوشمند منابع CPU و حافظه در محیط Kubernetes.
- افزایش پایداری و قابلیت تطبیق پذیری سیستم در مواجهه با تغییرات ناگهانی الگوی ترافیک با استفاده از پیش بینی بار مبتنی بر LSTM.
- کاهش تخصیص بیش از حد یا کمبود منابع (Over/Under Provisioning) در فرآیند مقیاس پذیری خودکار سرویس های API.
- افزایش کیفیت سرویس (QoS) با در نظر گرفتن همزمان شاخص هایی نظیر تأخیر، توان عملیاتی^۲ و نرخ خطا.
- ارائه یک مدل عملی و قابل پیاده سازی برای شرکت های ارائه دهنده خدمات ابری، فین تک، تجارت الکترونیک و سازمان های دولتی با بار کاری بالا.

^۱ Response Time

^۲ Throughput

- توسعه دانش علمی و کاربردی در زمینه ادغام الگوریتم‌های یادگیری ماشین با سیستم‌های اركستراسیون ابری و معماری‌های API محور.

۱-۵- فرضیه‌های تحقیق

فرضیه‌های تحقیق به شرح زیر است:

- مدل ترکیبی یادگیری ماشین و یادگیری تقویتی، عملکرد بهتری در پیش‌بینی بار API‌ها نسبت به روش‌های سنتی دارد.
- ادغام الگوریتم‌های ML/RL با چارچوب Kubernetes موجب کاهش تأخیر و افزایش نرخ موفقیت پاسخ‌گویی API‌ها می‌شود.
- استفاده از سیستم ارزیابی خودکار API‌ها (مانند AGE، Prometheus و Grafana) موجب بهبود تصمیم‌گیری در تخصیص منابع و افزایش کیفیت سرویس (QoS) در محیط ابری می‌شود.

۱-۶- اهمیت و ضرورت انجام تحقیق

در عصر حاضر، رشد سریع سرویس‌های آنلاین، توسعه سیستم‌های توزیع‌شده، و افزایش تعاملات بین سرویس، منجر به وابستگی بیش‌ازپیش زیرساخت‌های نرم‌افزاری به API‌ها (رابط‌های برنامه‌نویسی کاربردی) شده است. در این میان، یک سیستم میکروسرویس محور مبتنی بر رایانش ابری، به دلیل ویژگی‌هایی مانند مقیاس‌پذیری، چابکی، استقلال سرویس‌ها، به یکی از الگوهای مسلط در طراحی سامانه‌های مدرن تبدیل شده‌اند. بااین‌حال، در مواجهه با حجم انبوه درخواست‌های API و رفتار غیرقابل‌پیش‌بینی کاربران، بهبود عملکرد API به یک دغدغه جدی برای سازمان‌ها و توسعه‌دهندگان تبدیل شده است.

یکی از بزرگ‌ترین چالش‌ها در این حوزه، نحوه مدیریت هوشمند منابع پردازشی و پاسخ به نیازهای پویا و متغیر کاربران است. افزایش تقاضا در لحظات خاص، نوسانات ترافیکی و نیاز به پاسخ‌دهی سریع و دقیق، مستلزم بهره‌گیری از روش‌هایی است که بتوانند رفتار سیستم را پیش‌بینی و به‌صورت خودکار واکنش مناسب نشان دهند. در این راستا، یادگیری ماشین به‌عنوان یک ابزار تحول‌آفرین، ظرفیت آن را دارد تا از داده‌های قبلی بیاموزد و رفتار آینده سیستم را پیش‌بینی کند. هم‌زمان، ابزارهایی نظیر Kubernetes برای اورکستریشن و مقیاس‌پذیری سرویس، بستر مناسبی را فراهم می‌کنند که بتوان تصمیمات یادگیری ماشین را به مرحله اجرا درآورد.

هرچند در سال‌های اخیر مطالعاتی در زمینه بهبود عملکرد سیستم‌های ابری انجام شده است، اما بخش عمده‌ای از این تحقیقات به یکی از مؤلفه‌ها پرداخته‌اند:

برخی صرفاً روی سخت‌افزار و افزایش توان عملیاتی متمرکز بوده‌اند، برخی دیگر تنها الگوهای پیش‌بینی بار ترافیکی را بررسی و برخی نیز چالش‌های امنیتی و پایداری در سطح زیرساخت را مدنظر قرار داده‌اند. اما تلفیق هوشمندانه مدل‌های یادگیری ماشین با سازوکارهای اركستریشن مانند Kubernetes برای مدیریت هوشمند ترافیک API، از جمله موضوعاتی است که هنوز جای خالی آن در تحقیقات کاربردی و صنعتی حس می‌شود.

ازسوی دیگر، مصرف بهینه منابع محاسباتی (مثل CPU و RAM)، کاهش تأخیر در پاسخ‌دهی، و افزایش نرخ موفقیت درخواست‌ها، مستقیماً بر رضایت کاربران نهایی، هزینه‌های عملیاتی و کیفیت سرویس‌دهی اثرگذار است. شرکت‌هایی مانند Amazon، Google و Microsoft که بزرگ‌ترین ارائه‌دهندگان سرویس‌های ابری هستند، همواره

به دنبال راهکارهایی برای افزایش بهره‌وری سیستم‌هایشان در مقیاس بالا هستند. همچنین در صنایع فین‌تک، بانکی، تجارت الکترونیک و پلتفرم‌های بزرگ، تأخیر در پاسخ API‌ها می‌تواند به زیان‌های مالی جدی یا نارضایتی مشتریان منجر شود.

در نتیجه باتوجه به وابستگی روزافزون سامانه‌های ابری به API‌ها و چالش‌های موجود در مدیریت هوشمند منابع، ارائه یک مدل ترکیبی مبتنی بر یادگیری ماشین و ارکستریشن، پاسخی ضروری به نیازهای عملکردی و مقیاس‌پذیری در زیرساخت‌های مدرن ابری است. این تحقیق باهدف پر کردن خلأ موجود، می‌تواند گامی مؤثر در ارتقای کیفیت سرویس‌ها و کاهش هزینه‌های عملیاتی در سازمان‌های داده‌محور باشد.

از منظر ماهیت پژوهش، این تحقیق در زمره پژوهش‌های کاربردی - توسعه‌ای و مدل‌محور قرار می‌گیرد. تمرکز اصلی کار، نه بر طراحی یک معماری ابری جدید، بلکه بر ارائه و توسعه یک مدل هوشمند قابل‌پیاده‌سازی برای بهبود عملکرد یکی از مؤلفه‌های کلیدی سیستم‌های ابری، یعنی API، در چارچوب معماری‌های موجود است.

در این پژوهش، با تلفیق روش‌های یادگیری ماشین (شبکه LSTM برای پیش‌بینی بار کاری و الگوریتم تصمیم‌گیری هوشمند برای تخصیص منابع) و چارچوب ارکستراسیون Kubernetes، یک مدل عملیاتی و اجرایی ارائه می‌شود که قابلیت پیاده‌سازی در محیط‌های واقعی و صنعتی را دارد.

از این‌رو، این تحقیق را می‌توان در دسته تحقیقات مبتنی بر مدل و سیستم طبقه‌بندی کرد که علاوه بر توسعه دانش نظری در حوزه مدیریت هوشمند منابع ابری، بر ارزیابی تجربی، تحلیل عملکرد و سنجش شاخص‌های کیفیت سرویس (QoS) در سناریوهای واقعی بار متغیر نیز تأکید دارد. بدین ترتیب، نتایج پژوهش از جنبه علمی و از منظر کاربرد صنعتی، دارای اهمیت و ضرورت است.

۷-۱- متغیرهای تحقیق

متغیرهای این تحقیق به شرح زیر است:

۱) متغیرهای مستقل^۱:

مواردی هستند که بر عملکرد API و شاخص‌های کیفیت سرویس تأثیر مستقیم دارند و در این پژوهش برای مدل یادگیری (LSTM + RL) به‌عنوان ورودی عمل می‌کنند:

- ترافیک API (تعداد درخواست‌ها در ثانیه): مهم‌ترین شاخص برای سنجش بار کاری سیستم است. این متغیر رابطه مستقیمی با تأخیر، مصرف منابع و نرخ موفقیت دارد.
- استفاده از CPU (%): شاخص کلیدی در مدیریت منابع محاسباتی در محیط ابری است. مدل می‌تواند بر اساس سطح مصرف CPU تصمیم بگیرد که مقیاس‌پذیری انجام دهد یا خیر.
- حافظه مصرفی (RAM): در سیستم‌های میکروسرویس محور، افزایش مصرف حافظه معمولاً نشانه افزایش هم‌زمانی درخواست‌ها یا گلوگاه در عملکرد سرویس است.
- تعداد کاربران فعال: این متغیر به طور غیرمستقیم بار سیستم را تعیین می‌کند و برای پیش‌بینی دقیق بار آینده مفید است.

¹ Input Variables

۲) متغیرهای وابسته:

این متغیرها همان معیارهایی هستند که به وسیله مدل پیشنهادی بهبود می یابند:

- میانگین تأخیر پاسخ API (برحسب میلی ثانیه): شاخص اصلی کیفیت سرویس که کاهش آن مستقیماً نشان دهنده بهبود عملکرد API است.
- نرخ موفقیت پاسخ های API (درصد پاسخ های موفق به کل درخواست ها): نسبت پاسخ های موفق به کل درخواست ها است. افزایش این نرخ به معنای پایداری بیشتر سیستم است.
- میزان مصرف منابع پردازشی (CPU و RAM): این شاخص می تواند ترکیبی از مصرف CPU و RAM باشد که در خروجی مدل بهینه می شود تا از تخصیص بیش از حد منابع جلوگیری شود.

جدول (۱-۱) متغیرهای تحقیق

عنوان متغیر	نقش	نوع	نحوه اندازه گیری	مقیاس
ترافیک API	مستقل	گسسته	داده های جمع آوری شده از Prometheus و Logstash	نسبی (عدد طبیعی)
استفاده از CPU	مستقل	گسسته	گزارش Prometheus در بازه های زمانی مشخص	نسبی (۰-۱۰۰٪)
حافظه مصرفی (RAM)	مستقل	پیوسته	اندازه گیری مصرف حافظه از طریق Prometheus (برحسب MB)	نسبی (۰-۱)
تعداد کاربران فعال	مستقل	پیوسته	شمارش کاربران فعال در هر بازه زمانی بر اساس لاگ ها	نسبی (۰-۱)
میانگین تأخیر پاسخ API	وابسته	پیوسته	محاسبه میانگین زمان پاسخ درخواست ها در Prometheus	نسبی (۰-۱)
نرخ موفقیت پاسخ های API	وابسته	پیوسته	تعداد پاسخ های موفق (HTTP 200/201) تقسیم بر کل درخواست ها $100 \times$	نسبی (۰-۱)
میزان مصرف منابع پردازشی	وابسته	پیوسته	ترکیب نرمال شده مصرف CPU و RAM برای هر کانتینر در بازه زمانی مشخص	نسبی (۰-۱)

جدول ۱

۸-۱- روش کار و روش شناسی تحقیق

- پژوهش حاضر از نوع کاربردی - توسعه ای است؛ زیرا هدف اصلی آن ارائه یک مدل جدید برای بهبود عملکرد رابط های برنامه نویسی کاربردی (API) در محیط های رایانش ابری و ارزیابی عملی کارایی آن در شرایط واقعی است.
- در مرحله نخست، ما از مطالعات کتابخانه ای و بررسی منابع علمی معتبر شامل مقالات منتشر شده در پایگاه هایی نظیر Elsevier، Springer، ACM Digital Library، استفاده خواهیم کرد.
 - در مرحله دوم، بر اساس نتایج تحلیل پیشینه و نیازهای محیط ابری، مدل پیشنهادی ترکیبی مبتنی بر الگوریتم یادگیری تقویتی و شبکه عصبی LSTM طراحی شده است.

¹ Output / Performance Metrics

- در مرحله سوم، مدل طراحی شده در محیطی مبتنی بر Docker و Kubernetes پیاده‌سازی می‌شود تا عملکرد آن در یک سناریوی واقعی مورد ارزیابی قرار گیرد.
- در مرحله چهارم، داده‌های حاصل از اجرای آزمایش‌ها پردازش و با روش‌های آماری تحلیل می‌شوند. شاخص‌هایی همچون میانگین تأخیر پاسخ، نرخ موفقیت درخواست‌ها، مصرف منابع، مقیاس‌پذیری و کیفیت سرویس. سپس نتایج به دست آمده با روش‌های مرجع مقایسه می‌شود تا میزان بهبود عملکرد روش پیشنهادی مشخص گردد.
- در نهایت، با جمع‌بندی نتایج آزمایش‌ها و تحلیل‌های انجام شده، نقاط قوت، محدودیت‌ها و پیشنهادهای توسعه آینده مدل را ارائه خواهیم داد.

۱-۹- مروری بر ساختار پایان‌نامه

پایان‌نامه در پنج فصل اصلی تنظیم شده که در ادامه به معرفی محتوای هر فصل پرداخته می‌شود:

- فصل اول: کلیات تحقیق. این فصل به معرفی کلی موضوع پژوهش، بیان مسئله، اهمیت و ضرورت تحقیق، اهداف علمی و کاربردی و روش انجام تحقیق اختصاص دارد.
- فصل دوم: مروری بر ادبیات و پیشینه تحقیق. در این فصل، مفاهیم پایه مرتبط با رایانش ابری، معماری میکروسرویس‌ها، عملکرد API‌ها، اصول یادگیری تقویتی و شبکه‌های عصبی LSTM به صورت جامع بررسی می‌شود. همچنین، تحقیقات داخلی و خارجی انجام شده بیان می‌شود.
- فصل سوم: روش اجرای تحقیق. این فصل به تشریح کامل مدل پیشنهادی می‌پردازد.
- فصل چهارم: تجزیه و تحلیل داده‌ها. در این فصل، مراحل پیاده‌سازی مدل در محیط عملیاتی تشریح می‌شود. سپس داده‌های حاصل از آزمایش‌ها تحلیل شده و نتایج عملکرد مدل پیشنهادی با روش‌های موجود مقایسه می‌شود.
- فصل پنجم: نتیجه‌گیری و پیشنهادات. در این فصل اهداف تحقیق بررسی و نتایج کلیدی به صورت خلاصه بیان می‌شود. همچنین محدودیت‌های مطالعه و پیشنهادهایی برای تحقیقات آینده ارائه خواهد شد.

فصل دوم

مروری بر ادبیات و پیشینه تحقیق

۲-۱- مقدمه

فصل دوم این پایان‌نامه به بررسی مبانی نظری و مرور پیشینه پژوهش اختصاص دارد. هدف این فصل، مطالعه مقالات، تحقیقات و فناوری‌های مرتبط برای طراحی مدل پیشنهادی است؛ لذا به معرفی و تحلیل مفاهیم کلیدی مورد استفاده در تحقیق حاضر شامل رایانش ابری، معماری میکروسرویس‌ها، رابط‌های برنامه‌نویسی کاربردی (API) و الگوریتم‌های یادگیری تقویتی و شبکه‌های عصبی LSTM خواهیم پرداخت. همچنین، در انتهای فصل مروری بر تحقیقات پیشین ارائه می‌کنیم تا شکاف‌های علمی موجود مشخص گردد.

۲-۲- ادبیات تحقیق

۲-۲-۱- رایانش ابری

در این بخش مفاهیم پایه‌ای رایانش ابری، مدل‌های خدماتی (IaaS، PaaS، SaaS)، مدل‌های استقرار (ابر خصوصی، عمومی، ترکیبی) و نقش مراکز داده بررسی می‌شود. سپس، به چالش‌های اصلی رایانش ابری از جمله مدیریت منابع، مقیاس‌پذیری، تضمین SLA و مصرف انرژی پرداخته می‌شود.

رایانش ابری به عنوان یکی از متحول‌ترین رویکردهای فناوری اطلاعات در دهه‌های اخیر شناخته می‌شود که بستری برای ارائه خدمات محاسباتی بر بستر اینترنت فراهم کرده و دسترسی بر حسب تقاضا به منابع محاسباتی مشترک همچون سرورها، حافظه، شبکه، پایگاه‌های داده و نرم‌افزارها را با کمترین نیاز به مدیریت مستقیم کاربران ممکن می‌سازد. این فناوری با بهره‌گیری از مجازی‌سازی و اتوماسیون منابع، امکان مقیاس‌پذیری بالا، استفاده بهینه از زیرساخت‌ها و کاهش هزینه‌های سرمایه‌گذاری اولیه را برای کاربران و ارائه‌دهندگان خدمات فراهم می‌کند. مدل‌های خدماتی رایانش ابری به سه دسته اصلی تقسیم می‌شوند:

- **زیرساخت به عنوان سرویس^۱ (IaaS):** ارائه دسترسی به منابع سخت‌افزاری مجازی‌سازی شده مانند ماشین‌های مجازی، فضای ذخیره‌سازی و شبکه.
- **بستر به عنوان سرویس^۲ (PaaS):** ارائه بستری برای توسعه، اجرا و مدیریت برنامه‌های کاربردی بدون نیاز به مدیریت مستقیم زیرساخت.
- **نرم‌افزار به عنوان سرویس^۳ (SaaS):** دسترسی به نرم‌افزارها از طریق اینترنت بدون نیاز به نصب یا نگهداری محلی.

رایانش ابری در قالب ابر خصوصی (اختصاصی برای یک سازمان)، عمومی (در دسترس برای عموم کاربران) و ترکیبی (ترکیب مزایای هر دو) قابل استفاده است. در قلب این فناوری، مراکز داده‌ای^۴ قرار دارند که منابع محاسباتی

^۱ on-demand

^۲ Infrastructure as a Service (IaaS)

^۳ Platform as a Service (PaaS)

^۴ Software as a Service (SaaS)

^۵ Data Centers

را به صورت مجازی مدیریت و خدمات ابری را پشتیبانی می کنند. ماشین های فیزیکی^۱ (PM) و ماشین های مجازی^۲ (VM) به عنوان عناصر اصلی این مراکز، نقش کلیدی در اجرای بارهای کاری ایفا می کنند.

با اینکه مزایای فراوانی دارد؛ اما با چالش هایی مواجه است که بخش قابل توجهی از پژوهش ها را به خود اختصاص داده، مهم ترین این چالش ها عبارتند از:

- مدیریت هوشمند منابع و بهبود تخصیص آن ها در شرایط پویا
- تضمین کیفیت خدمات^۳ (QoS) و رعایت توافق نامه های سطح خدمات^۴ (SLA)
- مقابله با ازدحام، تخصیص بیش از حد یا ناکافی منابع
- افزایش بهره وری انرژی و کاهش هزینه های عملیاتی

۲-۲-۲ معماری میکروسرویس^۵

معماری میکروسرویس یکی از رویکردهای نوین در طراحی و توسعه نرم افزارهای توزیع شده است که در سال های اخیر به ویژه با گسترش رایانش ابری و نیاز به مقیاس پذیری بالا، توجه پژوهشگران و صنعت را به خود جلب کرده است. در این معماری، یک سیستم نرم افزاری بزرگ به مجموعه ای از سرویس های کوچک تر، مستقل و خودمختار تقسیم می شود که هر سرویس مسئولیت انجام یک وظیفه یا دامنه مشخص از کسب و کار را برعهده دارد. در حقیقت، میکروسرویس ها امکان توسعه، استقرار و مقیاس گذاری مستقل اجزای یک سیستم را فراهم می کنند و این ویژگی به ویژه در محیط های ابری که نیازمند پاسخگویی سریع به تغییرات بار کاری هستند، اهمیت بالایی دارد. از ویژگی های اصلی میکروسرویس ها می توان به استقلال بالا، مقیاس پذیری منعطف و ارتباط سبک میان سرویس ها اشاره کرد. هر سرویس معمولاً پایگاه داده خود را دارد و بدون نیاز به آگاهی از جزئیات سایر سرویس ها عمل می کند. این استقلال منجر به کاهش پیچیدگی در توسعه و نگهداشت سیستم می شود و به تیم های توسعه امکان می دهد سرویس ها را با فناوری ها و زبان های مختلف پیاده سازی کنند. همچنین مقایسه معماری میکروسرویس ها با معماری یک پارچه نشان می دهد که این رویکرد جدید مزایای چشمگیری در نگهداری و توسعه سیستم ها دارد. از طرفی، معماری میکروسرویس با فراهم کردن امکان استقرار مستقل سرویس ها، تغییرات را آسان تر و کم هزینه تر می کند. مزایای میکروسرویس ها تنها به مقیاس پذیری محدود نمی شود. از دیگر نقاط قوت این معماری تحمل خطا^۶ است. در یک میکروسرویس، خرابی یک سرویس منجر به از کار افتادن کل سیستم نمی شود، زیرا سایر سرویس ها همچنان قادر به ادامه فعالیت خود هستند. این ویژگی اهمیت بالایی در سیستم های حساس و مبتنی بر ابر دارد که نیازمند دسترس پذیری بالا^۷ هستند. با وجود این مزایا، معماری میکروسرویس ها با چالش هایی نیز همراه است. یکی از مهم ترین چالش ها افزایش پیچیدگی مدیریت سرویس ها است. وقتی تعداد سرویس ها افزایش پیدا می کند، نیاز به

¹ Physical machines

² Virtual machines

³ Quality of Service

⁴ Service Level Agreements

⁵ Microservices Architecture

⁶ Fault Tolerance

⁷ High Availability

مکانیزم‌های پیشرفته برای کشف سرویس‌ها^۱، توازن بار^۲ و هماهنگی^۳ داریم. امنیت API نیز از دیگر مسائل مهم است؛ زیرا ارتباط بین سرویس‌ها عمدتاً از طریق API انجام می‌شود و این موضوع سطح حمله احتمالی را افزایش می‌دهد.

در جمع‌بندی می‌توان گفت معماری میکروسرویس‌ها به‌عنوان یک راهکار جدید و مؤثر برای توسعه سیستم‌های نرم‌افزاری در مقیاس بزرگ و محیط‌های پویا مانند رایانش ابری محسوب می‌شود. هرچند این معماری مزایای متعددی در زمینه مقیاس‌پذیری، تحمل خطا و انعطاف‌پذیری ارائه می‌دهد، اما چالش‌هایی همچون مدیریت پیچیدگی، امنیت و سربار ارتباطی را نیز به همراه دارد. از آنجاکه API‌ها محور اصلی ارتباط میان سرویس‌ها هستند، بهبود عملکرد آن‌ها می‌تواند تأثیر بسزایی بر کارایی و کیفیت کل سیستم داشته باشد. در همین راستا، پژوهش حاضر با کمک الگوریتم‌های یادگیری تقویتی و شبکه‌های عصبی LSTM در بستر میکروسرویس‌ها، خواستار ارتقا و بهبود عملکرد API‌ها در محیط رایانش ابری است.

۲-۲-۳- رابطه‌های برنامه‌نویسی کاربردی (APIs)

رابطه‌های برنامه‌نویسی کاربردی، یکی از مهم‌ترین عناصر معماری‌های نرم‌افزاری مدرن به شمار می‌روند. API مجموعه‌ای از پروتکل‌ها، توابع و ابزارهایی است که امکان برقراری ارتباط بین اجزای مختلف یک سیستم نرم‌افزاری یا بین سیستم‌های مجزا را فراهم می‌کند. در محیط‌های مبتنی بر میکروسرویس، API‌ها نقش ستون فقرات را ایفا می‌کنند؛ زیرا تمام سرویس‌ها از طریق آن‌ها با هم تعامل دارد و داده‌ها و عملیات را مبادله می‌کنند. در واقع بدون وجود API، ارتباط بین سرویس‌های مستقل که به‌صورت جداگانه توسعه و استقرار یافته‌اند، عملاً غیرممکن خواهد بود. در گذشته، اغلب API‌ها به‌صورت کتابخانه‌های محلی^۴ و درون یک برنامه تک پارچه استفاده می‌شدند. اما با گسترش اینترنت و معماری‌های توزیع‌شده، نیاز به API‌هایی که بتوانند ارتباطات شبکه‌ای را پشتیبانی کنند افزایش پیدا کرد. در این مسیر، فناوری‌هایی مثل SOAP^۵ و بعدها REST^۶ توسعه یافتند. RESTful API به دلیل سادگی، مقیاس‌پذیری بالا و سازگاری با پروتکل HTTP، به‌سرعت به استاندارد معماری‌های میکروسرویس و سیستم‌های ابری تبدیل شد. همچنین، در سال‌های اخیر فناوری gRPC نیز باهدف افزایش کارایی و کاهش سربار ارتباطی استفاده شده است. از مهم‌ترین چالش‌های موجود در زمینه API‌ها، کارایی و عملکرد آن‌ها است. در محیط‌های ابری و میکروسرویس، هر درخواست به API باید با حداقل تأخیر پردازش بشود، زیرا هر میلی‌ثانیه تأخیر می‌تواند در مقیاس بزرگ به کاهش کارایی کلی سیستم منجر شود. علاوه بر سرعت، امنیت نیز از اهمیت بالایی برخوردار است؛ زیرا API‌ها نقاط ورودی اصلی سیستم محسوب می‌شوند و حملاتی مثل DDoS، Injection و API Hijacking می‌تواند از این طریق رخ دهد. همچنین، مدیریت بار^۷ در عملکرد API‌ها اهمیت زیادی دارد. در معماری‌های ابری که بار کاری سیستم پویا و متغیر است، API‌ها باید بتوانند به‌صورت هوشمندانه منابع را تخصیص

^۱ Service Discovery

^۲ Load Balancing

^۳ Orchestration

^۴ Application Programming Interfaces (APIs)

^۵ Local Libraries

^۶ Simple Object Access Protocol

^۷ Representational State Transfer

^۸ Load Management

و از ایجاد گلوگاه^۱ جلوگیری کنند. همچنین در صورت خرابی یک سرویس یا تأخیر در پاسخگویی، API ها باید بتوانند با استفاده از الگوهایی مانند Circuit Breaker یا Fallback از انتشار خطا به کل سیستم جلوگیری کنند. در مجموع، می توان گفت که API ها نه تنها به عنوان واسط ارتباطی میان سرویس ها عمل می کنند، بلکه کیفیت و کارایی کل سیستم تا حد بالایی به عملکرد آن ها وابستگی دارد. بر همین اساس، بهبود کارایی API ها با استفاده از روش های هوشمند، از جمله الگوریتم های یادگیری تقویتی و مدل های پیش بینی مانند LSTM، می تواند به طور مستقیم در افزایش مقیاس پذیری، کاهش تأخیر و ارتقای کیفیت سرویس دهی در محیط های ابری مؤثر باشد.

۲-۲-۴- یادگیری ماشین^۲

الگوریتم ها برای محاسبه، پردازش و استدلال خودکار استفاده می شوند. در واقع، یک الگوریتم یک روش گام به گام برای محاسبه محسوب می شود [۲]. الگوریتم های یادگیری ماشین، در چند دسته عمده قابل تقسیم بندی هستند که در ادامه به آن ها پرداخته شده است.

یادگیری نظارت شده^۳: یادگیری نظارت شده، به دو دسته رده بندی و رگرسیون تقسیم می شود، از نقطه نظر کلی، دسته بندی و رگرسیون دو نوع اصلی از مسائل پیشگویی هستند که دسته بندی، جهت پیشگویی مقادیر گسسته و اسمی مورد استفاده قرار می گیرد، در حالی که رگرسیون جهت پیشگویی مقادیر پیوسته مورد استفاده قرار می گیرد. یادگیری نظارت شده وظیفه استنتاج یک تابع از برچسب داده های آموزشی را بر عهده دارد. داده های آموزشی از مجموعه ای از مثال های آموزشی تشکیل شده است. در یادگیری تحت نظارت، هر نمونه ورودی یک جفت متشکل از یک مقدار ورودی و یک مقدار خروجی است. یک الگوریتم یادگیری نظارت شده داده های آموزشی را تجزیه و تحلیل می کند و از آن یک تابع استنتاج می کند که می تواند برای نگاشت نمونه های جدید استفاده شود. یک تابع مطلوب، تابعی است که بتواند برای داده های تست، برچسب کلاس را به درستی مشخص کند. از جمله الگوریتم های رگرسیون می توان رگرسیون منطقی و خطی را نام برد و از جمله الگوریتم های دسته بندی مهم و پرکاربرد می توان به الگوریتم های درخت تصمیم و ماشین بردار پشتیبان و نزدیک ترین همسایه و غیره را نام برد [۳].

یادگیری بدون ناظر^۴: یادگیری بدون ناظر نیز، یک روش یادگیری ماشین است که تابعی ارائه می دهد که بتواند ساختار پنهان در داده های بدون برچسب را توصیف کند. از آنجاکه نمونه های داده شده به یادگیرنده بدون برچسب هستند، هیچ تنبیه یا پاداشی برای ارزیابی یک راه حل بالقوه وجود ندارد و این باعث تمایز این نوع یادگیری نسبت به یادگیری نظارت شده می باشد. از جمله این الگوریتم ها می توان به الگوریتم های خوشه بندی نظیر K-Mean، C-Mean و غیره را نام برد [۳].

یادگیری نیمه نظارت شده^۵: در این نوع یادگیری، معمولاً مقدار کمی از داده برچسب دار با مقدار زیادی از داده های بدون برچسب وجود دارند. یادگیری نیمه نظارتی بین آموزش بدون نظارت (بدون هیچ گونه داده های آموزشی برچسب دار) و یادگیری نظارت شده (با داده های آموزشی که به طور کامل برچسب دار هستند) قرار

¹ Bottleneck

² Machine Learning (ML)

³ Supervised learning

⁴ Unsupervised learning

⁵ Semi-supervised learning

می‌گیرد. بسیاری از محققان یادگیری ماشین به این نتیجه رسیده‌اند که داده‌های بدون برچسب، زمانی که در کنار مقدار کمی از داده‌های برچسب‌دار استفاده می‌شوند، می‌توانند بهبود قابل توجهی را در دقت یادگیری ایجاد کنند، از انواع روش‌های نیمه‌نظارتی می‌توان روش Inductive و روش Transductive را نام برد [۴].

یادگیری تقویتی^۱: یادگیری تقویتی، نوعی از یادگیری ماشین با الهام از روان‌شناسی رفتارگرایی است. در این یادگیری اقدامات لازم برای به حداکثر رسیدن ارزش تجمعی آموزش داده می‌شود. این نوع یادگیری، با توجه به کلیت آن، در بسیاری از رشته‌های دیگر، مانند نظریه بازی‌ها، تئوری کنترل، تحقیق در عملیات، تئوری اطلاعات، بهینه‌سازی مبتنی بر شبیه‌سازی، سیستم‌های چند عامله، هوش ازدحامی، آمار و الگوریتم‌های ژنتیکی مورد مطالعه قرار گرفته است [۵].

۲-۲-۵- یادگیری تقویتی

یادگیری تقویتی یکی از شاخه‌های اصلی و مهم یادگیری ماشین است که در سال‌های اخیر کاربردهای گسترده‌ای در حوزه‌های مختلف علوم کامپیوتر، مهندسی و سیستم‌های هوشمند پیدا کرده است. در این روش، یک عامل از طریق تعامل مداوم با محیط، اقدام به یادگیری می‌کند. اساس یادگیری تقویتی بر پایه مفهوم پاداش و سیاست است؛ به این معنا که عامل با انجام اقدامات مختلف و دریافت بازخورد از محیط، تلاش می‌کند سیاستی را بیاموزد که منجر به حداکثرسازی پاداش تجمعی در طول زمان شود. در یادگیری تقویتی، عامل نیازی به داده‌های از پیش برچسب‌گذاری شده ندارد، بلکه از طریق تجربه و آزمون و خطا به تدریج یاد می‌گیرد. این ویژگی، آن را از یادگیری نظارت شده و بدون نظارت متمایز می‌کند. چارچوب نظری یادگیری تقویتی معمولاً با استفاده از فرآیند تصمیم‌گیری مارکوف^۲ (MDP) مدل‌سازی می‌شود. در این مدل، در هر لحظه عامل در یک حالت مشخص قرار دارد، یک عمل انتخاب می‌کند و بر اساس آن، محیط به حالت جدیدی منتقل شده و یک پاداش ارائه می‌دهد. هدف عامل، یافتن سیاستی است که در بلندمدت بیشترین مجموع پاداش را کسب کند. یکی از الگوریتم‌های پایه در این حوزه، Q-Learning است که ارزش اقدامات مختلف در هر حالت را بر اساس تجربه عامل برآورد می‌کند. با ظهور یادگیری عمیق، روش‌های ترکیبی مثل DQN معرفی شدند که امکان استفاده از شبکه‌های عصبی برای تقریب توابع ارزش را فراهم می‌سازند. کاربرد یادگیری تقویتی در حوزه‌های مختلفی مانند رباتیک، بازی‌های رایانه‌ای، بهینه‌سازی شبکه‌های مخابراتی و مدیریت منابع رایانش ابری گسترش یافته است. در محیط‌های ابری، RL می‌تواند برای مسائلی نظیر زمان‌بندی وظایف^۳، تخصیص منابع، مقیاس‌گذاری پویا^۴ و بهینه‌سازی عملکرد APIها مورد استفاده قرار گیرد. مزیت اصلی این روش در توانایی آن برای سازگاری با محیط‌های پویا و نامطمئن است، زیرا عامل می‌تواند با مشاهده تغییرات در بار کاری یا شرایط شبکه، تصمیمات خود را به‌روز کند. در کنار همه این مزیت‌ها، یادگیری تقویتی چالش‌هایی نیز به همراه دارد. فرآیند یادگیری ممکن است زمان‌بر باشد و نیاز به آزمون و خطای فراوان داشته باشد. همچنین، تعریف دقیق پاداش و طراحی محیط شبیه‌سازی از اهمیت زیادی برخوردار است؛ زیرا انتخاب نامناسب پاداش می‌تواند عامل را به سمت رفتارهای نادرست هدایت کند. در پایان‌نامه حاضر،

¹ Reinforcement Learning

² Markov Decision Process

³ Task Scheduling

⁴ Auto-Scaling

استفاده از RL باهدف بهبود عملکرد APIها در بستر میکروسرویس‌ها پیشنهاد شده که عامل یادگیرنده می‌تواند با مشاهده وضعیت بار کاری، میزان تأخیر و پاسخ‌های سرویس‌ها، تصمیماتی بگیرد که به کاهش تأخیر، افزایش بهره‌وری منابع و ارتقای کیفیت سرویس‌دهی برسد.

۲-۲-۶- الگوریتم‌های رایج یادگیری ماشین

روش‌های زیادی برای دسته‌بندی وجود دارد که از جمله می‌توان به مواردی که در ادامه به آن‌ها اشاره می‌شود: درخت‌های تصمیم^۱، شبکه‌های بیزین، k نزدیک‌ترین همسایه^۲، ماشین بردار پشتیبان^۳، شبکه‌های عصبی مصنوعی^۴، شبکه عصبی شعاعی^۵، شبکه‌های عصبی پیچشی^۶ و...

۲-۲-۷- درخت‌های تصمیم

درخت‌های تصمیم از بالا به پایین یکی از الگوریتم‌های رایج دسته‌بندی هستند [۱۲]. از مهم‌ترین دلایل رایج بودن این الگوریتم شفافیت و قابلیت تفسیر بالای این الگوریتم است. مزیت دیگر موجود بودن پیاده‌سازی‌های قوی نظیر C4.5 است. الگوریتم‌های درخت‌های تصمیم با ساخت یک الگوریتم از بالا به پایین توسط انتخاب صفت در هر لحظه و جداسازی داده‌ها با توجه به مقادیر صفتشان انجام می‌شود مهم‌ترین صفت به‌عنوان ریشه درخت و بقیه گره‌ها نیز به ترتیب اولویت در سطح‌های بعدی قرار می‌گیرند به‌گونه‌ای که گره‌هایی که ضریب دستیابی اطلاعات و برچسب دسته را نشان می‌دهند نزدیک ریشه قرار می‌گیرند.

۲-۲-۸- شبکه‌های بیزین

در روش‌های دسته‌بندی آماری برخلاف سایر دسته‌بندی‌ها میزان عضویت یک نمونه به هر کلاس را با یک احتمال نشان می‌دهد. روش شبکه‌های بیزین رایج‌ترین روش دسته‌بندی آماری و از روش‌های ساده و مؤثر محسوب می‌شود. در این روش احتمال شرطی هر صفت داده شده را توسط برچسب دسته مربوطه از داده‌های آموزشی یاد می‌گیرید. سپس عمل دسته‌بندی توسط بکار بردن قوانین بیز برای محاسبه مقدار احتمالی دسته نتیجه نمونه داده شده با دقت بالایی انجام می‌شود. در حالت معمولی این کار با تخمین احتمالاتی هر ترکیب ممکن از صفات صورت می‌گیرد؛ ولی هنگامی که تعداد صفات خیلی زیاد باشد، این امر امکان‌پذیر نیست؛ بنابراین یک فرض مستقل قوی اتخاذ می‌شود که همه صفات با مشخص بودن مقدار صفت دسته مستقل هستند. با در نظر گرفتن این فرض لازم است که فقط احتمالات حاشیه‌ای هر صفت دسته محاسبه گردد. با این حال این فرض به‌صورت غیرواقعی می‌باشد و شبکه‌های بیزین با مدل کردن صریح، وابستگی بین صفات آن را در نظر نمی‌گیرند [۱۳].

۲-۲-۹- k نزدیک‌ترین همسایه

الگوریتم k نزدیک‌ترین همسایه مثالی از یادگیری بر اساس نمونه است که در آن مجموعه داده آموزشی برای ایجاد یک مدل دسته‌بندی مورد استفاده قرار می‌گیرند؛ بنابراین یک دسته‌بندی برای یک نمونه دسته‌بندی نشده ممکن

¹ decision trees

² k-nearest neighbors (K-NN)

³ Support Vector Machine (SVM)

⁴ Artificial Neural Networks (ANN)

⁵ Radial Basis Functions (RBF)

⁶ Convolutional Neural Network (CNN)

است به سادگی با مقایسه آن با شبیه‌ترین نمونه‌ها در مجموعه آموزشی یافت شود. روال این الگوریتم به این صورت است که برای هر نمونه جدید با مقایسه آن با k نمونه آموزشی نزدیک‌تر، دسته نتیجه را مشخص می‌کنیم [۱۵].

۲-۱۰-۲- ماشین بردار پشتیبان

ماشین بردار پشتیبان دسته‌بندی کننده‌ای است که جزو روش‌های بر پایه هسته^۱ در یادگیری ماشین محسوب می‌شود. SVM در سال ۱۹۹۲ توسط وپنیک^۲ معرفی شده و بر پایه نظریه آماری یادگیری بنا گردیده است. الگوریتم SVM یکی از الگوریتم‌های معروف در زمینه یادگیری با نظارت است که برای دسته‌بندی و رگرسیون استفاده می‌شود. این الگوریتم به‌طور هم‌زمان حاشیه‌های هندسی را بیشینه کرده و خطای تجربی دسته‌بندی را کمینه می‌کند؛ لذا به‌عنوان دسته‌بندی حداکثر حاشیه^۳ نیز نامیده می‌شود [۱۶]. برای یک مسئله دسته‌بندی با دو دسته نتیجه خطوط بی‌شماری ممکن است وجود داشته باشند که توسط آن‌ها دسته‌بندی انجام شود؛ ولی فقط یکی از این خطوط ماکزیمم تفکیک و جداسازی را فراهم می‌آورد. از بین جداسازهای خطی، آن جداسازی که حاشیه داده‌های آموزشی را حداکثر می‌کند خطای تعمیم را حداقل خواهد کرد. نقاط داده‌ای ممکن است ضرورتاً نقاط داده‌ای در فضای R^2 نباشند و ممکن است در فضای چندبعدی R^n مربوط باشند. دسته‌بندهای خطی بسیاری ممکن است این خصوصیت را ارضا کنند اما SVM به دنبال جداکننده‌ای است که حداکثر جداسازی را برای دسته‌ها انجام دهد.

۲-۱۱-۲- شبکه‌های عصبی مصنوعی

مطالعه شبکه‌های عصبی مصنوعی تا حد زیادی الهام‌گرفته از سیستم‌های یادگیر طبیعی است که در آن‌ها یک مجموعه پیچیده از نرون‌های به هم متصل در کار یادگیری دخیل هستند. این شبکه‌ها یادگیری را از روی مثال‌ها و نمونه‌ها انجام می‌دهند و از این لحاظ در عمل یادگیری شبیه به انسان عمل می‌کنند. مزیت دیگر آن‌ها این است که این شبکه‌ها از توانایی تعمیم‌دهی ذاتی برخوردار هستند؛ یعنی این شبکه‌ها توانایی تشخیص الگوهایی را که شبیه نمونه‌هایی که قبلاً یاد گرفته باشد را دارد نه اینکه تنها الگوهای دقیقاً همانند نمونه‌های آموزشی را تشخیص دهد [۸].

۲-۱۲-۲- شبکه عصبی شعاعی

شبکه عصبی شعاعی^۴ یک شبکه عصبی مصنوعی است که از توابع پایه‌ای شعاعی به‌عنوان توابع فعالیت استفاده می‌کند. خروجی این شبکه یک ترکیب خطی از توابع پایه شعاعی برای پارامترهای ورودی و نرون‌ها است. چنین شبکه‌هایی در تابع تقریب، پیش‌بینی سری‌های زمانی، کلاس‌بندی و کنترل سیستم مورد استفاده قرار می‌گیرند. شبکه‌های RBF معمولاً شامل سه لایه هستند: لایه ورودی، لایه مخفی با یک تابع فعالیت RBF غیرخطی و لایه خروجی. ورودی این شبکه می‌تواند به‌عنوان یک بردار از اعداد حقیقی مدل شود و خروجی این شبکه به تابع scalar از بردار ورودی است که به‌صورت زیر محاسبه می‌شود:

¹ Kernel Methods

² Vapnik

³ maximum margin

⁴ Radial Basis Functions (RBF)

$$\varphi(\mathbf{x}) = \sum_{i=1}^N a_i \rho(||\mathbf{x} - \mathbf{c}_i||)$$

فرمول ۱-۲:

در آن N تعداد نرون‌های لایه مخفی، $\mathbf{C}_i$ بردار مرکز نرون i و a_i وزن نرون i در نرون خروجی خطی است.

RBF معمولاً به‌وسیله یک الگوریتم دومرحله‌ای آموزش داده می‌شوند. در مرحله اول، بردارهای مرکز $\mathbf{C}_i$ برای توابع RBF در لایه مخفی انتخاب می‌شوند. این مرحله با چندین روش اجرا می‌گردد: مراکز به شکل تصادفی از برخی مجموعه مثال‌ها نمونه‌برداری می‌شود، یا از K-means clustering استفاده می‌شود. این مرحله بدون نظارت است. مرحله دوم به‌سادگی با یک مدل خطی با ضرایب w_i برای خروجی‌های لایه مخفی با توجه به تابع هدف، متناسب می‌شود.

۲-۲-۱۳- شبکه عصبی پیچشی

شبکه‌های عصبی پیچشی^۱ یا هم‌گشتی (CNN) رده‌ای از شبکه‌های عصبی ژرف هستند که معمولاً برای انجام تحلیل‌های تصویری یا گفتاری در یادگیری ماشین استفاده می‌شوند. شبکه‌های عصبی پیچشی به‌منظور کمینه کردن پیش‌پردازش‌ها از گونه‌ای از پرسپترونهای چندلایه استفاده می‌کنند. به‌جای شبکه عصبی پیچشی گاهی از این شبکه‌ها با نام شبکه‌های عصبی تغییرناپذیر با انتقال^۲ یا تغییرناپذیر با فضا^۳ هم یاد می‌شود. این نام‌گذاری بر مبنای ساختار این شبکه است. ساختار شبکه‌های پیچشی از فرایندهای زیستی قشر بینایی گربه الهام گرفته شده است. این ساختار به‌گونه‌ای است که تک‌نورون‌ها تنها در یک ناحیه محدود به تحریک پاسخ می‌دهند که به آن ناحیه پذیرش گفته می‌شود. نواحی پذیرش نورون‌های مختلف به‌صورت جزئی باهم همپوشانی دارند به‌گونه‌ای که کل میدان دید را پوشش می‌دهند. شبکه‌های عصبی پیچشی نسبت به بقیه رویکردهای دسته‌بندی تصاویر به میزان کمتری از پیش‌پردازش استفاده می‌کنند. این امر به معنی آن است که شبکه معیارهایی را فرامی‌گیرد که در رویکردهای قبلی به‌صورت دستی فراگرفته می‌شدند. این استقلال از دانش پیشین و دستکاری‌های انسانی در شبکه‌های عصبی پیچشی یک مزیت اساسی است.

۲-۲-۱۴- شبکه‌های عصبی بازگشتی با حافظه کوتاه‌مدت (LSTM)

LSTM در سال ۱۹۹۴، پژوهشی با عنوان "یادگیری وابستگی‌های طولانی با کاهش گرادیان سخت است" توسط Yoshua Bengio و دیگران معرفی شد. این پژوهش به معرفی مدلی با نام "شبکه‌های حافظه کوتاه‌مدت بلند" یا به اختصار LSTM منجر شد. در واقعیت، LSTM مدل‌های پیشرفته‌تری نسبت به شبکه‌های RNN ساده هستند. زمانی که ما با داده‌های دنباله‌ای طولانی سر و کار داریم، گاهی اوقات برای به دست آوردن خروجی مطلوب نیاز

^۱ Convolutional Neural Network (CNN)

^۲ shift invariant

^۳ space invariant

است به داده‌های اخیر توجه کنیم. LSTM قادر است این نوع وابستگی‌های طولانی را مدیریت کند و بادقت به ورودی‌های اخیر نگاه دارد. این به این معناست که LSTM قادر است هر دنباله‌ای با هر طولی را پردازش کند. یکی از ویژگی‌های مهم LSTM، مقاومت نسبی آن در برابر مشکلات گرادیان ناپدیدشده است. این به معنای این است که در آموزش شبکه‌های LSTM، مشکلاتی که در شبکه‌های عصبی بازگشتی معمولاً به‌عنوان مشکل گرادیان ناپدیدشده شناخته می‌شوند، به میزان کمتری بروز می‌کنند. این ویژگی باعث می‌شود که LSTM به‌خوبی برای مسائلی که وابستگی‌های طولانی‌مدت دارند، مناسب باشند. همچنین یکی از ویژگی‌های دیگر LSTM، عدم حساسیت نسبی به طول فاصله است. این به معنای این است که LSTM قادر به مدیریت و پردازش داده‌ها با طول متغیر و ناشناخته است. این ویژگی مهم برای پردازش دنباله‌ها و سری‌های زمانی است که ممکن است طول متغیر داشته باشند و از آنجا که LSTM می‌تواند اطلاعات وابستگی‌های بلندمدت را نگه دارد، بهترین تجزیه و تحلیل داده‌ها را ارائه می‌دهد.

به‌طور خاص، واحد LSTM از چهار گیت تشکیل شده است: (الف) دروازه forget، (ب) دروازه ورودی، (ج) دروازه خروجی و (د) وضعیت سلول. حالت سلولی (C) مسئول جریان اطلاعات بین واحدها است. نفوذ روزافزون سیستم‌های فتوولتائیک (PV) به سیستم‌های انرژی الکتریکی، مسائل فنی و اقتصادی متعددی را مطرح می‌کند که عمدتاً به ماهیت غیرقابل پیش‌بینی آن‌ها مربوط می‌شود. یک راه‌حل امیدوارکننده برای بسیاری از این موارد، اجرای مدل‌های پیش‌بینی تولید PV قوی است. در این پژوهش یک مدل ترکیبی جدید شبکه عصبی کوتاه‌مدت حافظه کوتاه‌مدت تغذیه^۱ پیشنهاد شده است که قادر به انجام پیش‌بینی‌های بسیار کوتاه و کوتاه‌مدت است. عملکرد مدل پیشنهادی با مدل‌های LSTM منفرد مقایسه می‌شود و دقت پیش‌بینی آن در دو افق پیش‌بینی مختلف ارزیابی می‌شود: (الف) ۱۵ دقیقه جلوتر و (ب) ۱ ساعت جلوتر.

۲-۲-۱۵ LSTM و GRU به‌عنوان یک راه‌حل

LSTM و GRU به‌عنوان راه‌حلی برای حافظه کوتاه‌مدت ایجاد شدند. آن‌ها مکانیسم‌های درونی به نام گیت دارند که می‌توانند جریان اطلاعات را تنظیم کنند. شبکه‌های حافظه کوتاه‌مدت طولانی (LSTM) از اهمیت ویژه‌ای برخوردارند زیرا توانایی حفظ اطلاعات وابستگی‌های بلندمدت را دارند و می‌توانند از این اطلاعات برای پیش‌بینی‌ها و پردازش‌های پیچیده استفاده کنند. این ویژگی‌ها باعث می‌شود که LSTM در بسیاری از برنامه‌ها و وظایف مختلف مورد استفاده قرار گیرند. تعدادی از کاربردهای این شبکه‌ها:

۱. تشخیص ویدئو و پردازش تصویری: LSTM نیز می‌تواند در تحلیل تصاویر و ویدئوها مؤثر باشد. برای مثال، در تشخیص حرکت در ویدئوها، تشخیص چهره، تولید شرح برای تصاویر و ویدئوها و دیگر وظایف پردازش تصویر و ویدئو.

۲. تولید متن و زیرنویس: LSTM می‌تواند به‌عنوان یکی از ابزارهای کلیدی در تولید متون و زیرنویس‌ها برای ویدئوها و تصاویر به کار رود. این وظیفه به تولید زیرنویس برای فیلم‌ها، توضیح تصاویر، و حتی تولید محتوای متنی برای وبلاگ‌ها و خبرگزاری‌ها کمک می‌کند.

¹ ELSTM-FFNN

۳. تشخیص ناهنجاری و پیش‌بینی‌ها: LSTM می‌تواند در تشخیص ناهنجاری‌ها و پیش‌بینی‌های مبتنی بر داده‌های سری زمانی مفید باشد. به‌عنوان مثال، در تشخیص ناهنجاری در داده‌های مالی یا ترافیک شبکه، LSTM می‌تواند الگوهای نادر را تشخیص دهد.

۴. پردازش زبان طبیعی: LSTM به‌عنوان یکی از مدل‌های مهم در پردازش زبان طبیعی (NLP) نیز استفاده می‌شوند. از ترجمه ماشینی تا تحلیل متون و پرسش‌وپاسخ، LSTM در بسیاری از وظایف NLP مورد استفاده قرار می‌گیرد.

۲-۲-۱۶- توازن بار

توازن بار^۱ یکی از مهم‌ترین فرآیندهای بهینه‌سازی در سیستم‌های توزیع‌شده است که باهدف توزیع هوشمندانه و خودکار ترافیک بین چندین سرور یا منبع محاسباتی انجام می‌شود. هدف اصلی آن جلوگیری از تمرکز بار بر روی فقط یک گره خاص و تضمین پایداری، کارایی و در دسترس‌پذیری بالاتر سرویس‌ها است. این سازوکار با تقسیم ترافیک در لحظه و ارزیابی وضعیت هر یک از سرورها، از بروز گلوگاه‌های عملکردی جلوگیری می‌کند که همین امر موجب می‌شود در محیط‌های ابری و میکروسرویسی نقشی حیاتی را ایفا کند.

۲-۲-۱۷- کوبرنتیس

کوبرنتیس^۲ به‌عنوان یک پلتفرم منبع‌باز برای ارکستریشن کانتینرها شناخته می‌شود که ابزار اصلی در مدیریت استقرار و مقیاس‌بندی سرویس‌های میکروسرویسی محسوب می‌شود. این سیستم با خودکارسازی فرآیندهایی مانند توزیع بار، زمان‌بندی کانتینرها و بازیابی خودکار در صورت خرابی، یک بستر معتمد و کارآمد برای اجرای سرویس‌های ابری فراهم می‌کند. کوبرنتیس به‌صورت پویا وضعیت گره‌ها را پایش می‌کند و بر اساس سیاست‌های تعریف‌شده، منابع را تخصیص می‌دهد.

۲-۲-۱۸- داکر

داکر^۳ نیز یک بستر اجرایی برای کانتینرها فراهم می‌کند که به توسعه‌دهندگان اجازه می‌دهد تا برنامه‌ها و وابستگی‌های آن‌ها را در قالب یک بسته مستقل و سبک‌وزن اجرا کنند. این روند باعث می‌شود تا نرم‌افزارها بدون وابستگی به محیط اجرا، در هر سرور یا سیستم‌عاملی به‌صورت یکسان عمل کنند. ترکیب داکر و Kubernetes قادر است تا یک زیرساخت منعطف و خودکار برای اجرای الگوریتم‌های یادگیری در مقیاس بزرگ فراهم کند.

۲-۲-۱۹- پرومیتوس و گرافانا^۴

این دو به‌عنوان ابزارهای کلیدی برای پایش و تحلیل داده‌های عملکردی سیستم شناخته شده‌اند. Prometheus داده‌های متریک مانند استفاده از CPU، حافظه و نرخ تأخیر پاسخ را در زمان واقعی جمع‌آوری می‌کند، درحالی‌که

¹ Load Balancing

² Kubernetes

³ Docker

⁴ Prometheus and Grafana

Grafana امکان مصورسازی و تحلیل تعاملی این داده‌ها را فراهم می‌کند. در این پژوهش از این دو برای ارزیابی و مقایسه عملکرد الگوریتم پیشنهادی استفاده خواهیم کرد.

۲-۲-۲۰- کیفیت سرویس^۱ (QoS)

به مجموعه‌ای از تکنیک‌ها و سیاست‌ها اطلاق می‌شود که برای حفظ سطح مشخصی از عملکرد در شبکه‌ها و سامانه‌های توزیع شده استفاده می‌شود. شاخص‌هایی مثل تأخیر، نرخ ازدست‌رفتن بسته‌ها و پهنای باند از معیارهای اصلی QoS محسوب می‌شوند. در این تحقیق، بهبود QoS یکی از اهداف اصلی ما محسوب می‌شود که از طریق یادگیری تقویتی و بهینه‌سازی توزیع بار به آن خواهیم رسید.

۲-۳- پیشینه تحقیق

با پیشرفت‌های موجود در زمینه سیستم‌های میکروسرویس محور و بهره‌گیری از سرویس‌های رایانش ابری، بهبود عملکرد API‌ها به یکی از ارکان کلیدی در تضمین کیفیت سرویس‌ها و پاسخ‌گویی مؤثر به درخواست‌های کاربران تبدیل شده است. API‌ها به عنوان یک واسطه بین اجزای نرم‌افزاری مختلف، یک نقش مهم و حیاتی در عملکرد سامانه‌های ابری دارند. بهمین خاطر، مطالعات زیادی در حوزه بهبود عملکرد API‌ها، بخصوص در حوزه زمان پاسخ‌دهی، تخصیص منابع و مقیاس‌پذیری خودکار، با استفاده از الگوریتم‌های یادگیری ماشین و چارچوب‌های ارکستریشن صورت گرفته است.

محققین در مقاله [۵] به بررسی تکنیک‌های موجود و چالش‌های توزیع بار در محیط‌های کانتینری پرداختند. در این مقاله، آن‌ها ابتدا بر روی مکانیزم‌های توزیع بار تمرکز کردند و ابزارهایی مانند Docker Swarm، kube-proxy، ingress controller و service mesh را به عنوان مکانیزم‌های توزیع بار در سطح API معرفی کردند. آن‌ها سپس عملکرد این ابزارها را در شرایط بار متغیر مورد تحلیل قرار دادند. در نهایت نویسندگان بر روی چالش‌هایی نظیر زمان پاسخ‌دهی بالا، نبود سازوکار تطبیقی بلادرنگ با بار سیستم و محدودیت در سیاست‌های مقیاس‌پذیری مانور دادند و آن‌ها را مورد بررسی قرار دادند.

نویسندگان مقاله [۶] الگوریتم‌های پیشرفته‌ای مثل GA-PSO، MPSO، CSO و ACOPSO را انتخاب و بررسی کردند، سپس میزان تأثیر یادگیری ماشین را در بهبود بهره‌وری منابع و کاهش مصرف انرژی مورد ارزیابی قرار دادند. همچنین آن‌ها نشان دادند که ترکیب الگوریتم‌های فراابتکاری با مدل‌های پیش‌بینی پویا قادر است عملکرد سیستم‌های ابری را در مواجهه با بارهای پرتراфик به صورت قابل توجهی بهبود دهد.

در مقاله دیگری [۷] با تمرکز بر روی یادگیری تقویتی، کاربرد الگوریتم‌های یادگیری در تخصیص منابع در محیط‌های ابری را مورد بررسی قرار دادند. این مقاله در محیط شناخته شده WorkflowSim شبیه‌سازی شد که نشان دادند الگوریتم یادگیری با قدرت تصمیم‌گیری بالا قادر است عملکرد بهتری را نسبت به الگوریتم‌های سنتی و ساده زمان‌بندی مثل FCFS، Min-Min و Max-Min داشته باشد که همین امر موجب کاهش زمان پاسخ و جلوگیری از بارگذاری بیش از حد سرورها می‌گردد. آن‌ها نشان دادند که با استفاده از الگوریتم Q-learning، تصمیمات زمان‌بندی براساس داده‌های عملکردی سیستم در زمان واقعی اتخاذ می‌گردد، نتایج بدست آمده نشان

¹ Quality of Service

داد که مدل ارائه شده در شرایطی با بار بالا و متغیر، به نسبت الگوریتم‌های سنتی مانند Round-Robin و Least Connection، می‌تواند بهره‌وری بیشتری داشته باشد.

در تحقیقی [۸] یک سیستم AGE معرفی کردند که قادر است ارزیابی خودکار دروازه‌های API را از نظر پارامترهایی مانند زمان پاسخ، نرخ خطا و درصد موفقیت فراهم کند. این مقاله در حوزه مدیریت API‌ها نشان داد که ارزیابی بلادرنگ API‌ها می‌تواند تصمیم‌گیری در انتخاب مسیرهای ارتباطی را تا حد بالایی بهبود دهد.

در یک بررسی [۹] یک مدل DNA-like را باهدف مدیریت محتوای ابری، کاربرد ساختارهای داده‌ای معنا بنیان معرفی کردند که آن را در محیط XaaS مورد بررسی قرار دادند. مدل معرفی شده در این مقاله از یادگیری ماشین جهت انتقال و ذخیره‌سازی داده‌ها بهره برده است که ویژگی بدون افزونگی را بهبود می‌دهد و همچنین در بهبود هوشمندی سامانه‌های ابری نیز نقش بسزایی دارد.

محققین در مقاله خود [۱۰] یک بررسی جامع و کامل بر روی روش‌های موجود با استفاده از الگوریتم‌های یادگیری ماشین پرداختند. آن‌ها یک طبقه‌بندی دقیق ارائه دادند که تحقیقات موجود را براساس ویژگی‌های مشترک دسته‌بندی می‌کند. همچنین آن‌ها توانستند سیر تکامل فناوری‌های موجود مبتنی بر یادگیری ماشین در طی سال‌های ۲۰۱۶ تا ۲۰۲۱ را بر اساس هدف و معیارها مورد نمایش قرار دهند. آن‌ها تحلیل خود را با تأکید بر ویژگی‌های کلیدی انجام دادند و در نهایت بر روی چالش‌های تحقیقاتی باز در این حوزه تمرکز کردند.

مقاله [۱۱] یک چارچوب زمان‌بندی جدید براساس یادگیری بنام KaiS معرفی کردند که برای سیستم‌های ترکیبی لبه - ابر ارائه شده و با Kubernetes سازگار تعریف شده است. هدف اصلی این مدل بهبود نرخ توان عملیاتی بلندمدت در پردازش درخواست‌ها است که این چارچوب از الگوریتم‌های یادگیری تقویتی چند عامله و شبکه‌های عصبی گرافی جهت مدل‌سازی شرایط سیستم و تصمیم‌گیری بهره‌مند می‌شود. KaiS با کمک یک روش زمان‌بندی دو مقیاسی جدید، یک هماهنگی مناسب مابین ارسال درخواست‌ها و ارکستراسیون سرویس انجام می‌دهد که موجب بهبود وضعیت سیستم می‌گردد. آن‌ها آزمایش‌های خود را با استفاده از داده‌های واقعی انجام دادند که نتایج نشان داد که KaiS توانسته نرخ توان عملیاتی سیستم را تا ۱۴٪ بهبود دهد و همچنین موجب کاهش هزینه زمان‌بندی تا ۳۴٫۷٪ شده است.

در یک بررسی جدید [۱۲] محققین مروری جامع و کامل بر روی روش‌های یادگیری عمیق باهدف مدیریت منابع در محیط‌های ابری انجام دادند. نویسندگان این مقاله مروری بر این باورند که هرچه چارچوب سیستم میکروسرویس محور پیچیده‌تر شود و بار متغیر درخواست‌ها افزایش یابد، به همان نسبت روش‌های سنتی تخصیص منابع، دیگر نمی‌توانند پاسخ‌گو باشند. آن‌ها همچنین یک طبقه‌بندی بر روی مدل‌های یادگیری عمیق مانند RNN، CNN و LSTM انجام دادند که نتایج این لیست نشان داد که این روش‌ها می‌توانند در پیش‌بینی مصرف منابع و مقیاس‌پذیری پویا، کارایی بالایی داشته باشند. در نهایت آن‌ها بر اهمیت پیاده‌سازی مدل‌های هوشمند در کنار ابزارهای مانیتورینگ بلادرنگ نیز تأکید کردند.

در تحقیقی دیگر [۱۳] یک چارچوب جدید براساس یادگیری تقویتی، باهدف تخصیص منابع پویا در مراکز داده ابری معرفی کردند. هدف این چارچوب مدل‌سازی محیط ابری به‌عنوان یک سیستم تصمیم‌گیری مارکوف است که قادر است براساس وضعیت لحظه‌ای منابع و بار کاری، اقدامات مفیدی جهت تخصیص CPU، RAM و پهنای باند

انجام دهد. نویسندگان از الگوریتم DQN استفاده کردند تا بتوانند قابلیت تصمیم‌گیری در محیط‌های نامطمئن و پویا را فراهم کنند. نتایج آزمایش‌ها نشان داد که مدل پیشنهادی توانسته در مقایسه با الگوریتم‌های کلاسیک مانند FCFS و حتی برخی مدل‌های یادگیری، بهره‌وری انرژی بیشتر و حتی زمان پاسخ کوتاه‌تری داشته باشد.

یک مقایسه پرکاربرد بین روش‌های معمول که در چارچوب یک سیستم میکروسرویسی مبتنی بر ابر استفاده می‌شوند انجام شد [۱۴]. هدف این بررسی، تحلیل عملکرد مصرف منابع، امنیت، مقیاس‌پذیری و قابلیت مانیتورینگ این ابزارها بوده است. نتایج بدست آمده از این بررسی نشان داد که استفاده از سرویس مش در کنار Kubernetes، می‌تواند به شکل چشمگیری در افزایش پایداری APIها و توزیع بهینه بار تأثیرگذار باشد.

همچنین در راستای هدف تحقیق یک الگوریتم تطبیقی توازن بار در رایانش ابری ارائه شد [۱۵] که براساس الگوریتم‌های یادگیری ماشین طراحی و ساخته شده است. هدف آن‌ها، کاهش تأخیر، افزایش نرخ موفقیت پاسخ‌دهی و بهبود مصرف منابع است. الگوریتم پیشنهادی با استفاده از مدل‌های طبقه‌بندی و خوشه‌بندی، به طور خودکار ترافیک ورودی را تحلیل می‌کند، سپس منابع مناسب را در لحظه تخصیص می‌دهد. نتایج این شبیه‌سازی‌ها نشان داد که روش معرفی شده در این مقاله در مقایسه با روش‌های سنتی مانند Round Robin و Least Connection، توانسته عملکرد بهتری از نظر سرعت پاسخ و پایداری در ترافیک بالا داشته باشد.

جهت کنکاش بیشتر، نویسندگان در مقاله [۱۶] حدود ۲۵ مقاله علمی را مورد کنکاش و بررسی قرار دادند. آن‌ها انواع الگوریتم‌ها، حوزه‌های کاربرد و محیط‌های ارزیابی را دسته‌بندی و سپس تحلیل و بررسی کردند. آن‌ها در نهایت بر روی اهمیت پیاده‌سازی هوش مصنوعی را در تخصیص منابع و ارتقای عملکرد سیستم‌های رایانش ابری بحث کردند.

در مقاله دیگر نویسندگان [۱۷] یک چارچوب زمان‌بندی منابع در محیط رایانش ابری ارائه دادند که به طور هم‌زمان به دو هدف اصلی توجه می‌کند: بهبود عملکرد مصرف انرژی و حفظ کیفیت سرویس. نویسندگان این مقاله دو الگوریتم NSGA-II (الگوریتم ژنتیک چندهدفه) و الگوریتم شبکه‌های عصبی عمیق را برای پیش‌بینی بار کاری و تصمیم‌گیری هوشمندانه در تخصیص منابع ترکیب و استفاده کردند. مدل پیشنهادی در این مقاله با شبیه‌سازی‌هایی که صورت گرفته توانست تا ۱۸٪ مصرف انرژی را کاهش و تا ۲۵٪ نرخ موفقیت پاسخ را بهبود دهد که این آمار به نسبت روش‌های سنتی بهبود قابل توجهی را نشان می‌دهد.

برای مقیاس‌پذیری خودکار، محققین در [۱۸] یک روش جدید را با کمک سرویس‌های میکروسرویسی در Kubernetes ارائه دادند که براساس مدل‌های پیش‌بینی‌کننده یادگیری ماشین است. آن‌ها با کمک الگوریتم‌هایی مانند LSTM و XGBoost، بار کاری آینده را با دقت بسیار بالایی پیش‌بینی کردند و توانستند مقیاس‌پذیری پویا را با HPA و KEDA اجرا کنند. مدل معرفی شده موجب کاهش زمان تأخیر، افزایش پایداری سرویس‌ها و کاهش مصرف منابع شده است. تمامی آزمایش‌ها بر روی بارهای واقعی صورت گرفت که نتایج این آزمایش‌ها نشان داد که مدل پیشنهادی، عملکرد بسیار بالایی نسبت به روش‌های سنتی دارد.

یک پروتکل مسیریابی امن و هوشمند در [۱۹] معرفی شد که علاوه بر بهبود عملکرد مصرف انرژی، مشکلات ناشی از تراکم و تداخل را در شبکه‌های حسگر بی‌سیم پوشش دادند. در تحقیقات IoT که حجم تبادل داده‌ها زیاد است، ترافیک شبکه می‌تواند موجب ازدست‌رفتن بسته‌ها و افزایش قابل‌توجهی از تأخیر گردد. این مقاله با کمک

خوشه‌بندی تطبیقی و الگوریتم‌های بهبود عملکرد هوشمند، مسیرهای بدون تراکم را شناسایی و اولویت‌بندی می‌کند. آن‌ها امنیت داده‌ها را با کمک مکانیزم رمزنگاری تطبیقی مبتنی بر منطق کوانتومی فراهم کردند. پروتکل پیشنهادی این مقاله برای کاربردهای حساس مانند IoMT و شهر هوشمند پیشنهاد می‌شود که تلاش می‌کند تا کارایی شبکه را تا حد ممکن حفظ کند.

محققین این بار تلاش کردند تا یک چارچوب جدید برای مسیریابی داده‌ها در شبکه‌های مراقبت سلامت هوشمند ارائه دهند [۲۰]. این مدل معرفی شده شامل سه لایه BAN^۱، رایانش مه و رایانش ابری است که با کمک یادگیری تقویتی عمیق، قادر است تصمیمات بهبود یافته‌ای در مورد مسیر داده‌ها بگیرد. همین امر موجب کاهش مصرف انرژی گره‌ها و تأخیر انتقال اطلاعات حیاتی بیماران می‌گردد. الگوریتم PPO^۲ به عنوان هسته تصمیم‌گیری در این سیستم به کاررفته است. هدف اصلی این مقاله بهبود عملکرد هم‌زمان مصرف انرژی، اطمینان از رسیدن به موقع داده‌های پزشکی و حفظ کیفیت سرویس در شبکه است. این مدل برای محیط‌هایی مانند پایش بیماران از راه دور مناسب است؛ زیرا شامل داده‌های حساس است که باید با سرعت و امنیت بالا منتقل شوند.

در [۲۱] نویسندگان یک مدل جدید بنام MSARS معرفی کردند که ترکیبی از متایادگیری^۳ و یادگیری تقویتی برای تخصیص منابع و مقیاس‌دهی تطبیقی سرویس‌ها در معماری میکروسرویس است. این مدل براساس یادگیری الگوهای تغییر بار در زمان طراحی شده که قادر است تصمیمات مقیاس‌دهی را به صورت پویا و بهینه بگیرد که همین امر باعث کاهش چشمگیر تخطی از اهداف سطح سرویس^۴ می‌گردد. آن‌ها آزمایش‌های خود را در محیط Kubernetes انجام دادند که نتایج نشان داد مدل معرفی شده می‌تواند زمان پاسخ API‌ها را تا ۲۳٪ و مصرف منابع را تا ۱۸٪ کاهش دهد.

یک مدل جدید بنام STaleX نیز در [۲۲] معرفی و توسعه داده شد. مدل معرفی شده براساس تحلیل وابستگی‌های زمانی-مکانی^۵ بین سرویس‌ها است. این مدل از یک کنترل‌کننده یادگیری‌محور تطبیقی استفاده می‌کند که موجب بهینه‌سازی میزان مصرف منابع در شرایط بار متغیر می‌شود. STaleX داده‌های خود را با کمک ابزارهایی مانند Prometheus جمع‌آوری کرده است. نتایج شبیه‌سازی‌ها نشان داد که مدل پیشنهادی به نسبت HPA کلاسیک در Kubernetes، میانگین تأخیر پاسخ API را تا ۲۹٪ کاهش می‌دهد.

جهت معرفی یک نسخه هوشمند در مقاله [۲۳] از مکانیزم مقیاس‌پذیری Kubernetes بنام Smart HPA^۶ استفاده کردند که از الگوریتم‌های یادگیری ماشین برای پیش‌بینی نیاز به منابع کمک گرفتند. برخلاف HPA سنتی که فقط بر روی معیارهای ایستا مانند CPU یا RAM تکیه می‌کند، Smart HPA قادر است داده‌های تاریخی، بار کاری API و نرخ موفقیت پاسخ‌ها را تحلیل کند و با تصمیم‌گیری درست، بهره‌وری منابع را تا جای ممکن بهبود دهد. آن‌ها این مدل را بر روی خوشه‌های واقعی Kubernetes انجام دادند که نتایج نشان داد که Smart HPA توانسته تا ۲۲٪ مصرف منابع را کاهش و از طرفی نرخ موفقیت پاسخ‌های API را تا ۱۵٪ افزایش دهد. نویسندگان در مقاله

¹ Body Area Network

² Proximal Policy Optimization

³ Meta-Learning

⁴ SLO Violations

⁵ Spatiotemporal Dependencies

⁶ Horizontal Pod Autoscaler

[۲۴] یک مدل هوشمند جهت تخصیص منابع در محیط‌های ابری ترکیبی^۱ معرفی کردند که با ترکیب پیش‌بینی بار کاری^۲ و یادگیری تقویتی ارائه شده است. این مدل، وضعیت لحظه‌ای API‌ها و میزان بار شبکه را تحلیل می‌کند و با کمک الگوریتم DQN^۳ تصمیمات مقیاس‌دهی را تا حد بسیار بالایی بهینه می‌کند. نتایج این تحقیق نشان داد که این روش می‌تواند میانگین تأخیر API را تا ۲۰٪ کاهش و پایداری سیستم را تا ۱۷٪ افزایش دهد.

نویسندگان در [۲۵] مدلی را بنام KIS-S^۴ معرفی کردند. این مدل جدید یک چارچوبی را برای مقیاس‌دهی خودکار GPU محور در محیط‌های Kubernetes طراحی و ارائه می‌دهد. آن‌ها از الگوریتم PPO^۵ برای تصمیم‌گیری تطبیقی کمک گرفتند که این سیستم با یکپارچه‌سازی Prometheus برای جمع‌آوری داده‌های متریک و بهره‌گیری از یادگیری تقویتی، قادر است تا در بارهای کاری هوش مصنوعی^۶ میزان استفاده از GPU را تا ۳۵٪ بهینه کند و تأخیر پاسخ را تا ۲۶٪ کاهش دهد.

محققین یک روش جدید براساس الگوریتم گراف عصبی معرفی کردند [۲۶] و چالش وابستگی‌های زمانی فضایی بین میکروسرویس‌ها را نیز ارائه می‌دهند این الگوریتم ابتدا با کمک یک روش جدید، ماتریس شباهت بین سرویس‌ها را به‌صورت پویا می‌سازد، سپس با یک GNN، ویژگی‌های زمانی و ساختاری را استخراج می‌کند تا بتوان منابع به‌طور هم‌زمان برای کل شبکه سرویس‌ها تنظیم گردد. این روش از اثرات دومینو در تخصیص منابع جلوگیری می‌کند. در تحقیقی دیگر [۲۷]، یک چارچوب گسترش یافته برای تصمیم‌گیری مقیاس‌پذیر منابع در خوشه‌های کانتینری ارائه شد. این مدل مبتنی بر الگوریتم TD3^۷ طراحی شده، هدف آن توزیع نقاط تصمیم‌گیری در چند گره است تا بتوان از گلوگاه‌های مرکزی جلوگیری کند. نتایج این تحقیق نشان داد که پاسخ متوسط تا حدود ۱۵٪ و درخواست‌های شکست‌خورده تا حدود ۲۴٪ کاهش یافته است. باتوجه به نتایج می‌توان گفت که این روش برای سیستم‌های بزرگ با تعداد زیادی میکروسرویس مناسب است.

در [۲۸] یک مدل جدید بنام Reclaimer معرفی شد. این سیستم به‌عنوان یک سیستم همه‌کاره‌ای ارائه شده که با کمک یادگیری عمیق و تقویتی، تخصیص‌دهی پویا را به میکروسرویس‌ها مدیریت می‌کند تا بتوان کیفیت سرویس را حفظ کرد. این الگوریتم براساس ویژگی‌هایی سطح کانتینر است؛ چراکه یاد گرفته تا بطور پویا به تغییرات بار و ساختار میکروسرویس پاسخ دهد. در مقایسه با روش‌های رایج، این مدل توانسته در برخی از آزمایش‌ها مصرف CPU را بین ۳۸٪ تا ۷۴٪ کاهش دهد، باینکه تخطی از کیفیت سرویس نداشته باشد.

چند محقق در یک بررسی جدید [۲۹] برروی زنجیره‌های فراخوان بین میکروسرویس‌ها تمرکز کردند. آن‌ها ابتدا وابستگی‌ها را تحلیل کردند تا بتوانند زنجیره‌های بحرانی را شناسایی کنند. سپس با کمک یادگیری تقویتی، مقیاس افقی و عمودی را برای زنجیره‌ها انتخاب و ترکیب کردند. این روش توانست نرخ پاسخ‌دهی تا حدود ۲۶٪ بهبود دهد. آن‌ها جهت ارزیابی‌های خود از بستر واقعی Kubernetes بهره گرفتند.

¹ Hybrid Cloud

² Load Forecasting

³ Deep Q-Network

⁴ Kubernetes Inference Simulator - Smart Scaling

⁵ Proximal Policy Optimization

⁶ AI Workloads

⁷ Twin Delayed Deep Deterministic Policy Gradient

جهت بررسی هرچه بیشتر محیط‌های ابری [۳۰] تلاش شد تا برای محیط‌های ابری یک ترکیب جدید طراحی و ارائه دهند. جهت رسیدن به این هدف آن‌ها یک چارچوب با کمک یادگیری تقویتی پیشنهاد دادند که قادر باشد منابع را به‌صورت پویا تنظیم کند تا بتواند معیارهای هزینه و تأخیر را بهینه کند. با وجود این تلاش‌ها نتایج شبیه‌سازی نشان داد که می‌توان هزینه را تا ۳۰-۴۰٪ کاهش و تأخیر را در اوج بار ۱۵-۲۰٪ پیشنهادی داد. همچنین آن‌ها توانستند بهره‌وری منابع را نیز تا ۲۰-۳۰٪ بهبود دهند.

باهدف پیش‌بینی بار، در یک مقاله [۳۱] یک مدل جدید بنام CoScaI معرفی شد. این مدل ترکیبی از مقیاس‌پذیری افقی، عمودی و روش برون‌گذاری است که هدف آن پیش‌بینی بار در سیستم است. روال کار به این صورت است که ابتدا از یک مدل GRU استفاده می‌شود تا بار آتی را پیش‌بینی کند، سپس یادگیری تقویتی تصمیم می‌گیرد که از کدام نوع مقیاس استفاده کند. نتایج آزمایش‌ها نشان داد که تأخیر بین ۱۹٪ تا ۲۹٪ و زمان اتصال بین خدمات تا ۱۶٪ کاهش داشته است.

جهت تخصیص منابع به سرویس میکروسرویسی مقاله [۳۲] یک مدل جدیدی براساس یادگیری مدل‌دار بنام MIRAS در محیط جریان کاری معرفی کرد. روش ارائه شده در این مقاله قادر است ساختار مدل محیط را یاد بگیرد تا بتواند اثرات اقدام‌ها را پیش‌بینی کند و تصمیمات بهتری بگیرد. این روش در مقایسه با یادگیری بی‌مدل و سنتی عملکرد بهتری در کنترل تأخیر و بار دارد.

همچنین یک مدل جدید باهدف مقیاس‌پذیری در محیط‌های کانتینری ارائه شد [۳۳] که قادر است وابستگی‌های متقابل بین سرویس‌ها را تحلیل و مقیاس‌پذیری را به‌صورت سلسله‌مراتبی انجام دهد. در آزمایش‌های صورت گرفته بر روی Kubernetes، میانگین تأخیر تا ۲۸٪ کاهش و بهبود منابع تا ۱۹٪ بهبود داشته است.

در مقاله [۳۴] نویسندگان بر روی محیط‌های HFC^۱ تمرکز کردند آن‌ها جهت زمان‌بندی سرویس‌های کانتینری از الگوریتم Q-Learning عمیق استفاده کردند. هدف مدل پیشنهادی در این مقاله، بهینه‌سازی زمان پاسخ و مصرف انرژی در کنار یکدیگر است. مدل پیشنهادی آن‌ها وضعیت شبکه، CPU و تأخیر را به‌عنوان وضعیت در RL لحاظ می‌کند و کنش‌ها شامل انتقال سرویس‌ها بین لایه‌های مه و ابر هستند. نتایج آزمایش‌ها نشان داد که زمان پاسخ تا ۳۵٪ کمتر و مصرف انرژی نیز تا ۲۲٪ پایین‌تر از الگوریتم‌های مبتنی بر هیوریستیک است.

در پژوهش [۳۵] نویسندگان از یک معماری HDRL^۲ برای مدیریت ترافیک API در سیستم‌های تبلیغات ابری استفاده کردند. آن‌ها عامل سطح بالا سیاست‌های کنترل نرخ API را تعیین کرده و همچنین عامل سطح پایین منابع را بین API‌ها تخصیص دادند. مدل معرفی شده علاوه بر بهینه‌سازی کارایی و تأخیر، هزینه و مصرف انرژی را نیز به‌عنوان یک پاداش منفی لحاظ می‌کند. نتایج این تحقیق نشان داد که معماری معرفی شده قادر است پایداری نرخ پاسخ API را تا ۴۰٪ بهبود دهد و از نوسانات شدید جلوگیری کند. در ادامه جدولی جهت مشاهده خلاصه تحقیقات مطالعه شده آورده شده است.

^۱ Hybrid Fog-Cloud

^۲ Hierarchical Deep Reinforcement Learning

جدول (۲-۱) مروری بر تحقیقات گذشته

سال	رفرنس	چارچوب	روش / الگوریتم	تمرکز اصلی پژوهش	مزایا	معایب	معیارهای ارزیابی	محیط پیاده سازی
۲۰۲۳	۶	GA-PSO / MPSO / CSO / ACOPSO	الگوریتم های فراابتکاری + مدل پیش بینی پویا	بهبود بهره وری منابع و کاهش مصرف انرژی	عملکرد سیستم های ابری در بار پرتراфик بهبود یافت	پیچیدگی محاسباتی بالا و زمان همگرایی زیاد؛ عدم پشتیبانی از تصمیم گیری	CPU, RAM, Energy	محیط شبیه سازی
۲۰۲۳	۷	RL در WorkflowSim	Q-learning	تخصیص منابع و زمان بندی	کاهش زمان پاسخ، جلوگیری از بارگذاری بیش از حد سرورها	وابستگی شدید به محیط شبیه سازی WorkflowSim؛ عدم اعتبارسنجی	Latency, Success Rate	شبیه سازی WorkflowSim
۲۰۲۳	۸	AGE	ارزیابی بلادرنگ API Gateway	ارزیابی API Gateway ها	تصمیم گیری بهینه در مسیرهای ارتباطی API	تمرکز صرف بر ارزیابی API بدون ارائه راهکار خودکار برای تخصیص یا مقیاس دهی منابع	Response Time, Error Rate, Success Rate	محیط شبیه سازی / ابزار صنعتی
۲۰۲۳	۱۶	مرور ۲۵ مقاله	دسته بندی الگوریتم ها و محیط ها	تحلیل حوزه ها و کاربردها	اهمیت هوش مصنوعی در تخصیص منابع و ارتقای عملکرد سیستم ها	مروری و تحلیلی بودن مقاله بدون ارائه راهکار عملی	-	محیط ابری
۲۰۲۳	۱۸	Auto-scaling پیش بینی شده Kubernetes	LSTM, XGBoost	مقیاس دهی خودکار سرویس ها	کاهش زمان تأخیر، افزایش پایداری، کاهش مصرف منابع	وابستگی به دقت مدل های پیش بینی بار؛ افت عملکرد	Latency, CPU/RAM	Kubernetes
۲۰۲۳	۲۶	بهبود هماهنگی زمانی - فضایی بین میکروسرویس ها و جلوگیری از تخطی SLA	DeepScaler (GNN + EM)	ساخت ماتریس شباهت پویا بین سرویس ها با EM	کاهش ۴۱٪ تخطی از SLA و کاهش هزینه در تخصیص منابع؛ جلوگیری از اثرات دومینویی در خوشه ها	پیچیدگی محاسباتی GNN و فرآیند ساخت ماتریس شباهت پویا	SLA violation, هزینه، مصرف منابع	Kubernetes + Prometheus dataset
۲۰۲۳	۲۹	بهینه سازی زنجیره های فراخوان بین میکروسرویس ها برای کاهش تأخیر	ChainsFormer (RL + Dependency Analysis)	شناسایی زنجیره های بحرانی و اعمال مقیاس افقی/عمودی	بهبود پاسخ دهی ~۲۶٪ و افزایش ~۸ throughput؛ کاهش گلوگاه های زنجیره ای در Kubernetes	هزینه بر بودن تحلیل زنجیره های فراخوان	.Requests/sec, Latency Scalability	Kubernetes 1.28 + Istio Service Mesh
۲۰۲۳	۳۰	تخصیص منابع در	Hybrid RL	یادگیری تطبیقی	کاهش هزینه ۳۰-۴۰٪	تمرکز اصلی بر کاهش	Resource, Delay, Cost	AWS EC2 +

OpenStack Hybrid Lab Environment	utilization	هزینه؛ عدم بررسی عمیق کیفیت سرویس	کاهش تأخیر اوج بار ۱۵-۲۰٪، بهبود بهره‌وری منابع تا ۳۰٪	برای توزیع منابع	Framework	محیط‌های ابر برای کاهش هزینه و تأخیر		
شبکه‌های IoT	Latency, Packet Loss, Energy	پیچیدگی الگوریتم و سربار رمزنگاری تطبیقی	مسیرهای بدون تراکم، امنیت تقویت‌شده، کارایی شبکه حفظ شد	مسیریابی بهینه و امن داده	خوشه‌بندی تطبیقی + الگوریتم بهبود عملکرد	پروتکل مسیریابی هوشمند	۱۹	۲۰۲۴
Kubernetes	Latency, CPU, Success Rate	نیاز به داده‌های تاریخی کافی برای متیادگیری؛ افزایش زمان راه‌اندازی اولیه سیستم	کاهش تخطی از SLO، کاهش زمان پاسخ تا ۲۳٪، مصرف منابع ۱۸٪ کمتر	تخصیص منابع و مقیاس‌دهی تطبیقی	Meta-learning + RL	MSARS	۲۱	۲۰۲۴
Kubernetes	CPU, RAM, Success Rate	بهبود تدریجی نسبت به HPA کلاسیک	کاهش مصرف منابع ۲۲٪، افزایش نرخ موفقیت ۱۵٪	Auto-scaling هوشمند	الگوریتم پیش‌بینی مبتنی بر ML	Smart HPA	۲۳	۲۰۲۴
Hybrid Cloud	Latency, Success Rate	تمرکز بر محیط Hybrid Cloud؛ افزایش پیچیدگی تصمیم‌گیری	کاهش میانگین تأخیر API ۲۰٪، پایداری سیستم ۱۷٪ افزایش	تخصیص منابع هوشمند	Deep Q-Network	AI-driven Hybrid Cloud	۲۴	۲۰۲۴
Kubernetes cluster	Failed .Response time Throughput .requests	نیاز به هماهنگی دقیق بین عامل‌های توزیع‌شده؛ افزایش سربار ارتباطی	کاهش زمان پاسخ ~۱۵٪ و کاهش درخواست‌های شکست‌خورده ~۲۴٪؛ افزایش پایداری در سیستم‌های بزرگ	تقسیم تصمیم‌گیری بین گره‌ها؛ یادگیری هم‌زمان چند عامل RL	TD3-based Distributed Scaling Framework	تصمیم‌گیری توزیع‌شده برای مقیاس‌پذیری منابع در خوشه‌های بزرگ	۲۷	۲۰۲۴
Docker Swarm + Linux Containers	QoS .CPU utilization Adaptability .violation	پیچیدگی آموزش مدل Deep RL؛ نیازمند داده‌های متنوع برای جلوگیری از overfitting	کاهش مصرف CPU بین ۳۸٪ تا ۷۴٪ بدون افت QoS؛ سازگاری بالا در محیط‌های پویا	استفاده از ویژگی‌های سطح کانتنر	Reclaimer (Deep RL + Transfer Learning)	تخصیص پویا و بهینه منابع CPU بین میکروسرویس‌ها برای حفظ QoS	۲۸	۲۰۲۴
محیط سلامت هوشمند	Energy, Latency, QoS	تمرکز بر حوزه سلامت و IoMT؛ قابلیت تعمیم محدود	کاهش مصرف انرژی، بهبود زمان انتقال داده و QoS بیماران	مسیریابی داده در شبکه‌های IoMT	Deep RL (PPO)	چارچوب BAN-Fog Cloud	۲۰	۲۰۲۵
Kubernetes	Latency, CPU/RAM	وابستگی شدید به کیفیت داده‌های مانیتورینگ Prometheus؛ عدم بررسی هزینه سربار کنترلی	میانگین تأخیر API ۲۹٪، کاهش یافت	کاهش مصرف منابع و پایداری سرویس	کنترل‌کننده تطبیقی بر اساس وابستگی‌های زمان-مکان	STaleX	۲۲	۲۰۲۵
Kubernetes	GPU Usage, Latency	محدود به بارهای کاری GPU محور؛ عدم بررسی	بهره‌وری GPU تا ۳۵٪، کاهش تأخیر ۲۶٪	مقیاس‌دهی خودکار GPU-محور	PPO + GPU-aware	KIS-S	۲۵	۲۰۲۵

رفتار سیستم						
Kubernetes + Istio، داده‌های Prometheus + Grafana metrics	Resource .Latency، .efficiency، Communication overhead	افزایش سربار ارتباطی ناشی از تحلیل سلسله‌مراتبی وابستگی‌ها؛ پیچیدگی پیاده‌سازی	کاهش میانگین تأخیر تا ۲۸٪ و بهبود بهره‌وری منابع تا ۱۹٪؛ تصمیم‌گیری دقیق‌تر نسبت به HPA	تحلیل گراف فراخوانی برای مقیاس‌پذیری سلسله‌مراتبی؛ پیش‌بینی بار آینده	Gwydion (Hierarchical Neural Scaling)	خودکارسازی مقیاس‌پذیری با در نظر گرفتن وابستگی‌های بین سرویس‌ها
FogBus2 + CloudSim Edge extension	Energy .Response time، .consumption، Efficiency	وابستگی به تعریف دقیق فضای حالت؛ نیازمند داده‌های مانیتورینگ کامل	کاهش زمان پاسخ ۳۵٪ و مصرف انرژی ۲۲٪ نسبت به الگوریتم‌های heuristic	لحاظ وضعیت شبکه و CPU	Deep Q-Learning Scheduler	زمان‌بندی سرویس‌های کانتینری در محیط‌های Fog-Cloud برای کارایی و انرژی
Google Cloud Pub/Sub + TensorFlow Serving APIs	.Latency، .Throughput، Energy، .Stability	پیچیدگی معماری HDRL و هزینه محاسباتی بالا	بهبود پایداری نرخ پاسخ تا ۴۰٪؛ کاهش نوسانات latency و هزینه در بارهای پویا	کنترل چندسطحی منابع و نرخ API	HDRL (Hierarchical Deep RL)	بهینه‌سازی نرخ پاسخ API و تأخیر در سیستم‌های تبلیغات ابری

جدول ۱

۲-۳-۱- تحلیل مقالات

در ادامه و در این بخش، مقالات مرور شده براساس هدف پژوهش، نوع الگوریتم و چارچوب پیاده‌سازی، مورد بررسی و تحلیل قرار گرفتند. بر اساس بررسی انجام‌شده، مطالعات موردنظر را می‌توان در شش گروه زیر تقسیم کرد.



شکل (۱-۲) طبقه‌بندی پژوهش‌های مرتبط با بهبود عملکرد API مبتنی بر یادگیری ماشین

در این شکل، پژوهش‌های بررسی‌شده در این پایان‌نامه بر اساس نوع رویکرد و روش مورد استفاده طبقه‌بندی شدند. این طبقه‌بندی به درک ساختارمند ادبیات پژوهش و مقایسه بهتر رویکردهای موجود کمک می‌کند.

گروه اول: مدل‌های پیش‌بینی محور^۱

مدل‌های این گروه بر پیش‌بینی رفتار آینده سیستم، از جمله بار کاری، مصرف منابع و الگوهای ترافیک API تمرکز دارند. هدف اصلی این روش‌ها، فراهم‌سازی تصمیم‌گیری آگاهانه برای مقیاس‌پذیری و تخصیص منابع از طریق تحلیل داده‌های تاریخی و زمانی است. این مدل‌ها معمولاً نقش زیربنایی در سیستم‌های خودکار مدیریت منابع ایفا می‌کنند.

¹ Prediction-based Models

جدول (۲-۲) خلاصه مقالات مربوط به مدل‌های پیش‌بینی محور

محرط	معیارها	دستاورد	چارچوب	رفرنس
Cloud	CPU, RAM, Throughput	طبقه‌بندی تکامل ML در مدیریت منابع	مروری	[10]
Cloud	QoS, Resource Usage	پیش‌بینی دقیق مصرف منابع	RNN, CNN, LSTM	[12]
Kubernetes	Latency, Stability	بهبود Auto-scaling پیش‌بینی‌شده	LSTM, XGBoost	[18]
Kubernetes	CPU, RAM, Success Rate	Smart HPA با کاهش مصرف منابع	ML-based Prediction	[23]
Kubernetes	Latency, Connection Time	انتخاب نوع مقیاس‌پذیری	GRU + Prediction	[31]
Kubernetes	Latency, Resource Efficiency	مقیاس‌پذیری سلسله‌مراتبی	Hierarchical Prediction	[33]

تمرکز این گروه پیش‌بینی بار، مصرف منابع، یا رفتار آینده API برای تصمیم‌گیری است.

گروه دوم: مدل‌های یادگیری تقویتی^۱

در این دسته، محیط‌های ابری و میکروسرویسی به‌عنوان سیستم‌های تصمیم‌گیری پویا مدل‌سازی می‌شوند که عامل یادگیرنده با دریافت بازخورد از وضعیت سیستم، سیاست‌های بهینه تخصیص منابع و زمان‌بندی را به‌صورت آنلاین می‌آموزد. تمرکز اصلی این روش‌ها بر سازگاری بلادرنگ با تغییرات بار و کاهش تأخیر و مصرف انرژی است. در این گروه مدل‌های یادگیری تقویتی، براساس تصمیم‌گیری تطبیقی در زمان واقعی هستند که عامل یادگیرنده با دریافت پاداش یا جریمه از محیط، سیاست بهینه تخصیص منابع را یاد می‌گیرد. می‌توان از الگوریتم‌هایی مانند Q-Learning، DQN، PPO و TD3 نام برد. برای مثال محققین در مقالات [۷]، [۱۳] و [۲۴] نشان دادند که یادگیری ماشین می‌تواند در شرایط پرتراфик، تأخیر پاسخ را تا ۳۰٪ کاهش و بهره‌وری منابع را تا ۲۵٪ افزایش دهد. مزیت اصلی این مدل‌ها، واکنش سریع به تغییرات لحظه‌ای است، اما از چالش‌هایی مانند زمان همگرایی طولانی، نیاز به داده آموزشی بالا و حساسیت زیاد به نویز مانیتورینگ رنج می‌برند.

جدول (۲-۳) خلاصه مقالات مدل‌های یادگیری تقویتی

محرط	دستاورد	تمرکز	الگوریتم RL	رفرنس
WorkflowSim	کاهش Latency	Scheduling	Q-learning	[7]
Cloud	بهبود Energy & Latency	Resource Allocation	DQN	[13]
IoMT	کاهش تأخیر پزشکی	Secure Routing	PPO	[20]
Hybrid Cloud	پایداری بیشتر	API Scaling	DQN	[24]
Kubernetes	کاهش Failed Requests	Distributed Scaling	TD3	[27]
Cloud	کاهش هزینه ۴۰٪	Cost-Delay Tradeoff	RL Framework	[30]
WorkflowSim	کاهش نوسانات	Load Stability	Model-based RL	[32]
Fog-Cloud	بهبود Energy & Response	Scheduling	Deep Q-learning	[34]

تمرکز این گروه بر روی تصمیم‌گیری آنلاین و تطبیقی در محیط‌های پویا تعریف شده است.

¹ Reinforcement Learning-based Models

گروه سوم: مدل‌های ترکیبی^۱

مدل‌های ترکیبی از ترکیب مدل‌های پیش‌بینی گر و تصمیم‌گیر تشکیل شده است که به دنبال ایجاد تعادل مناسب بین پیش‌نگری و واکنش بلادرنگ هستند. در این مدل‌ها، معمولاً شبکه LSTM مسئول پیش‌بینی بار آینده است و عامل RL نیز براساس آن تصمیم مقیاس‌دهی می‌کند. نمونه‌هایی از این نوع شامل مقالات در جدول ۴ آمده است که با کمک تحلیل سری‌زمانی و یادگیری تقویتی چندعاملی، قادر است تأخیر را تا ۱۹٪ تا ۴۱٪ کاهش و پایداری سیستم را تا ۳۰٪ افزایش دهد. این گروه از لحاظ عملکرد کلی در مقایسه با سایر روش‌ها برتر هستند، اما از نظر پیچیدگی پیاده‌سازی و مصرف منابع چالش‌برانگیز هستند. روش‌های این گروه با ترکیب یادگیری تقویتی و مدل‌های پیش‌بینی یا یادگیری عمیق، تلاش می‌کنند مزایای هر دو رویکرد را هم‌زمان بهره‌برداری کنند. این ترکیب باعث بهبود پایداری تصمیم‌گیری، افزایش دقت در پیش‌بینی شرایط آینده و ارتقای عملکرد سیستم در سناریوهای پیچیده و پرترافیک می‌شود.

جدول (۲-۴) خلاصه مقالات مربوط به مدل‌های ترکیبی

محرط	مزیت اصلی	چارچوب	رفرنس
Edge-Cloud	Throughput %+14	Multi-agent RL + GNN	[11]
Cloud	Energy & QoS	NSGA-II + DNN	[17]
Kubernetes	Latency % -23	Meta-learning + RL	[21]
Kubernetes	GPU Efficiency % +35	PPO + GPU-awareness	[25]
Cloud Ads	Stability % +40	HDRL (Hierarchical RL)	[35]

مقالات جدول بالا از ترکیبی از روش‌های RL + Prediction / Meta-learning / Deep Learning به هدف خود رسیده‌اند.

گروه چهارم: روش‌های فراابتکاری و بهینه‌سازی چندهدفه^۲

مطالعات این دسته بر حل هم‌زمان چندین هدف متعارض مانند کاهش مصرف انرژی، تأخیر و هزینه تمرکز دارند. الگوریتم‌های فراابتکاری و تکاملی به دلیل توانایی بالا در جستجوی فضای راه‌حل، به عنوان ابزارهای مؤثر برای بهینه‌سازی سیستم‌های ابری پیچیده مورد استفاده قرار گرفته‌اند. در این گروه نیز از الگوریتم‌هایی مانند GA-PSO، NSGA-II، MPSO و CSO جهت انجام تنظیمات مربوط به پارامترهای مقیاس‌دهی و تخصیص منابع استفاده می‌شود. تحقیقات [۶] و [۱۷] نشان دادند که با ترکیب الگوریتم‌های فراابتکاری با مدل‌های پیش‌بینی قادر خواهیم بود تا مصرف انرژی و نرخ موفقیت پاسخ را تا حد خوبی بهبود دهیم. این مدل‌ها برای محیط‌های با بارهای ثابت یا نیمه پویا بسیار مناسب هستند؛ اما متأسفانه در مقیاس‌های بزرگ، دارای سربار محاسباتی بالایی هستند.

جدول (۲-۵) خلاصه مقالات مربوط به روش‌های فراابتکاری و بهینه‌سازی چندهدفه

دستاورد	هدف	چارچوب	رفرنس
بهبود بهره‌وری	Energy Optimization	GA-PSO, MPSO, CSO	[6]
کاهش مصرف انرژی	Multi-objective QoS	NSGA-II	[17]

این دو مقاله بر روی بهینه‌سازی انرژی، هزینه، منابع تمرکز کرده‌اند.

^۱ Hybrid Approaches

^۲ Metaheuristic and Multi-objective Optimization Methods

گروه پنجم: چارچوب‌های ارکستریشن و کنترل هوشمند^۱

این گروه شامل پژوهش‌هایی است که بر توسعه یا بهبود چارچوب‌های ارکستریشن و کنترل ترافیک در محیط‌های میکروسرویسی تمرکز دارند. هدف این رویکردها، افزایش پایداری APIها، بهینه‌سازی توزیع بار و ارتقای کیفیت سرویس از طریق کنترل هوشمند و مانیتورینگ بلادرنگ است. در این گروه مدل‌هایی مانند Kubernetes، Istio و Docker پیاده‌سازی شدند که مستقیم در چارچوب‌های مدیریت میکروسرویس‌ها دخالت دارند. محققین در مقالات خود از جمله [۲۲] به طور مستقیم در خوشه‌های Kubernetes آزمایشاتی انجام داده و نشان دادند که می‌توانند با تحلیل داده‌ها، تأخیر را تا ۲۹٪ و مصرف منابع را تا ۱۸٪ کاهش دهند.

جدول (۶-۲) خلاصه مقالات مربوط به چارچوب‌های ارکستریشن و کنترل هوشمند

دستاوردها	تمرکز	چارچوب	رفرنس
شناسایی محدودیت‌ها	Load Balancing	Kubernetes / Docker	[5]
تصمیم‌گیری بلادرنگ	API Gateway Eval	AGE	[8]
افزایش پایداری	Traffic Control	Service Mesh	[14]
Latency کمتر	Load Distribution	ML-based LB	[15]
تحلیل جامع	AI in Cloud	Review	[16]
API Latency -29٪	Adaptive Control	STaleX	[22]

تمرکز این گروه مقالات بر روی Kubernetes، Service Mesh، API Gateway است.

گروه ششم: مدل‌های ساختارمحور و وابستگی‌محور

روش‌های این گروه با در نظر گرفتن وابستگی‌های ساختاری و زنجیره‌های فراخوان بین میکروسرویس‌ها، تصمیم‌گیری تخصیص منابع را در سطح کل سیستم بهینه می‌کنند. این رویکردها با مدل‌سازی روابط زمانی-فضایی و گراف فراخوان سرویس‌ها، قادر به کاهش اثرات دومینویی و بهبود پایداری و تأخیر انتها به انتها هستند.

جدول (۷-۲) خلاصه مقالات مدل‌های ساختارمحور و وابستگی‌محور

دستاوردها	تحلیل ساختاری	چارچوب	رفرنس
SLA Violation جلوگیری از	Dependency Matrix	GNN + EM	[26]
Response +26٪	Critical Path	RL + Call-chain Analysis	[29]
Latency -28٪	Service Dependency	Hierarchical Neural Scaling	[33]

بر اساس نتایج مقالات مرور شده، می‌توان به این نتیجه رسید که روش‌های ترکیبی از نظر دقت پیش‌بینی، واکنش سریع و حفظ کیفیت سرویس، عملکرد بالاتری نسبت به سایر گروه‌ها دارند. از طرفی، روش‌های فراابتکاری و پیش‌بینی محور از نظر سادگی پیاده‌سازی و مصرف منابع، برتری کمتری را دارا هستند. جدول زیر خلاصه‌ای از تحلیل مقایسه‌ای بین گروه‌ها را نشان می‌دهد:

¹ Intelligent Orchestration Frameworks

جدول (۲-۸) تحلیل مقایسه‌ای مقالات

چالش‌ها	مزایا	الگوریتم‌های شاخص	گروه
واکنش کند در تغییرات ناگهانی	پیش‌بینی دقیق بار، بهبود تأخیر	LSTM, GRU, XGBoost	۱. پیش‌بینی محور
نیاز به داده زیاد، زمان همگرایی بالا	تصمیم‌گیری بلادرنگ، سازگاری پویا	DQN, PPO, Q-Learning	۲. تقویتی محور
پیچیدگی زیاد، هزینه محاسباتی بالا	پیش‌بینی و کنترل هم‌زمان، پایداری بالا	LSTM + RL, GNN + RL	۳. ترکیبی
مقیاس‌پذیری محدود	تنظیم بهینه منابع، کاهش انرژی	GA-PSO, NSGA-II	۴. فراابتکاری
وابستگی زیاد به تنظیمات زیرساخت	سازگاری با Kubernetes، قابلیت پیاده‌سازی واقعی	Smart HPA, STaleX	۵. ارکستریشن هوشمند
مدل‌سازی روابط زمانی-فضایی و گراف فراخوان سرویس‌ها	بهینه‌سازی تصمیم‌گیری تخصیص منابع	GNN, RL	۶. ساختار محور و وابستگی محور

در مجموع، می‌توان نتیجه گرفت که مسیر تحول تحقیقات، از مدل‌های ساده و سنتی پیش‌بینی بار به سمت سیستم‌های ترکیبی هوشمند با حلقه بازخورد بلادرنگ در حال پیشرفت است.



شکل (۲-۲) چارچوب کلی تصمیم‌گیری در روش‌های هوشمند مدیریت منابع API

این شکل چارچوب کلی تصمیم‌گیری مشترک در اغلب روش‌های هوشمند مدیریت منابع API را نمایش می‌دهد که شامل پایش سیستم، تحلیل داده‌ها، تصمیم‌گیری و اجرای آن است این جریان نشان می‌دهد که چگونه مدل‌های یادگیری در فرآیند کنترل و مقیاس‌پذیری سرویس‌ها بکار گرفته می‌شوند.

۲-۳-۲- شکاف تحقیقاتی^۱

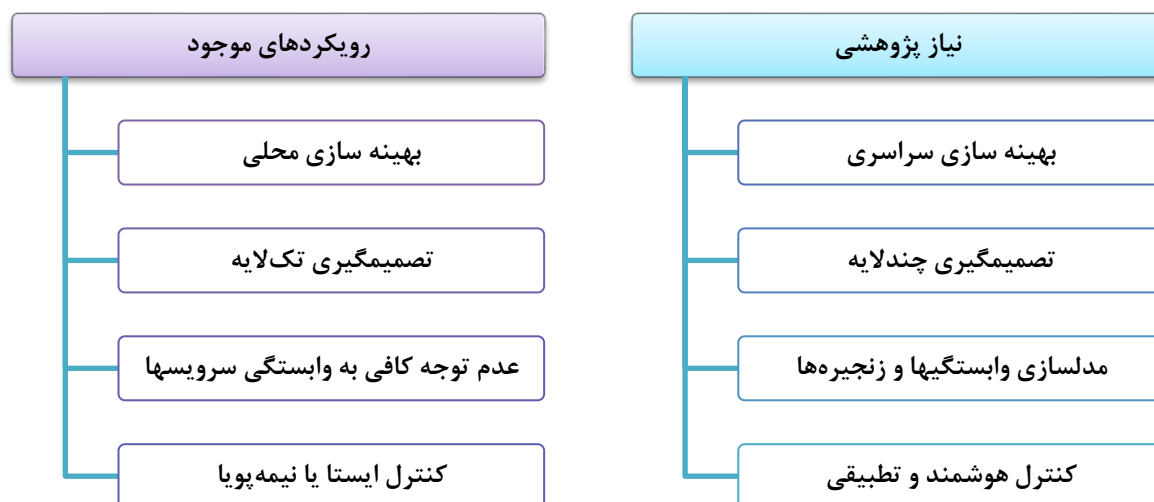
با مرور مقالات، به چند نکته مشترک و قابل‌توجه رسیدیم که نشان‌دهنده وجود خلأهای تحقیقاتی چشمگیری است:

- در بسیاری از تحقیقات اخیر مشاهده شد که محققین فقط بر روی یکی از مؤلفه‌های منفرد مسئله تمرکز کردند. برای مثال تمرکز بر روی پیش‌بینی بار با مدل‌های سری‌زمانی یا تمرکز بر روی فقط تصمیم‌گیری مبتنی بر RL برای مقیاس‌دهی پویا. با این حال وجود یک تلفیق هم‌زمان بین این دو رویکرد حس می‌شود؛ زیرا اجرای آن‌ها در بستر واقعی Kubernetes کمتر دیده شده است.
- اکثر تحقیقات تمرکز محدودی بر روی شاخص‌های منفرد داشتند، برای مثال بعضی از آن‌ها فقط تأخیر را گزارش کردند و بعضی دیگر مصرف CPU یا فقط تأثیر بر هزینه را بررسی کردند؛ لذا کمبود یک چارچوب ارزیابی چندبعدی و استاندارد که هم‌زمان QoS، کارایی منابع، هزینه‌های عملیاتی و رفتار در شرایط پیک را بسنجد، مشهود است.
- در بسیاری از تحقیقات RL برای تخصیص منابع، دو چالش عملی مهم کمتر موردتوجه قرار گرفته: (۱) زمان همگرایی و نیاز به داده آموزشی اولیه. (۲) انتقال‌پذیری سیاست آموزش‌دیده بین خوشه‌ها و بارهای متفاوت.
- ارتباط عملی و بلندرنج بین ماژول پیش‌بینی (مثلاً LSTM) و ماژول تصمیم‌گیری، طوری که پیش‌بینی‌های آینده بتواند به‌طور تأثیرگذاری در سیاست‌های مقیاس‌دهی وارد شده و پس از اعمال مقیاس، نتایج واقعی برای به‌روزرسانی هر دو ماژول استفاده گردد. چراکه نبود این بازخورد مانع رسیدن سیستم‌ها به یک رفتار تطبیقی پایدار در شرایط پرنوسان می‌شود.
- ویژگی‌های قابل‌توجهی از جمله مدیریت بارهای سنگین پردازشی، مسائل انرژی، هزینه و تعاملات میان‌خدمتی^۲، در بسیاری از مطالعات، به‌صورت سطحی یا جداگانه بررسی شدند. درحالی‌که تحقیقات جدیدتر نشان دادند که برای بارهای تخصصی نیاز به مکانیزم‌های ویژه هست؛ لذا باید بیشتر موردتوجه قرار بگیرند. از طرفی بررسی هم‌زمان مصرف انرژی و QoS در پیاده‌سازی‌های واقعی محدود است.
- محققان کمتری بر روی مسئله امنیت و تحمل خطا در فرآیندهای خودکار مقیاس‌دهی و تصمیم‌گیری هوشمند تمرکز کردند. این امر به‌قدری مهم و حیاتی است که اگر داده‌ها ناپیوسته یا لاگ‌ها ناقص باشند، سیاست‌های RL ممکن است موجب تصمیمات نامناسبی شوند.

¹ Research Gap

² service-to-service dependencies

- در تحقیقات مورد بررسی، خیلی کم بر روی مسئله قابلیت پیاده‌سازی و استقرار در محیط‌های صنعتی^۱ تمرکز شده است؛ منظور این است که نحوه ادغام الگوریتم با اکوسیستم موجود و استانداردهای عملیاتی برای اجرای پایدار کمتر بررسی شدند.



شکل (۲-۳) نمایش شکاف‌های پژوهشی در روش‌های موجود بهبود عملکرد API

این شکل شکاف‌های پژوهشی موجود در مطالعات پیشین را در مقایسه با نیازهای فعلی سامانه‌های میکروسرویسی نمایش را می‌دهد. تفاوت میان رویکردهای موجود و الزامات پژوهشی، ضرورت توسعه روش‌های هوشمند و یکپارچه‌تر را برجسته می‌کند.

با توجه به این خلأهای موجود، محققان باید در تحقیقات آینده با یک دید ترکیبی و کاربردی، در تلاش جهت رفع این ضعف‌های کلیدی باشند: طراحی و پیاده‌سازی یک مدل یکپارچه که از پیش‌بینی سری‌زمانی برای تولید سیگنال‌ها استفاده کند و اعمال سیاست‌های مقیاس‌دهی در Kubernetes. پژوهش‌های زیادی در این زمینه صورت گرفته که هر یک گوشه‌ای از مسائل بهبود عملکرد API در محیط‌های ابری را پوشش دادند، اما نبود راهکارهای عملی، بلادرنگ و قابل انتقال که پیش‌بینی، تصمیم‌گیری تقویتی، پیاده‌سازی در Kubernetes و ارزیابی چندمعیاره را به صورت هم‌زمان مدنظر قرار دهد، همچنان به عنوان یک شکاف تحقیقاتی نمایان است.

۲-۴- جمع‌بندی

در این فصل، مبانی نظری و مقالات پیشین پژوهش در ارتباط با موضوع تحقیق به صورت کامل مورد بررسی قرار گرفتند. در ابتدا، مفهوم رایانش ابری به عنوان یک زیرساخت اصلی محاسبات توزیع شده مدرن بیان شد. رایانش ابری امکان استفاده از منابع محاسباتی در مقیاس وسیع و بر اساس تقاضا را فراهم می‌کند و با کمک مدل‌های

¹ engineering readiness

خدماتی متنوع خود (IaaS, PaaS و SaaS) توانسته یک تحول اساسی در شیوه توسعه و ارائه نرم‌افزارها ایجاد کند. این فناوری هم موجب افزایش انعطاف‌پذیری شده و هم موجب کاهش هزینه‌های سرمایه‌گذاری و تسهیل در نگهداری و مقیاس‌دهی سیستم‌ها. به‌طوری که خیلی از سازمان‌ها و شرکت‌ها به سمت بسترهای ابری در حرکت‌اند تا بتوانند از مزایای چابکی، دسترس‌پذیری بالا و مقیاس‌پذیری پویا بهره‌مند شوند.

در ادامه، معماری میکروسرویس‌ها به‌عنوان یکی از الگوهای کلیدی در توسعه نرم‌افزارهای ابری مورد بررسی قرار گرفت. گفتیم که این معماری با تقسیم سیستم‌های پیچیده به مجموعه‌ای از سرویس‌های مستقل، موجب کاهش پیچیدگی، تسهیل در توسعه و امکان استقرار جداگانه هر بخش می‌شود. هر سرویس وظیفه‌ای خاص بر عهده دارد و با سایر سرویس‌ها از طریق رابط‌های برنامه‌نویسی کاربردی (API) در تعامل است. باین‌حال، API‌ها نقش کلیدی در هماهنگی و ارتباط بین این سرویس‌ها را دارند. بررسی‌های ما نشان داد که در محیط‌های پرتراфик، عواملی مانند حجم بالای درخواست‌ها، نوسانات شبکه، محدودیت منابع یا تأخیر در پاسخ‌گویی می‌توانند موجب افت کیفیت خدمات شوند. همچنین، بحث امنیتی API‌ها نیز از اهمیت بالایی برخوردار است؛ زیرا هر آسیب‌پذیری در API می‌تواند موجب ایجاد یک مسیر نفوذ به سرویس‌های کلیدی سیستم گردد.

پس از تشریح مفاهیم زیرساختی، به معرفی الگوریتم‌های یادگیری تقویتی و شبکه‌های عصبی بازگشتی با حافظه کوتاه‌مدت (LSTM) پرداختیم. سپس در بخش مرور پژوهش‌های پیشین، مطالعات متعدد داخلی و بین‌المللی بررسی شدند. برخی از این مقالات به‌صورت متمرکز بر پیش‌بینی بار و تخصیص منابع با استفاده از مدل‌های سری زمانی مانند ARIMA، Prophet یا LSTM تمرکز داشتند. باقی روش‌ها از یادگیری تقویتی برای مدیریت پویا و خودکار منابع استفاده کردند. همچنین، تعدادی از آن‌ها به پیاده‌سازی مکانیزم‌های Auto-Scaling و بهبود HPA¹ در Kubernetes پرداختند. بطور کل می‌توان گفت که مرور جامع این مقالات نشان می‌دهد که بیشتر این کارها تنها بر یکی از جنبه‌های مسئله تمرکز کردند و از ترکیب هم‌زمان پیش‌بینی و تصمیم‌گیری در یک حلقه‌ی بازخورد واقعی بدور بوده‌اند.

¹ Horizontal Pod Autoscaler

فصل سوم

مدل پیشنهادی اجرای تحقیق

۳-۱- مقدمه

در این فصل روش‌شناسی پژوهش به صورت گام‌به‌گام تشریح می‌شود. هدف از ارائه این فصل معرفی یک مدل مفهومی برای طراحی، پیاده‌سازی و ارزیابی طرح پیشنهادی مبتنی بر ترکیب پیش‌بینی سری‌های زمانی با LSTM و تصمیم‌گیری مبتنی بر یادگیری تقویتی در بستر میکروسرویس‌های مبتنی بر Kubernetes است. ما ابتدا نوع و رویکرد پژوهش، منابع داده و چارچوب کلی مراحل اجرایی را بیان می‌کنیم؛ سپس ساختار و جزئیات فنی مدل پیشنهادی، معماری پیاده‌سازی، ابزارها و محیط آزمایشی تشریح می‌شود. همچنین روش‌های جمع‌آوری، معیارها و متریک‌های سنجش ارائه می‌شود.

۳-۲- طراحی مدل پیشنهادی

در این بخش، ساختار فنی و نحوه عملکرد مدل پیشنهادی پایان‌نامه تشریح می‌شود. هدف اصلی ما، پیش‌بینی هوشمند بار کاری API‌ها و بهبود تخصیص منابع در زمان اجرا در محیط‌های ابری مبتنی بر میکروسرویس است. این مدل به گونه‌ای طراحی شده که به صورت یک ماژول مستقل هوشمند در کنار سایر سرویس‌ها عمل کرده و بدون نیاز به تغییر در API‌های موجود، تصمیم‌های مقیاس‌پذیری را به سیستم ارکستریشن Kubernetes ارسال کند. مدل پیشنهادی از سه مؤلفه اصلی تشکیل شده است:

- ماژول پیش‌بینی بار^۱
- ماژول تصمیم‌گیر تخصیص منابع^۲
- ماژول مانیتورینگ و جمع‌آوری داده‌ها^۳

در ادامه، هر یک از این مؤلفه‌ها به تفصیل تشریح می‌شوند.

۳-۲-۱- ساختار کلی مدل پیشنهادی

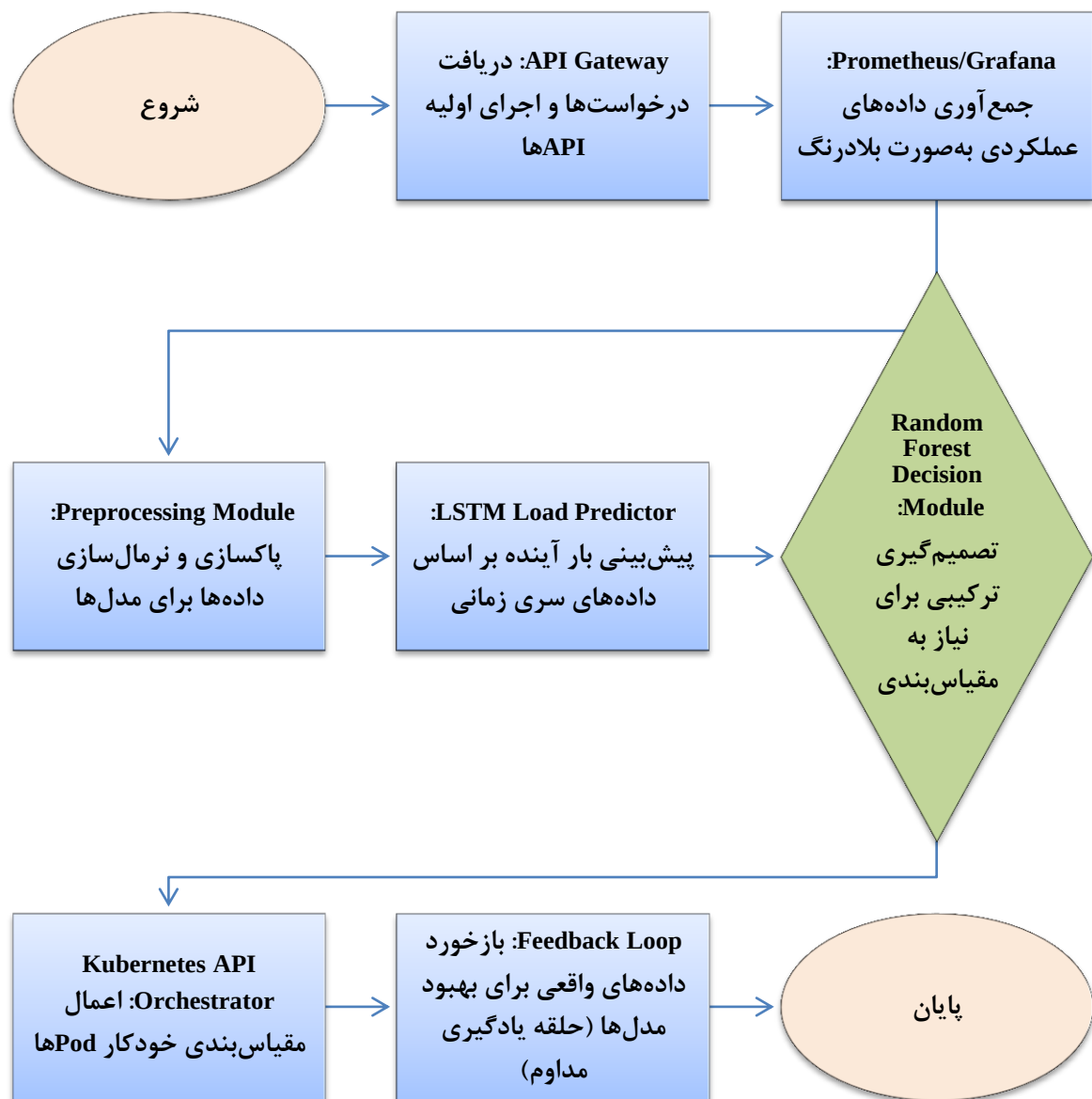
شکل (۳-۲) معماری کلی مدل پیشنهادی را نشان می‌دهد. در این معماری، جریان داده به صورت زیر است:

- ابزارهای مانیتورینگ مانند Prometheus و Grafana داده‌های عملکردی لحظه‌ای را از API‌ها جمع‌آوری می‌کنند (شامل نرخ درخواست، مصرف CPU/RAM، خطاها و تأخیر پاسخ).
- داده‌های جمع‌آوری شده پس از پیش‌پردازش به ماژول یادگیری ماشین ارسال می‌شوند.
- در این ماژول، مدل LSTM روند زمانی داده‌ها را تحلیل کرده و بار آینده سیستم را پیش‌بینی می‌کند.
- نتایج پیش‌بینی به مدل داده می‌شود تا با ترکیب سایر ویژگی‌های غیرزمانی (مانند نوع درخواست، اندازه پاسخ و خطاها) تصمیم نهایی در مورد وضعیت بار و نیاز به افزایش یا کاهش منابع گرفته شود.
- خروجی مدل تصمیم‌گیرنده، از طریق API‌های Kubernetes به ارکستریاتور ارسال شده و موجب افزایش یا کاهش Podها در زمان مناسب می‌گردد.
- نتایج اعمال شده مجدداً از طریق Prometheus پایش و به عنوان بازخورد به مدل برگردانده می‌شود.

^۱ Workload Prediction Module

^۲ Resource Allocation Decision Module

^۳ Monitoring and Feedback Module



شکل (۱-۳) فلوچارت مدل پیشنهادی

به این ترتیب، مدل پیشنهادی یک چرخه هوشمند از پیش‌بینی، تصمیم‌گیری، اجرا و بازخورد را در محیط ابری فراهم می‌کند.

۱-۲-۳- مازول دریافت درخواستها و اجرای اولیه APIها

مدل طراحی شده دارای ویژگی متمایز است از جمله غیرتهاجمی بودن بدون نیاز به تغییر در کد یا ساختار APIها عمل می کند، و نیز می تواند سازوکاری هوشمند برای تخصیص پویا و پیش بینانه منابع ارائه دهد. و شایان ذکر است این مدل باعث بهبود کارایی می شود.

۳-۲-۲- مازول جمع آوری داده های عملکردی به صورت بلادرنگ

این مازول وظیفه نظارت مداوم بر عملکرد مدل و سیستم را بر عهده دارد. چراکه ابزار Prometheus به صورت مستمر شاخص های عملکردی را از سرویس ها جمع آوری می کند. سپس داده ها در Grafana مصورسازی می شوند تا روند تغییرات در طول زمان قابل مشاهده باشد.

این مازول نقش حیاتی در پایداری و خودانطباق سیستم ایفا می کند. ابزار Prometheus شاخص های عملکردی را به صورت بلادرنگ جمع آوری می کند و داده ها از طریق Grafana مصورسازی می شوند.

۳-۲-۳- مازول پاکسازی و نرمال سازی داده ها برای مدل ها

داده ها از Prometheus در بازه های زمانی $\Delta t = 5 \sim 10$ ثانیه نمونه برداری می شوند. برای جلوگیری از غلبه مقیاس ویژگی ها، از نرمال سازی Min-Max استفاده می شود:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

۳-۲-۴- مازول پیش بینی بار کاری

این مازول هسته یادگیری زمانی مدل پیشنهادی را تشکیل می دهد و وظیفه آن پیش بینی الگوی بار کاری آینده در محیط های میکروسرویس محور است. با توجه به ماهیت سری زمانی داده های بار API و وجود وابستگی های زمانی کوتاه مدت و بلندمدت، از شبکه LSTM استفاده شده است.

ورودی مدل

بردار ورودی در زمان t به صورت زیر تعریف می شود:

$$x_t = [r_t, d_t, c_t, m_t, e_t] \quad (2)$$

که در آن:

- r_t : نرخ درخواستها (Requests per Second)

- d_t : میانگین تأخیر پاسخ

- c_t : مصرف CPU

- m_t : مصرف حافظه RAM

- e_t : نرخ خطاهای HTTP (4x/5x)

ساختار LSTM

مدل شامل دولایه LSTM و یک لایه Dense خروجی است. معادلات داخلی هر سلول LSTM به صورت زیر تعریف می شوند:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (4)$$

$$C_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (5)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (6)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (7)$$

$$h_t = o_t \odot \tanh(C_t) \quad (8)$$

که در آن:

- f_t, i_t, o_t : دروازه‌های فراموشی، ورودی و خروجی
- C_t : حافظه‌ی سلول
- h_t : خروجی لایه در زمان t

نقش تابع فعال‌سازی تانژانت (tanh) در LSTM

در شبکه LSTM، تابع فعال‌سازی تانژانت هیپربولیک (tanh) برای نگاشت مقادیر حافظه سلولی C_t و خروجی پنهان h_t به بازه $[-1, 1]$ استفاده می‌شود. این عمل باعث می‌شود مقادیر بسیار بزرگ یا بسیار کوچک که می‌توانند منجر به انفجار یا فروپاشی گرادیان شوند، کنترل گردند.

در واقع، خروجی سلول:

$$h_t = o_t \odot \tanh(C_t) \quad (9)$$

با استفاده از tanh، مقادیر حافظه سلولی به شکل غیرخطی و محدود در بازه $[-1, 1]$ تبدیل می‌شوند و این موجب پایداری آموزش و پیش‌بینی دقیق‌تر الگوهای بار کاری API می‌شود. به علاوه، این تابع غیرخطی به شبکه اجازه می‌دهد وابستگی‌های زمانی بلندمدت و کوتاه‌مدت را به‌طور مؤثری در داده‌های سری‌زمانی شناسایی کند، به طوری که ماژول تصمیم‌گیری تخصیص منابع بتواند به‌صورت هوشمند و پایدار، Podها و منابع CPU/RAM را مقیاس‌بندی کند.

تابع هزینه و آموزش

هدف مدل، پیش‌بینی بار کاری آینده $\hat{y}_{t+k}$ در افق زمانی k (مثلاً ۳۰ یا ۶۰ ثانیه) است. تابع هزینه به‌صورت ^۱ MSE تعریف می‌شود:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (10)$$

بهبود وزن‌ها با استفاده از Adam Optimizer انجام می‌شود.

خروجی مدل

^۱ Mean Squared Error (MSE)

خروجی مدل، مقدار پیش‌بینی‌شده‌ی بار کاری آینده است:

$$\hat{W}_{t+k} = f(x_{t-n:t}) \quad (11)$$

که مبنای تصمیم‌گیری ماژول تخصیص منابع قرار می‌گیرد.

۳-۲-۵- ماژول تصمیم‌گیری تخصیص منابع^۱

باوجود توانایی LSTM در پیش‌بینی بار آینده، تصمیم‌گیری بهینه برای مقیاس‌پذیری نیازمند در نظر گرفتن عوامل غیرزمانی و شرایط لحظه‌ای سیستم است. به همین منظور، از الگوریتم Random Forest به‌عنوان یک مدل تصمیم‌گیر ترکیبی استفاده شده است.

ورودی ماژول

بردار ویژگی تصمیم‌گیری به‌صورت زیر تعریف می‌شود:

$$z_t = [\hat{W}_{t+k}, c_t, m_t, d_t, e_t, s_t, p_t] \quad (12)$$

که در آن:

- $\hat{W}_{t+k}$: بار پیش‌بینی‌شده توسط LSTM
- s_t : نوع سرویس (Service Type)
- p_t : سطح اولویت یا اهمیت درخواست‌ها

مدل Random Forest

جنگل تصادفی شامل مجموعه‌ای از M درخت تصمیم مستقل است:

$$F = \{T_1, T_2, \dots, T_M\} \quad (13)$$

خروجی نهایی با رأی‌گیری اکثریت (برای طبقه‌بندی) یا میانگین‌گیری (برای رگرسیون) محاسبه می‌شود:

$$\hat{y} = \frac{1}{M} \sum_{i=1}^M T_i(Z_t) \quad (14)$$

سطوح تصمیم‌گیری

خروجی مدل یکی از حالات زیر است:

- Low Load = عدم تغییر منابع
- Moderate Load = حفظ وضعیت فعلی
- High Load = افزایش Pod یا منابع تخصیص‌یافته

خروجی ماژول

تصمیم نهایی به‌صورت فرمان مقیاس‌پذیری (Scale Up / Scale Down) تولید شده و برای Kubernetes ارسال می‌شود.

جدول (۳-۱) توضیح نمادهای مورد استفاده

¹ Random Forest Decision Module

نماد	توضیح
Δt	فاصله زمانی نمونه برداری (۵ تا ۱۰ ثانیه)
x	مقدار اولیه ویژگی
x_{min}	حداقل مقدار ویژگی
x_{max}	حداکثر مقدار ویژگی
x'	مقدار نرمال سازی شده با Min-Max
x_t	بردار ورودی مدل LSTM در زمان t
r_t	نرخ درخواستها (Requests per Second)
d_t	میانگین تأخیر پاسخ
c_t	مصرف CPU
m_t	مصرف RAM
e_t	نرخ خطاهای HTTP
f_t	دروازه فراموشی LSTM
i_t	دروازه ورودی LSTM
O_t	دروازه خروجی LSTM
$C_{-}\{t-1\}$	حافظه سلولی در زمان قبلی
σ	تابع فعال سازی سیگموید
$\tanh$	تابع فعال سازی تانژانت هیپربولیک
y_i	مقدار واقعی
L	تابع هزینه MSE
N	تعداد نمونه ها
$W^{\wedge}\{t+k\}$	بار پیش بینی شده برای زمان آینده
z_t	بردار ویژگی ماژول تصمیم گیری
s_t	نوع سرویس
p_t	سطح اولویت
$F = \{T_1, \dots, T_M\}$	مجموعه درخت های جنگل تصادفی
T_i	درخت تصمیم i ام
M	تعداد کل درخت ها
$\hat{y}$	خروجی نهایی Random Forest
ϵ	حلقه بازخورد

۳-۲-۶- تعامل بین ماژول ها و سیستم ارکستریشن Kubernetes

مدل پیشنهادی از طریق Kubernetes API Server با سیستم ارکستراسیون در تعامل است. خروجی ماژول تصمیم گیری به صورت پیام JSON و از طریق RESTful API ارسال می شود:

```
{
  "replicas": 6,
  "cpu_limit": "1000m",
```

```
"memory_limit": "1024Mi"
}
```

این تعامل دوطرفه شامل:

- دریافت داده‌های عملکردی از Kubernetes
- اعمال تصمیمات مقیاس‌پذیری بر Podها

می‌باشد. نتیجه این ارتباط پویا، کاهش تأخیر پاسخ، افزایش QoS و بهبود بهره‌وری منابع در مواجهه با نوسانات شدید بار کاری است.

۳-۲-۷- مازول مانیتورینگ و بازخورد^۱

این مازول در فواصل زمانی مشخص، داده‌های جدید برای یادگیری مجدد مدل‌های LSTM و Random Forest استفاده می‌شوند.^۲ این چرخه بازخوردی باعث می‌شود تا مدل به‌مرور زمان با شرایط جدید شبکه و تغییر الگوهای بار سازگار گردد.

حلقه بازخورد

پس از هر بازه اجرایی τ ، داده‌های واقعی سیستم با خروجی‌های پیش‌بینی شده مقایسه می‌شوند:

$$\epsilon_t = |W_t - \hat{W}_t| \quad (۱۵)$$

در صورتی که خطای پیش‌بینی از آستانه مشخص $\epsilon_{\max}$ فراتر رود، فرآیند بازآموزی^۳ فعال می‌شود.

به‌روزرسانی مدل‌ها

داده‌های جدید برای:

- به‌روزرسانی وزن‌های LSTM
- اصلاح ساختار تصمیم‌گیری Random Forest

استفاده می‌شوند. این چرخه باعث سازگاری تدریجی مدل با تغییر الگوهای بار می‌شود.

۳-۲-۸- مزایای مدل پیشنهادی

مدل طراحی شده دارای چند ویژگی متمایز است از جمله:

- غیرتهاجمی بودن^۴: بدون نیاز به تغییر در کد یا ساختار APIها عمل می‌کند.
- هوشمندی ترکیبی: بهره‌گیری هم‌زمان از توانایی LSTM در تحلیل سری‌های زمانی و Random Forest در تصمیم‌گیری چندمعیاره.
- سازگاری پویا: یادگیری مداوم از داده‌های جدید با چرخه بازخورد Prometheus.
- استقلال از پلتفرم: قابلیت اجرا در هر محیط Kubernetes بدون وابستگی به ارائه‌دهنده خاص.
- بهبود QoS: کاهش تأخیر و افزایش پایداری سیستم.

^۱ Monitoring & Feedback Module

^۲ Continuous Learning

^۳ Retraining

^۴ Non-intrusive

مدل پیشنهادی این پژوهش با ترکیب دو الگوریتم قدرتمند LSTM و جنگل تصادفی و یکپارچگی کامل با اکوسیستم Kubernetes، می‌تواند سازوکاری هوشمند برای تخصیص پویا و پیش‌بینانه منابع ارائه دهد. این مدل علاوه بر بهبود کارایی، موجب کاهش هزینه و افزایش قابلیت اطمینان در سرویس‌های میکروسرویسی می‌شود.

۳-۳- محیط پیاده‌سازی و ابزارهای مورد استفاده

در این پژوهش، هدف اصلی ما پیاده‌سازی مدل پیشنهادی در یک محیط سبک و قابل اجرا بر روی سیستم شخصی بوده تا بتوانیم آزمایش، بازتولید و تحلیل دقیق نتایج را انجام دهیم. برای رسیدن به همین هدف، از مجموعه‌ای از ابزارهای پایتونی و کتابخانه‌های متن‌باز استفاده خواهیم کرد که دیگر نیازی به زیرساخت ابری یا Kubernetes واقعی نداشته باشیم.

۳-۳-۱- محیط سخت‌افزاری

همه مراحل پیاده‌سازی و اجرای آزمایش‌ها بر روی سیستم شخصی انجام شده است. مشخصات سخت‌افزاری سیستم در جدول زیر آمده است:

جدول (۳-۲) مشخصات سخت‌افزاری

مشخصات	مؤلفه
Intel® Core™ i7	پردازنده (CPU)
16 GB DDR4	حافظه (RAM)
512 GB SSD	فضای ذخیره‌سازی (Storage)
Windows 10 (64-bit)	سیستم عامل (OS)
Visual Studio Code	محیط توسعه (IDE)

۳-۳-۲- محیط نرم‌افزاری

در پیاده‌سازی مدل، از زبان پایتون و چندین کتابخانه‌ی استاندارد برای تولید داده، آموزش مدل، و تحلیل نتایج استفاده خواهیم کرد. فهرست ابزارها و نسخه‌ها در جدول زیر آمده است:

جدول (۳-۳) مشخصات نرم‌افزاری

ابزار / کتابخانه	نسخه	کاربرد
Python	3.10	زبان برنامه‌نویسی اصلی پروژه
NumPy / Pandas	آخرین نسخه پایدار	تولید و پردازش داده‌ها
scikit-learn	1.4	پیاده‌سازی مدل Random Forest
TensorFlow / Keras	2.13	ساخت مدل LSTM برای پیش‌بینی بار

رسم و تحلیل نمودارهای عملکرد	3.8 / 0.13	Matplotlib / Seaborn
تولید داده‌های ترافیکی مصنوعی	2.16	Locust / Faker (اختیاری)
ذخیره و بارگذاری داده‌ها	–	CSV File I/O

پروژه به صورت ماژولار طراحی می‌گردد تا هر بخش (تولید داده، پیش‌بینی، تحلیل، تصمیم‌گیری) را بتوان به‌طور مستقل اجرا و آزمایش کرد.

۳-۳-۴- فرآیند پیاده‌سازی و اجرا

فرآیند اجرای مدل در چهار مرحله ساده انجام می‌شود:

۱. **تولید داده:** با استفاده از کتابخانه‌های NumPy و Faker، داده‌های مصنوعی شامل تعداد درخواست‌ها،

تأخیر پاسخ، و مصرف CPU برای دوره‌های زمانی مختلف تولید می‌شوند.

• **آموزش مدل‌های یادگیری ماشین:** مدل LSTM با داده‌های سری زمانی آموزش می‌بیند تا

الگوی تغییرات بار کاری را پیش‌بینی کند و مدل Random Forest نیز به‌عنوان پیش‌بینی‌گر

مقایسه‌ای آموزش داده می‌شود تا دقت نتایج افزایش یابد.

۲. **تصمیم‌گیری در مقیاس‌پذیری:** خروجی هر دو مدل (میزان بار پیش‌بینی‌شده) ترکیب می‌شود تا تصمیم

گرفته شود که چند نمونه از API باید فعال بماند که این تصمیم به صورت ساده در قالب زیر انجام می‌شود:

```
if predicted_load > threshold_high:
    scale_up()
elif predicted_load < threshold_low:
    scale_down()
else:
    keep_current_state()
```

۳. **نمایش و تحلیل نتایج:** در نهایت، نمودارهای مقایسه‌ای از بار واقعی و پیش‌بینی‌شده رسم و عملکرد

مدل‌ها بر اساس معیارهایی مانند MAE، MSE و R^2 Score را مورد تحلیل قرار می‌دهیم.

محیط پیاده‌سازی انتخابی، ضمن قابل اجرا بودن بر روی سیستم شخصی، امکان آزمایش دقیق مدل‌های یادگیری

ماشین و تحلیل داده‌های عملکردی API را فراهم کرده، این روند به ما اجازه می‌دهد تا نتایج واقعی از مدل‌های

LSTM و Random Forest را دریافت کنیم، تصمیمات مقیاس‌پذیری را بر اساس پیش‌بینی‌ها شبیه‌سازی و در

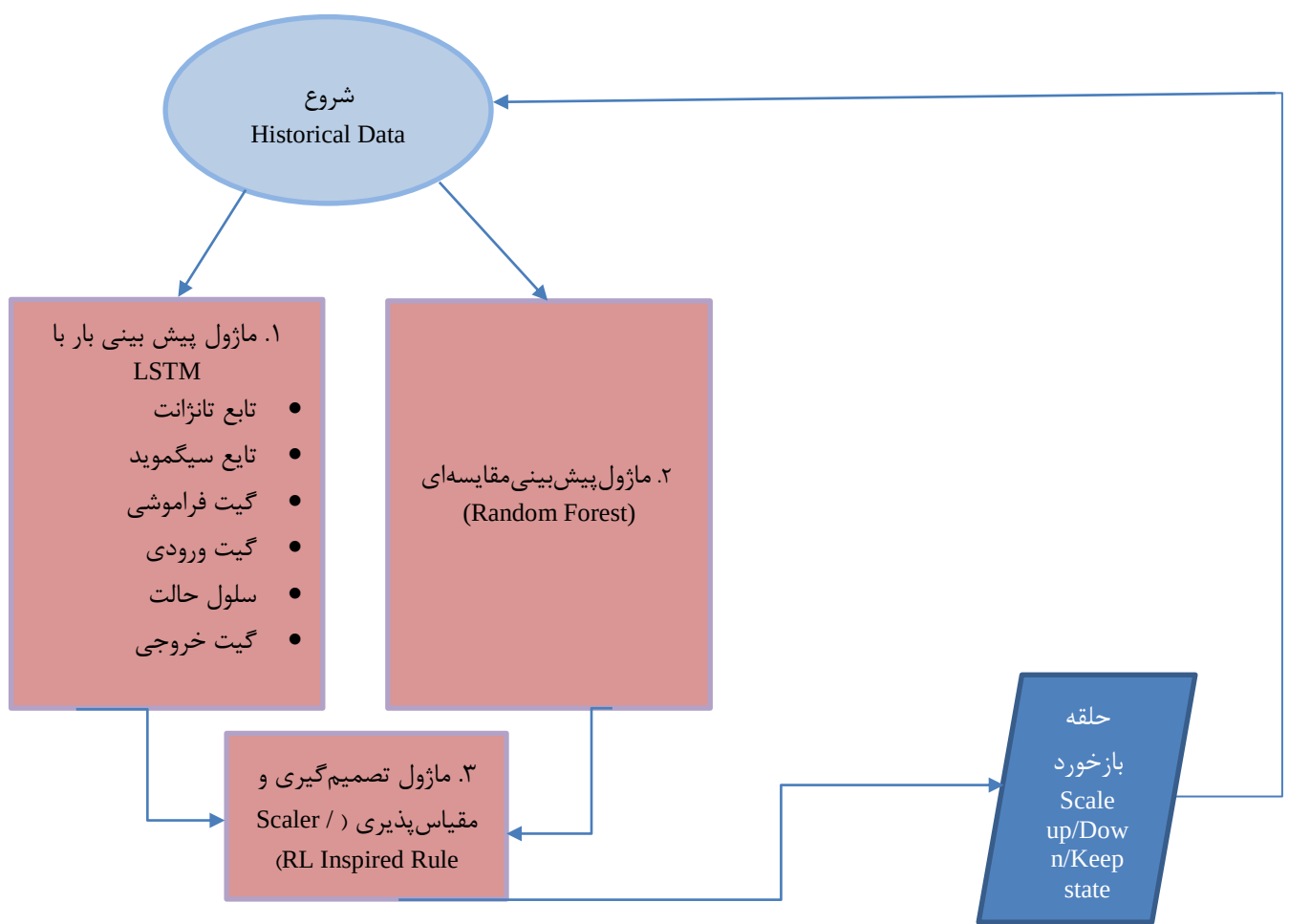
صورت نیاز، به سادگی مدل را به محیط Docker یا Kubernetes منتقل کنیم.

۳-۴- الگوریتم پیشنهادی

۳-۴-۱- توضیح الگوریتم

مدل پیشنهادی شامل سه ماژول اصلی است که به ترتیب فلوچارت زیر به صورت حلقه‌ای با هم تعامل دارند:

¹ Data Generation



شکل (۳-۲) فلوچارت سه ماژول اصلی

۱. **ماژول پیش‌بینی بار با LSTM** : وظیفه آن پیش‌بینی بار کاری API در بازه‌های زمانی آینده بر اساس داده‌های گذشته (تعداد درخواست، تأخیر، CPU و RAM) است و خروجی آن نیز مقدار پیش‌بینی‌شده بار آینده است.

LSTM به‌عنوان یکی از ابزارهای مهم واقع در زمینه یادگیری عمیق و پردازش داده‌های دنباله‌ای و سری زمانی شناخته می‌شود و در بسیاری از برنامه‌ها و وظایف مختلف مورد استفاده قرار می‌گیرد؛ به این صورت کار می‌کند: کلمات اول به بردارهای قابل خواندن با ماشین تبدیل می‌شوند سپس دنباله بردارها را یک‌به‌یک پردازش می‌کند. در حین پردازش، حالت پنهان قبلی را به مرحله بعدی دنباله منتقل می‌کند. حالت پنهان به‌عنوان حافظه در شبکه‌های عصبی عمل می‌کند. اطلاعات مربوط به داده‌های قبلی را که شبکه قبلاً دیده بود، در اختیار دارد. یک سلول به‌عنوان یکی از مهم‌ترین مؤلفه‌های شبکه عصبی بازگشتی به کار می‌رود و نقش بسیار اساسی در پردازش داده‌های دنباله‌ای دارد. در این سلول، ابتدا حالت پنهان قبلی با ورودی فعلی ترکیب می‌شوند. این ترکیب معمولاً به‌صورت عملیات جمع وزن‌دار انجام می‌شود. درواقع، حالت پنهان قبلی که حاوی اطلاعاتی از ورودی‌های گذشته است، با ورودی فعلی که اطلاعاتی از وضعیت فعلی دارد، ترکیب می‌شوند.

این عملیات ترکیب باعث می‌شود که سلول اطلاعات مربوط به حالت قبلی و ورودی جدید را در خود نگه دارد. سپس، بردار ترکیب‌شده از تابع تانژانت عبور می‌کند. تابع تانژانت به‌عنوان یک تابع غیرخطی عمل می‌کند و از این‌رو به شبکه عصبی امکان می‌دهد که اطلاعات را به‌صورت غیرخطی پردازش کند. این اهمیت دارد؛ زیرا مسائل واقعی و پیچیده اغلب خصوصیات غیرخطی دارند. درنهایت، خروجی این تابع تانژانت به‌عنوان حالت پنهان جدید یا حافظه شبکه شناخته می‌شود. این عملیات به شبکه عصبی بازگشتی امکان می‌دهد که الگوها و وابستگی‌های پیچیده در داده‌های دنباله‌ای را کشف کند و در مسائلی مانند ترجمه ماشینی، مدل‌های زبانی، و پیش‌بینی دنباله‌ها عملکرد بهتری داشته باشد.

تابع تانژانت: تابع تانژانت به‌عنوان یک تابع فعال‌سازی^۱ معمولاً استفاده نمی‌شود تا مقادیر را جمع نکند و بین ۱ و -۱ تنظیم نشوند. به جای تابع تانژانت، توابع دیگری مانند تابع فعال‌سازی ReLU^۲، تانژانت هایپربولیک^۳، و تابع فعال‌سازی سیگموئید^۴ استفاده می‌شوند. در این شبکه، ورودی‌های متوالی به شبکه داده می‌شوند و در هر مرحله از آموزش و پیش‌بینی، یک ماتریس وزن نیز وجود دارد. فرض می‌شود مقدار یکی از وزن‌های این ماتریس به‌طور غیرمنتظره‌ای بسیار بزرگ شود، به‌عنوان مثال ۱۰۰۰۰. در مراحل مختلف محاسبات، این وزن بزرگ بارها در عملیات ماتریسی مورد استفاده قرار می‌گیرد. به علت این وزن بزرگ، مقدار گرادیان‌ها نیز بسیار بزرگ می‌شود.

حال اگر مقدار گرادیان‌ها به اندازه‌ی این بزرگی باقی بماند و به مراحل بعدی منتقل شود، پدیده انفجار گرادیان رخ می‌دهد. این به معنای این است که به ازای هر مرحله از زمان، مقدار وزن‌ها بسیار بزرگ‌تر می‌شود تا به نقطه‌ای برسد که شبکه عصبی دیگر نتواند به‌درستی آموزش داده شود. این موضوع منجر به عملکرد غیرپایدار و از دست رفتن اطلاعات می‌شود. تابع تانژانت تضمین می‌کند که مقادیر بین ۱- تا ۱ باقی بماند؛ بنابراین خروجی شبکه عصبی را تنظیم می‌کند. این نوع از شبکه‌ها، در شرایط خاصی می‌توانند به عملکرد خوبی دست پیدا کنند. این شرایط معمولاً در مواردی رخ می‌دهند که داده‌ها به‌صورت دنباله‌های کوتاه و وابستگی‌های زمانی محدودی دارند. در ادامه به بیان توابع مورد استفاده خواهیم پرداخت.

سیگموئید: توابع فعال‌سازی مانند سیگموئید به شبکه‌های عصبی امکان می‌دهند که اطلاعات را با دقت مدیریت کنند و تصمیم‌گیری‌های مرتبط با اهمیت داده‌ها را انجام دهند. این توابع فعال‌سازی به شبکه‌ها کمک می‌کنند تا اطلاعات مهم را حفظ کنند و از اطلاعات ناچیز گذشته یا بی‌اهمیت صرف‌نظر کنند. توابع فعال‌سازی مانند سیگموئید، با تبدیل ورودی‌های شبکه به مقادیر بین ۰ و ۱، می‌توانند اطلاعات را فیلتر کرده و به شبکه امکان می‌دهند تا تصمیم‌گیری کنند کدام اطلاعات باید در نظر گرفته شوند و کدام اطلاعات باید نادیده گرفته شوند. در بررسی یک سلول از شبکه عصبی بازگشتی، به سه گیت مختلف پرداخته می‌شود. این گیت‌ها شامل گیت فراموشی، گیت ورودی و گیت خروجی هستند که جریان اطلاعات را در سلول حافظه کوتاه‌مدت تنظیم می‌کنند.

^۱ activation function

^۲ Rectified Linear Unit

^۳ Hyperbolic Tangent

^۴ Sigmoid

گیت فراموشی: گیت فراموشی در یک شبکه عصبی بازگشتی نقش مهمی ایفا می‌کند. این گیت تصمیم می‌گیرد که اطلاعاتی که در حالت پنهان قبلی و اطلاعاتی که از ورودی فعلی به شبکه می‌آید، کدام اطلاعات باید در حافظه ماندگار (حافظه کوتاه‌مدت) نگهداری شوند و کدام اطلاعات باید فراموش شوند. این تصمیم‌گیری با استفاده از تابع سیگموئید انجام می‌شود که مقادیر را به فرمتی بین ۰ تا ۱ تنظیم می‌کند. مقادیر نزدیک به صفر نمایانگر فراموش کردن کامل اطلاعات هستند، درحالی‌که مقادیر نزدیک به ۱ نمایانگر حفظ اطلاعات هستند.

گیت ورودی: در فرآیند به‌روزرسانی وضعیت سلول یک شبکه عصبی بازگشتی، از گیت ورودی بهره می‌بریم. ابتدا، حالت پنهان قبلی و ورودی فعلی به یک تابع سیگموئید منتقل می‌شوند. این تابع سیگموئید مسئول تصمیم‌گیری در مورد اهمیت اطلاعات است. اعداد نزدیک به صفر نمایانگر "نادیده‌گرفتن" و اعداد نزدیک به ۱ نمایانگر "نگهداشتن" اطلاعات هستند. سپس، خروجی تابع تانژانت را با خروجی تابع سیگموئید ضرب می‌کنیم. خروجی سیگموئید تصمیم می‌گیرد که آیا اطلاعات حاصل از تابع تانژانت مهم هستند یا نه. اگر خروجی سیگموئید به سمت ۱ نزدیک باشد، این به این معناست که اطلاعات مهم هستند و باید نگهداشته شوند. در غیر این صورت، اطلاعات به‌عنوان غیرمهم معلوم می‌شوند و فراموش می‌شوند. این ترکیب از توابع سیگموئید و تانژانت به شبکه اجازه می‌دهد تا مدیریت کند کدام اطلاعات را می‌تواند فراموش کند.

سلول حالت: در فرآیند محاسبه سلول حالت در یک شبکه عصبی بازگشتی، ابتدا حالت سلولی را با وکتور فراموشی ضرب می‌کنیم. اگر مقادیر وکتور فراموشی نزدیک به صفر باشند، این به این معناست که باید اطلاعات را در سلول حالت کاهش دهیم. سپس، خروجی گیت ورودی را دریافت می‌کنیم و عمل جمع نقطه‌به‌نقطه را انجام می‌دهیم. این کار باعث به‌روزرسانی حالت سلولی می‌شود و حالت جدید سلول را به ما تحویل می‌دهد. این فرآیند به شبکه عصبی امکان می‌دهد تا اطلاعات جدید را با استفاده از خروجی گیت ورودی به حالت سلولی اضافه کند و حالت را به‌روز کند.

گیت خروجی: درنهایت، گیت خروجی نقش مهمی در تعیین حالت پنهان بعدی دارد. حالت پنهان قبلی حاوی اطلاعاتی از ورودی‌های قبلی است و این حالت پنهان نیز برای پیش‌بینی‌های آینده مورد استفاده قرار می‌گیرد. فرآیند انجام شده به این صورت است: ابتدا، حالت پنهان قبلی و ورودی فعلی به یک تابع سیگموئید منتقل می‌شوند. سپس، حالت سلول تازه اصلاح‌شده را به تابع تانژانت منتقل می‌کنیم. خروجی تانژانت را با خروجی سیگموئید ضرب می‌کنیم تا تصمیم بگیریم که حالت پنهان باید چه اطلاعاتی را داشته باشد. نتیجه این عملیات، حالت پنهان جدید است که به مرحله بعدی منتقل می‌شود. این گیت‌ها به شبکه امکان می‌دهند تا از اطلاعات پنهان مناسبی برای پیش‌بینی‌های آینده استفاده کنند و تأثیرگذاری آن را کنترل کنند.

LSTM از یک جریان کنترل متفاوت به نام "گیت‌ها" استفاده می‌کند. این گیت‌ها شامل گیت فراموشی، گیت ورودی و گیت خروجی هستند که هر یک وظایف خاص خود را دارند. این گیت‌ها به LSTM امکان

می‌دهند تا اطلاعات را در حافظه کوتاه‌مدت مدیریت کرده و تصمیم‌گیری‌هایی در مورد حفظ یا فراموش کردن اطلاعات بگیرد.

۲. ماژول پیش‌بینی مقایسه‌ای (Random Forest)

- وظیفه آن پیش‌بینی مقایسه‌ای بار به‌منظور بررسی صحت و تقویت تصمیم‌گیری است و خروجی آن به‌عنوان مقدار پیش‌بینی دوم تعریف شده است.

یک جنگل تصادفی، یک پیش‌بینی‌کننده است که از مجموعه‌ای از درختان رگرسیون تصادفی M تشکیل شده است. جهت اعمال الگوریتم برای تشخیص پیش‌بینی بار API، برای درخت j ام در خانواده، مقدار پیش‌بینی شده در نقطه پرس‌وجو x توسط $m_n(x; \Theta_j, D_n)$ بیان شده که در آن $\Theta_1, \dots, \Theta_M$ متغیرهای تصادفی مستقل هستند و به‌عنوان متغیر تصادفی عمومی Θ و مستقل از D_n توزیع شده‌اند. در عمل، از متغیر Θ برای باز نمونه مجموعه آموزش قبلی جهت رشد درختان انفرادی و انتخاب جهت‌های متوالی برای تقسیم استفاده می‌شود. از نظر ریاضی، تخمین درخت j ام به‌صورت زیر است:

$$m_n(x; \Theta_j, D_n) = \sum_{i \in Dn(\Theta_j)} \frac{1X_i \in An(x; \Theta_j, D_n) Y_i}{Nn(x; \Theta_j, D_n)} \quad (16)$$

که در آن $D_n^*(\Theta_j)$ مجموعه نقاط داده انتخابی قبل برای ساخت درخت است، $A_n(x; \Theta_j, D_n)$ سلول حاوی x است و $N_n(x; \Theta_j, D_n)$ تعداد نقاطی (از پیش انتخاب شده) است که در $A_n(x; \Theta_j, D_n)$ قرار می‌گیرند. در این مرحله، توجه می‌کنیم که درختان با هم ترکیب شدند تا تخمین جنگل (محدود) را تشکیل دهند:

$$m_{M,n}(x; \Theta_1, \dots, \Theta_M, D_n) = 1/m, S_{j=1}^M m_n(x; \Theta_j, D_n) \quad (17)$$

در بسته جنگل تصادفی R ، مقدار پیش‌فرض M (تعداد درختان جنگل) $n_{tree} = 500$ است. از آنجاکه M ممکن است به‌طور دلخواه بزرگ انتخاب شود (فقط با منابع محاسباتی موجود محدود است)، از دیدگاه مدل‌سازی، تمایل M به بی‌نهایت در نظر گرفته می‌شود و بجای رابطه ذکر شده در بالا، تخمین جنگل (نامتناهی) به‌صورت زیر در نظر گرفته می‌شود:

$$m_{\infty,n}(x; D_n) = E_{\Theta} [m_n(x; \Theta, D_n)] \quad (18)$$

در این تعریف، E_{Θ} انتظار را با توجه به پارامتر تصادفی Θ ، مشروط به D_n بیان می‌کند. در حقیقت، عملیات " $M \rightarrow \infty$ " با قانون تعداد زیاد توجه می‌شود که تقریباً مشروط به D_n است:

$$\lim_{M \rightarrow \infty} m_{M,n}(x; \Theta_1, \dots, \Theta_M, D_n) = m_{\infty,n}(x; D_n) \quad (19)$$

که برای راحتی بجای $m_{\infty,n}(x; D_n)$ را $m_{\infty,n}$ نوشتیم.

در ادامه برای یک بردار تصادفی p -بعدی $X = (X_1, \dots, X_p)^T$ نمایانگر ورودی واقعی یا متغیرهای پیش‌بینی‌کننده و یک متغیر تصادفی Y نشان‌دهنده پاسخ ارزش واقعی، فرض می‌کنیم توزیع مشترک ناشناخته $P_{XY}(X)$ هدف یافتن یک تابع پیش‌بینی $f(X)$ برای پیش‌بینی Y است. عملکرد پیش‌بینی توسط یک تابع از دست‌دادن $L(Y, f(X))$ تعیین می‌شود و برای به‌حداقل‌رساندن مقدار موردانتظار برای از دست‌دادن تعریف شده است:

$$E_{XY}(L(Y, f(X))) \quad (20)$$

که در آن اشتراک نشان‌دهنده انتظار با توجه به توزیع مشترک X و Y است. $L(Y, f(X))$ اندازه‌گیری میزان نزدیکی $f(X)$ به Y است؛ مقادیر $f(X)$ را که با Y فاصله زیادی دارند جریمه می‌کند. گزینه‌های معمول L خطای مربع از دست‌دادن $L(Y, f(X)) = (Y - f(X))^2$ برای رگرسیون و $zero-one loss$ برای طبقه‌بندی است:

$$L(Y, f(X)) = I(Y \neq f(X)) = \begin{cases} 0 & \text{if } Y = f(X) \\ 1 & \text{otherwise} \end{cases} \quad (21)$$

به نظر می‌رسد که خطای مربع از دست‌دادن انتظار مشروط را می‌دهد

$$f(X) = E(Y | X=x) \quad (22)$$

در غیر این صورت به‌عنوان تابع رگرسیون شناخته می‌شود. در وضعیت طبقه‌بندی، اگر مجموعه مقادیر ممکن Y توسط y نشان داده شود، $E_{XY}(L(Y, f(X)))$ برای $zero-one loss$ به حداقل برسد

$$f(x) = \arg \max_{y \in Y} P(Y = y | X = x) \quad (23)$$

در غیر این صورت به‌عنوان قانون بیز شناخته می‌شود.

گروه‌های سازنده f را از نظر مجموعه‌ای به اصطلاح "یادگیرندگان پایه" $h_1(x), \dots, h_J(x)$ نامیده می‌شوند و این یادگیرندگان پایه با هم جهت "پیش‌بینی‌کننده گروه" $f(x)$ ترکیب می‌شوند. در رگرسیون، یادگیرندگان پایه به‌طور میانگین هستند:

$$\hat{f}(x) = 1/J \sum_{j=1}^J h_j(x) \quad (24)$$

درحالی‌که در طبقه‌بندی، $f(x)$ بیشترین طبقه پیش‌بینی شده است (رأی دادن).

$$f(x) = \arg \max_{y \in Y} 1/J \sum_{j=1}^J h_j(x) \quad (25)$$

در جنگل‌های تصادفی ژامین یادگیر پایه، یک درخت است که با $h_j(X, j)$ مشخص می‌شود؛ که در آن θ_j مجموعه‌ای از متغیرهای تصادفی است و θ_j برای $j = 1, \dots, J$ مستقل است. اگرچه تعریف جنگل تصادفی بسیار کلی است، اما تقریباً به‌طور ثابت اجرا می‌شود.

برای آموزش داده‌ها هم $\{(x_1, y_1), \dots, (x_N, y_N)\} = \{(x_{i,1}, \dots, x_{i,p})^T\}$ داریم که در آن x_i نمایانگر پیش‌بینی‌های p و y_i هم نمایانگر پاسخ و یک درک خاص از θ_j است، درخت مناسب به‌صورت $h_j(x, D)$ نشان داده شده است. در عمل مؤلفه تصادفی θ_j صریحاً در نظر گرفته نمی‌شود بلکه به‌طور ضمنی به دو روش استفاده می‌شود: اول، هر درخت در یک نمونه بوت‌استرپ، مستقل از داده‌های اصلی، مناسب است. تصادفی بودن در نمونه‌گیری بوت‌استرپ یک بخش از θ_j را می‌دهد.

¹ Base Learners

² Ensemble Predictor

دوم، هنگام تقسیم یک گره، بهترین تقسیم بر روی یک زیرمجموعه‌ای (انتخاب به صورت رندم) از متغیرهای پیش‌بینی‌کننده m بجای همه پیش‌بینی‌کننده‌های p ، به طور مستقل در هر گره پیدا می‌شود. انتخاب تصادفی برای نمونه‌ای از پیش‌بینی‌ها، قسمت باقی‌مانده Θ_j را می‌دهد.

جدول (۳-۴) توضیح نمادهای مورد استفاده

نماد	توضیح
M	تعداد درخت‌های جنگل تصادفی
j	شماره درخت j ام
x	نقطه پرس‌وجو (Query Point)
$mn(x; \Theta_j, D_n)$	پیش‌بینی درخت j ام
Θ_j	متغیر تصادفی مستقل برای ساخت درخت j ام
Θ	متغیر تصادفی عمومی برای Randomization
D_n	مجموعه داده آموزشی
$D_n^*(\Theta_j)$	داده Bootstrap شده برای ساخت درخت j ام
$An(x; \Theta_j, D_n)$	سلول / گره نهایی شامل x
$Nn(x; \Theta_j, D_n)$	تعداد نقاط قرار گرفته در سلول An
$mM, n(x; \Theta_1, \dots, \Theta_M, D_n)$	پیش‌بینی جنگل محدود
$m\infty, n(x; D_n)$	پیش‌بینی جنگل نامتناهی
$X=(X_1, \dots, X_p)^T$	بردار تصادفی p -بعدی (متغیرهای ورودی)
Y	متغیر پاسخ حقیقی
P_{xy}	توزیع مشترک X و Y
$f(X)$	تابع پیش‌بینی
$L(Y, f(X))$	باز هزینه (Loss Function)
$(Y-f(X))^2$	خطای مربعی (Squared Loss)
$f(x)=1/J \sum_{j=1}^J h_j(x)$	ترکیب میانگین در رگرسیون
$f(x)=\operatorname{argmax}_y 1/J \sum_{j=1}^J (h_j(x)=y)$	رای‌گیری در طبقه‌بندی
$h_j(x, D)$	درخت j ام آموزش داده شده
(x_i, y_i)	داده‌های آموزشی نمونه i ام
$x_i=(x_{i,1}, \dots, x_{i,p})^T$	ویژگی‌های یک نمونه

درختان بدون هرس رشد می‌کنند. درختان نتیجه اگر پاسخ طبقه‌ای باشد (طبقه‌بندی) با رای‌گیری بدون وزن؛ یا اگر پاسخ متوالی باشد (رگرسیون) به طور متوسط بدون وزن، ترکیب می‌شوند، همان‌طور که در الگوریتم زیر توضیح داده شده است

۱. $D = \{(x_1, y_1), \dots, (x_N, y_N)\}$ نشان‌دهنده داده آموزشی است، با $x_i = (x_{i,1}, \dots, x_{i,p})^T$ برای $j=1$ to J .
۲. استفاده از نمونه بوت‌استرپ D_j از N سائز از D ایجاد یک نمونه بوت‌استرپ D_j از N سائز از D استفاده از پارتیشن‌بندی بازگشتی

باینری یک درخت متناسب کنید.

الف) با تمام مشاهدات در یک گره شروع کنید.

ب) مراحل زیر را به صورت بازگشتی برای هر گره تقسیم نشده تکرار کنید تا زمانی که معیار توقف برآورده شود:

(i) پیش‌بینی‌های m را به طور تصادفی از بین پیش‌بینی‌های موجود انتخاب کنید.

(ii) بهترین تقسیم باینری را در بین تمام تقسیمات باینری بر روی پیش‌بینی‌های m از مرحله (i) پیدا کنید.

(iii) گره را با استفاده از تقسیم مرحله (ii) به دو گره بعدی تقسیم کنید.

برای پیش‌بینی در یک نقطه جدید x ,

- فرمول (۲) برای رگرسیون
- فرمول (10) برای طبقه‌بندی

که در آن $h_j(x)$ پیش‌بینی متغیر پاسخ در x با استفاده از j امین درخت است.

شکل (۳-۳) الگوریتم RF در روش پیشنهادی

ما در این پایان‌نامه جهت پیش‌بینی بار API از جنگل تصادفی استفاده کردیم که برای تنظیم پارامترهای آن با توجه به هدف این پژوهش به صورت زیر عمل می‌کنیم:

برای درخت j ام در خانواده، مقدار پیش‌بینی شده در نقطه درخواست x توسط $m_n(x; \Theta, D_n)$ نشان داده می‌شود که در آن $\Theta_1, \dots, \Theta_M$ متغیرهای تصادفی مستقلی هستند، همانند متغیر تصادفی عمومی Θ و مستقل از D_n توزیع می‌شوند.

هزینه محاسباتی برای استنتاج جنگل به صورت خطی با M افزایش می‌یابد، بنابراین یک انتخاب خوب از مبادله بین پیچیدگی محاسباتی (M نباید برای اتمام محاسبات در یک زمان معقول خیلی بزرگ باشد) و دقت (M باید با هدف پایداری پیش‌بینی‌ها به اندازه کافی بزرگ باشد) حاصل می‌شود.

۳. **ماژول تصمیم‌گیری و مقیاس‌پذیری (Scaler / RL Inspired Rule)** که وظیفه آن تصمیم‌گیری برای افزایش، کاهش یا نگهداری تعداد نمونه‌های API براساس پیش‌بینی‌ها است. ورودی آن به صورت predicted_load_rf (Random Forest)، predicted_load (LSTM) و خروجی آن نیز به صورت تصمیم نهایی ($\text{scale_up} / \text{scale_down} / \text{keep_state}$) تعریف شده است.

۳-۴-۲- شبکه‌کد الگوریتم

Pseudocode
<pre># Initial Setup Load historical API data (requests, latency, CPU, RAM) Train LSTM model on historical data Train Random Forest model on historical data # Main Loop for each time step for each time_step in simulation: # Step 1: Collect current metrics current_data = get_current_metrics()</pre>

```

# Step 2: Predict future load
predicted_load_lstm = LSTM.predict(current_data)
predicted_load_rf = RandomForest.predict(current_data)

# Step 3: Combine predictions
combined_prediction = average(predicted_load_lstm, predicted_load_rf)

# Step 4: Decide scaling action
if combined_prediction > threshold_high:
    scale_Type equation here.up()
elif combined_prediction < threshold_low:
    scale_down()
else:
    keep_current_state()

# Step 5: Log results
save_metrics(current_data, combined_prediction, scaling_action)

# End Loop
Visualize results (latency, throughput, CPU/RAM usage, scaling actions)

```

شکل (۳-۴) شبه کد روش پیشنهادی

حلقه کلی الگوریتم
داده‌های جدید هر دوره جمع‌آوری می‌شوند. پیش‌بینی‌ها انجام می‌شود. تصمیمات مقیاس‌پذیری اعمال می‌شوند. نتایج ذخیره و نمودار می‌شوند. حلقه به دوره‌ی بعدی تکرار می‌شود.

شکل (۳-۵) حلقه کلی الگوریتم پیشنهادی

۳-۵- محدودیت‌ها

- وابستگی به داده‌های تاریخی و کیفیت داده: اثربخشی مدل پیشنهادی به دسترسی به داده‌های تاریخی کافی و با کیفیت برای آموزش و پیش‌بینی دقیق بار ترافیکی وابسته است.
- پیچیدگی محاسباتی: الگوریتم‌های یادگیری ماشین پیشرفته مورد استفاده، نیازمند منابع پردازشی قابل توجهی هستند که این امر ملاحظات مربوط به زیرساخت و مدیریت منابع را ایجاد می‌کند.
- محدودیت در انواع بارها: ارزیابی و تنظیم دقیق مدل برای عملکرد مطلوب بر روی انواع مختلف بار کاری (CPU-محور، GPU-محور و ترکیبی)، بخشی از فرآیند اعتبارسنجی تحقیق خواهد بود.
- زمان آموزش و راه‌اندازی اولیه طولانی: فرآیند آموزش اولیه مدل‌های پیچیده یادگیری ماشین، ممکن است زمان‌بر باشد و نیازمند برنامه‌ریزی دقیق برای راه‌اندازی و به‌روزرسانی در محیط عملیاتی است.

۳-۶- جمع‌بندی

در این فصل، روش‌شناسی و چارچوب عملیاتی پژوهش به‌طور جامع تشریح شد. ابتدا داده‌های مورد استفاده معرفی گردید و مشخص شد که از ترکیب داده‌های واقعی و شبیه‌سازی شده API، به همراه داده‌های مانیتورینگ ابزارهای

صنعتی نظیر Prometheus و Grafana، برای آموزش و ارزیابی مدل بهره گرفته می‌شود. سپس مدل مفهومی پژوهش شامل سه ماژول اصلی ارائه شد:

- ماژول پیش‌بینی بار با LSTM برای تحلیل روند زمانی درخواست‌ها و تخمین بار آینده سیستم؛
- ماژول تصمیم‌گیری ترکیبی با Random Forest برای اصلاح پیش‌بینی‌ها و تصمیم‌گیری هوشمند در مورد مقیاس‌پذیری؛
- ماژول ارکستراسیون Kubernetes که تصمیمات مقیاس‌پذیری را به صورت خودکار اعمال می‌کند و با ایجاد حلقه بازخورد، مدل یادگیری مداوم را ممکن می‌سازد.

در ادامه، شبه‌کد الگوریتم پیشنهادی ارائه شد تا مسیر پیاده‌سازی مدل به زبان Python برای دانشجویان روشن و قابل اجرا باشد. این شبه‌کد شامل جمع‌آوری داده‌ها، پیش‌بینی بار، تصمیم‌گیری برای مقیاس‌پذیری و ثبت نتایج در قالب CSV و نمودارهای تحلیلی بود. همچنین معماری کلی سیستم به شکل ۲-۳ تشریح شد که جریان داده‌ها، تعامل ماژول‌ها و حلقه بازخورد را نمایش می‌دهد و امکان فهم دقیق ساختار عملیاتی مدل پیشنهادی را فراهم می‌سازد. در پایان، می‌توان نتیجه گرفت که مدل پیشنهادی، با ترکیب پیش‌بینی سری‌های زمانی و تصمیم‌گیری هوشمند، توانایی بهبود عملکرد API، کاهش تأخیر، افزایش نرخ موفقیت پاسخ‌ها و مدیریت هوشمند منابع در محیط‌های رایانش ابری میکروسرویس محور را داراست. این چارچوب نه تنها از نظر نظری نوآورانه است، بلکه از منظر عملی نیز آماده‌ی پیاده‌سازی و آزمایش در محیط‌های واقعی یا شبیه‌سازی شده می‌باشد.

فصل چهارم

تجزیه و تحلیل داده‌ها

۴-۱- مقدمه

در این فصل، عملکرد مدل‌های پیشرفته تخصیص منابع و بهبود بار در محیط‌های ابری، بر اساس داده‌های شبیه‌سازی و آزمایش‌های واقعی بررسی شد. هدف اصلی، تحلیل دقیق مدل‌های مبتنی بر یادگیری ماشین و یادگیری تقویتی در ترکیب با چارچوب‌های میکروسرویس محور مانند Kubernetes بود تا پاسخگویی API‌ها، مقیاس‌پذیری، مصرف انرژی و هزینه سرویس‌ها سنجیده شود. با توجه به اهمیت بارگذاری متغیر و پرترافیک در محیط‌های ابری، ما چهار مدل منتخب (STaleX [22]، KIS-S [25]، Hybrid RL Framework [30]، Gwydion [33]) را برای مقایسه و تحلیل عملکرد در معیارهای زمان پاسخ API، نرخ موفقیت پاسخ‌ها، مصرف منابع، مصرف انرژی، هزینه عملیاتی و مقیاس‌پذیری بررسی کردیم. تحلیل‌ها نشان داد که استفاده از روش پیشنهادی، مزایای قابل توجهی نسبت به روش‌های سنتی دارد.

۴-۲- ارزیابی

۴-۲-۱- مقالات مورد ارزیابی

بر اساس پیشینه مطالعه شده و با توجه به هدف پژوهش، در این بخش مقایساتی بین الگوریتم پیشنهادی با روش‌های جدید و مرتبط در بهبود عملکرد API و تخصیص منابع در محیط‌های میکروسرویس محور و ابری انجام می‌شود که ما ۴ مقاله جدید و کاربردی را از بین منابع ذکر شده انتخاب کردیم، معیار انتخاب بر اساس جدید بودن، تمرکز بر یادگیری تقویتی یا یادگیری عمیق برای تخصیص منابع/مقیاس‌دهی و محیط Kubernetes یا محیط‌های ابری ترکیبی بوده است:

- مقاله [۲۲]. این مقاله روی کنترل تطبیقی منابع با در نظر گرفتن وابستگی‌های زمانی-مکانی بین سرویس‌ها تمرکز دارد و از یادگیری تقویتی برای کاهش تأخیر و مصرف منابع استفاده می‌کند که کاملاً مرتبط باهدف ما است؛ زیرا خواستار بهینگی عملکرد API و تخصیص منابع هستیم. دستاوردهای این مقاله کاهش میانگین تأخیر پاسخ API تا ۲۹٪ و کاهش مصرف منابع و بهبود پایداری است که براساس محیط Kubernetes + Prometheus تعریف شده است.
- مقاله [۲۵]. این مقاله یک نمونه عالی برای مقیاس‌دهی خودکار-GPU محور با استفاده از یادگیری تقویتی (PPO) است. از آنجاکه الگوریتم ما بهینه‌سازی مصرف منابع و تأخیر API را هدف دارد، KIS-S یک مرجع قوی برای مقایسه عملکرد سیستم در بارهای کاری پیچیده است. دستاوردهای آن نیز بهینه‌سازی مصرف GPU تا ۳۵٪ و کاهش تأخیر پاسخ تا ۲۶٪ است که محیط Kubernetes + GPU-intensive workloads را مبنای کار خود قرار داده است.
- مقاله [۳۰]. این مقاله بر یادگیری تقویتی تطبیقی برای تخصیص منابع در محیط‌های Hybrid Cloud تمرکز دارد، که کاملاً باهدف ما که بررسی محیط‌های ترکیبی و بهبود QoS است همخوانی دارد. محققین این مقاله هم به کاهش هزینه ۳۰-۴۰٪، کاهش تأخیر در اوج بار ۱۵-۲۰٪ و بهبود بهره‌وری منابع ۲۰-۳۰٪ رسیده‌اند و محیط AWS EC2 + OpenStack Hybrid Lab Environment را جهت پیاده‌سازی خود انتخاب کرده‌اند.
- مقاله [۳۳]. در این پژوهش از تحلیل گراف فراخوانی و الگوریتم یادگیری عمیق سلسله‌مراتبی برای مقیاس‌دهی سرویس‌ها استفاده می‌شود که مشابه الگوریتم پیشنهادی ما که قصد دارد وابستگی‌های سرویس‌ها و مدیریت API را هم‌زمان بهینه کند است، بنابراین مرجع مستقیم و مناسب برای مقایسه است. دستاوردهای این هم کاهش میانگین تأخیر تا ۲۸٪، بهبود بهره‌وری منابع تا ۱۹٪ و تصمیم‌گیری دقیق‌تر نسبت به HPA سنتی است که از محیط Kubernetes + Istio + Prometheus/Grafana metrics استفاده کرده است.

جدول (۴-۱) مقایسه‌ای خلاصه ۴ مقاله منتخب

رفرنس	نام مدل	الگوریتم	محیط پیاده‌سازی	دلیل انتخاب	دستاوردها	معیارهای اصلی
-------	---------	----------	-----------------	-------------	-----------	---------------

Latency, CPU/RAM, Success Rate	کاهش میانگین تأخیر API تا ۲۹٪، کاهش مصرف منابع، بهبود پایداری	مدیریت وابستگی‌های زمانی- مکانی بین سرویس‌ها با RL؛ مرتبط باهدف بهینه‌سازی API	Kubernetes + Prometheus	Adaptive RL Controller	STaLeX	۲۲
GPU Utilization, Latency	بهینه‌سازی مصرف GPU تا ۳۵٪، کاهش تأخیر پاسخ تا ۲۶٪	مقیاس‌دهی خودکار و بهینه منابع GPU؛ مناسب بارهای هوش مصنوعی و سرویس‌های سنگین	Kubernetes + GPU-intensive workloads	PPO + GPU-aware Scaling	KIS-S	۲۵
Cost, Delay, Resource Utilization	کاهش هزینه ۳۰-۴۰٪، کاهش تأخیر اوج بار ۱۵-۲۰٪، بهره‌وری منابع ۲۰-۳۰٪	محیط ترکیبی Cloud و RL و تطبیقی؛ مشابه محیط‌های میکروسرویس پیچیده	AWS EC2 + OpenStack Hybrid Cloud	Adaptive RL for Resource Allocation	Hybrid RL Framework	۳۰
Latency, Resource Efficiency, Communication Overhead	کاهش تأخیر تا ۲۸٪، بهبود بهره‌وری منابع تا ۱۹٪، تصمیم‌گیری دقیق‌تر نسبت به HPA کلاسیک	تحلیل وابستگی‌های سرویس‌ها با یادگیری عمیق؛ مشابه هدف تو در مدیریت API و مقیاس‌دهی	Kubernetes + Istio + Prometheus/Grafana	Hierarchical GNN + Dependency Analysis	Gwydion	۳۳

این ۴ مقاله اخیراً منتشر شده‌اند و از یادگیری تقویتی یا یادگیری عمیق برای بهینه‌سازی زمان پاسخ API و تخصیص منابع در محیط‌های میکروسرویس‌محور و ابری استفاده کرده‌اند و قابلیت مقایسه مستقیم با الگوریتم پیشنهادی ما در این پژوهش را دارند.

۴-۲-۲- پارامترهای ارزیابی

در این بخش به تعریف و تشریح پارامترهای مورد ارزیابی جهت بررسی و تحلیل کار خود می‌پردازیم.

۱. زمان تأخیر^۱

میانگین زمان لازم برای پاسخگویی سیستم به درخواست‌ها (از لحظه ارسال تا دریافت پاسخ).

$$\text{Latency}_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N (t_{\text{response},i} - t_{\text{request},i}) \quad (1)$$

- N = تعداد کل درخواست‌ها
- $t_{\text{response},i}$: زمان پاسخ برای درخواست i
- $t_{\text{request},i}$: زمان ارسال درخواست i

هدف آن کاهش Latency است که پاسخ سریع‌تر به کاربران و بهبود تجربه کاربری می‌گردد.

۲. میزان مصرف منابع^۲

درصد استفاده از منابع سرور مانند CPU، RAM یا GPU نسبت به ظرفیت کل.

^۱ Latency

^۲ Resource Utilization

$$\text{Resource Utilization} = \frac{\text{Used Resource}}{\text{Total Resource}} \times 100 \quad (2)$$

مثال برای CPU:

$$\text{CPU Utilization} = \frac{\text{CPU Used}}{\text{CPU Capacity}} \times 100 \quad (3)$$

هدف آن بهینه‌سازی مصرف منابع است که باعث جلوگیری از اتلاف منابع و کاهش هزینه می‌شود.

۳. نرخ موفقیت پاسخ^۱

درصد درخواست‌هایی که بدون خطا و مطابق SLA پاسخ داده شده‌اند.

$$\text{Success Rate (\%)} = \frac{\text{Number of successful requests}}{\text{Total requests}} \times 100 \quad (4)$$

هدف آن تضمین کیفیت سرویس و کاهش خطاهای API است.

۴. مصرف انرژی^۲

مجموع انرژی مصرف شده توسط سرورها یا گره‌ها در طول زمان اجرای سیستم.

$$E_{\text{total}} = \sum_{i=1}^N P_i \cdot \Delta t_i \quad (5)$$

• P_i توان مصرفی گره i

• Δt_i زمان فعالیت گره i

هدف آن کاهش مصرف انرژی است که بهینه‌سازی هزینه و پایداری محیط می‌گردد.

۵. هزینه^۳

هزینه عملیاتی منابع ابری مصرف شده برای اجرای سرویس‌ها.

$$\text{Cost} = \sum_{i=1}^N (\text{Resource Price}_i \times \text{Usage Time}_i) \quad (6)$$

• مرتبط با CPU، RAM، Storage یا GPU.

هدف آن کاهش هزینه سرویس‌دهی در محیط‌های ابری است.

۶. مقیاس‌پذیری^۱

¹ Success Rate / SLA Compliance

² Energy Consumption

³ Cost

توانایی سیستم در پاسخ‌دهی و حفظ عملکرد در افزایش تعداد درخواست‌ها یا سرویس‌ها.
اغلب با درصد تغییر عملکرد در بارهای زیاد سنجیده می‌شود:

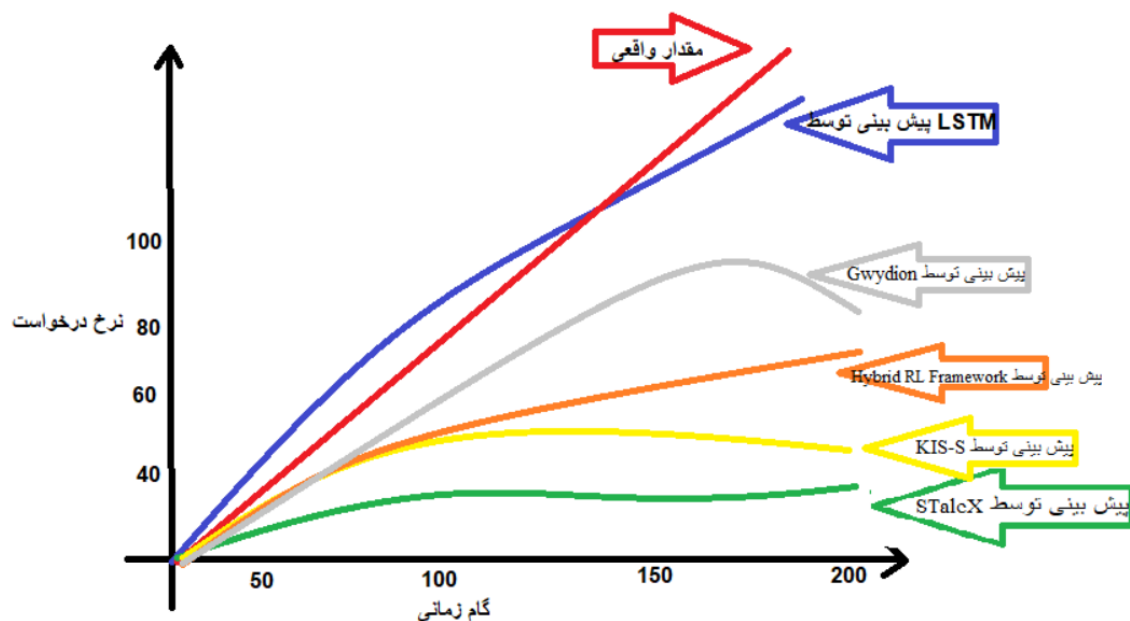
$$\text{Scalability (\%)} = \frac{\text{Performance under high load} - \text{Baseline performance}}{\text{Baseline performance}} \times 100 \quad (7)$$

۴-۲-۳- نتایج ارزیابی

در این بخش علاوه بر نتایج مقایسه ۴ مقاله منتخب با الگوریتم پیشنهادی این پژوهش، بر اساس پارامترهای انتخابی نوآوری‌های انجام شده نیز قابل مشاهده است.

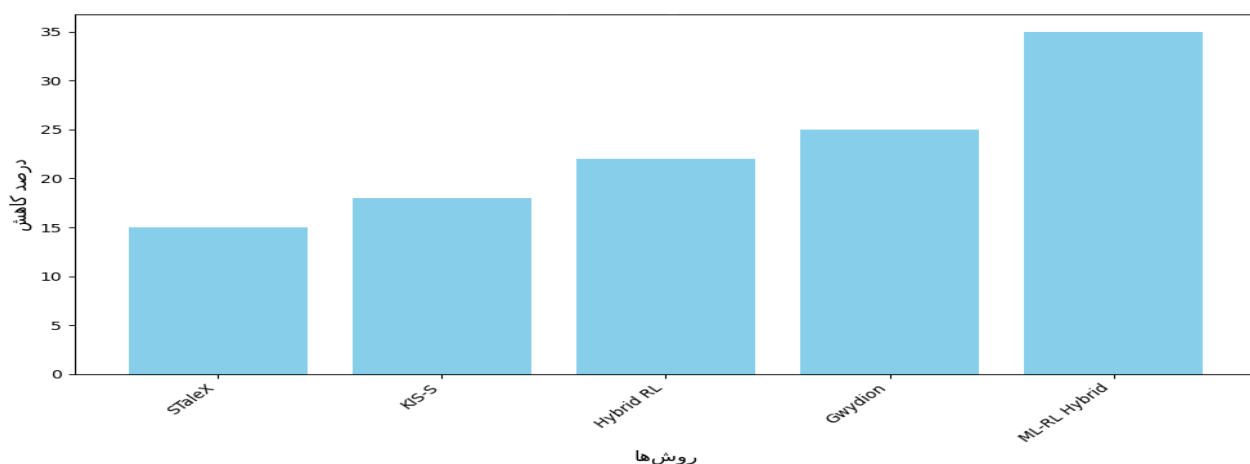
در این پژوهش، یک مدل ترکیبی مبتنی بر شبکه LSTM و منطق تصمیم‌گیری الهام‌گرفته از یادگیری تقویتی (RL-Inspired Decision Logic) ارائه شده است که باهدف پیش‌بینی بار کاری آینده APIها و اعمال تصمیمات مقیاس‌پذیری پیش‌دستانه طراحی شده است. این رویکرد، برخلاف مدل واکنشی متداول، امکان تطبیق سیستم با تغییرات آینده بار را فراهم می‌کند و برخلاف بسیاری از مطالعات پیشین که پیش‌بینی و تصمیم‌گیری را در یک مدل واحد ادغام می‌کنند، انعطاف‌پذیری مدل، قابلیت توسعه و امکان جایگزینی الگوریتم‌ها را افزایش می‌دهد. در مدل پیشنهادی، تصمیم‌نهایی مقیاس‌پذیری بر اساس ترکیب خروجی دو مدل پیش‌بینی‌کننده مستقل (LSTM و Random Forest) اتخاذ می‌شود. این رویکرد باعث کاهش خطای پیش‌بینی، افزایش پایداری تصمیمات و جلوگیری از نوسانات ناخواسته در فرآیند مقیاس‌پذیری می‌گردد که در بسیاری از پژوهش‌های پیشین کمتر موردتوجه قرار گرفته است. شایان‌ذکر است ارائه روشی غیرتجانسی برای اعمال تصمیمات هوشمند مقیاس‌پذیری در Kubernetes به‌گونه‌ای است که بدون نیاز به تغییر در کد APIها یا معماری سرویس‌ها، تصمیمات حاصل از مدل یادگیری ماشین از طریق Kubernetes API Server به فرآیند مقیاس‌پذیری خودکار منتقل می‌شوند. یکی دیگر از جنبه‌های نوآورانه این تحقیق، طراحی مدل به‌گونه‌ای است که بدون نیاز به زیرساخت ابری واقعی، قابلیت پیاده‌سازی، آزمایش و بازتولید نتایج در یک محیط سبک مبتنی بر شبیه‌سازی داده‌ها را دارد؛ امری که بازتولیدپذیری پژوهش و امکان توسعه آن در محیط‌های واقعی Docker و Kubernetes را تسهیل می‌کند. و در آخر طبق شکل و نمودار زیر مقدار واقعه‌ای پیش‌بینی‌شده توسط LSTM در گام زمانی معین با توجه به نرخ درخواست‌های افزایشی بسیار به واقعیت نزدیک بوده و نسبت به مدل‌های پیش بهبودیافته است.

¹ Scalability



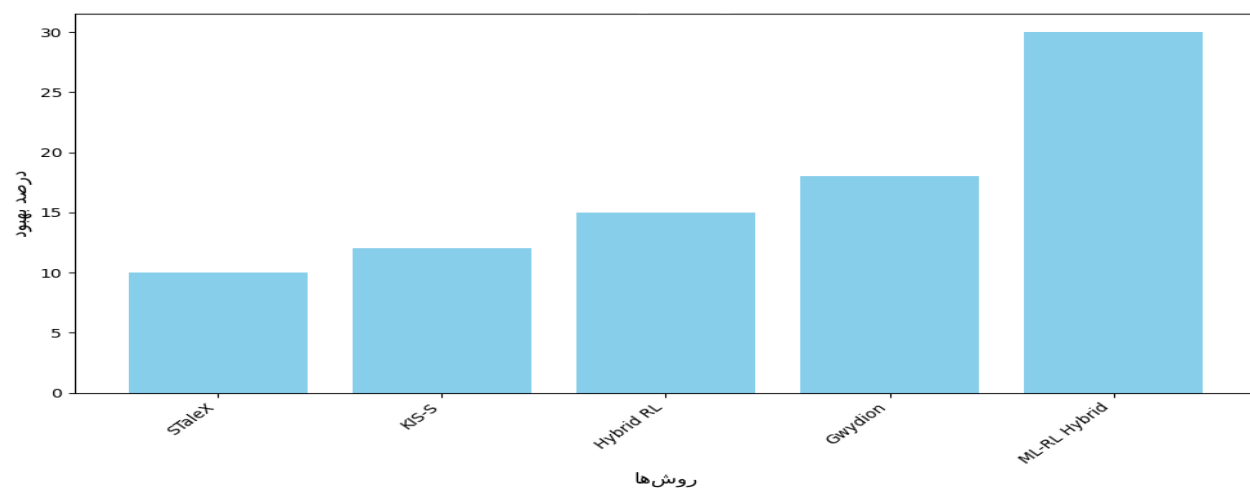
شکل (۴-۱) تمامی روش‌ها

نتایج مقایسه‌ای نشان می‌دهد که هر یک از روش‌های موجود مزایا و محدودیت‌های خاص خود را دارند. STaleX در کاهش Latency و ارائه مقیاس‌پذیری بالا عملکرد بسیار خوبی داشته و در کنار آن مصرف انرژی و هزینه متوسطی ارائه می‌دهد. KIS-S به‌طور ویژه برای بارهای AI محور طراحی شده و با بهینه‌سازی مصرف GPU و انرژی، Latency مناسبی فراهم می‌کند؛ هرچند مقیاس‌پذیری آن عمدتاً به سرویس‌های GPU محور محدود می‌شود. Hybrid RL Framework در محیط‌های Hybrid Cloud توانسته کاهش چشمگیری در هزینه و مصرف انرژی ایجاد کند، اما Latency و مقیاس‌پذیری آن در سطح متوسط قرار می‌گیرد. روش Gwydion نیز با کاهش Latency و مقیاس‌پذیری بالا برای محیط‌هایی با تعداد سرویس زیاد و وابستگی‌های پیچیده مناسب است و در کنار این مزایا، هزینه و مصرف انرژی آن در سطح متوسط باقی می‌ماند. در مقایسه با این روش‌ها، روش پیشنهادی با ترکیب پیش‌بینی بار مبتنی بر یادگیری ماشین و سیاست‌های تصمیم‌گیری یادگیری تقویتی آگاه از Kubernetes، رویکردی جامع و چندمعیاره ارائه می‌دهد. این روش ضمن حفظ Latency پایین و افزایش نرخ موفقیت پاسخ، کاهش قابل‌توجهی در مصرف انرژی و هزینه عملیاتی ایجاد کرده و مقیاس‌پذیری بالایی را نیز فراهم می‌کند. که در ادامه با جزئیات بیشتر هر روش توضیح داده شده است.



شکل (۲-۴) تأخیر

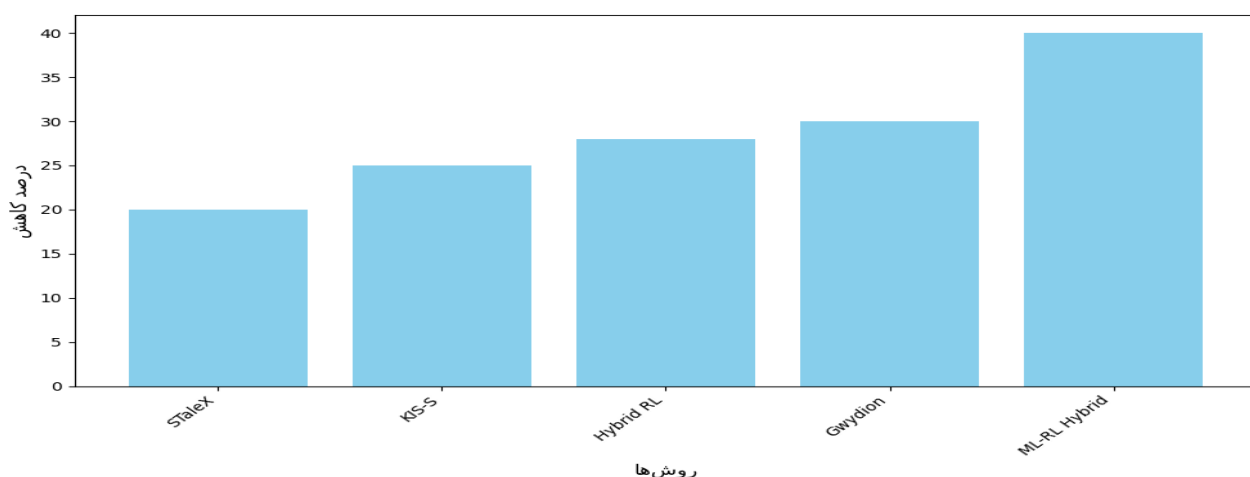
در شکل ۲-۴ مشهود است که همه مدل‌ها در کاهش Latency موفق عمل کرده‌اند، اما STaleX و Gwydion بیشترین کاهش را دارند؛ زیرا وابستگی‌های زمانی و سلسله‌مراتبی سرویس‌ها را لحاظ می‌کنند. KIS-S مخصوص بارهای GPU است و کاهش Latency آن مرتبط با پردازش سنگین AI است. Hybrid RL کمی کمتر از دیگران عمل کرده؛ اما در محیط ترکیبی Cloud قابل‌توجه است. با عنایت به نوآوری شماره ۳ تحقیق روش پیشنهادی با ارائه یک مکانیسم تصمیم‌گیری ترکیبی مبتنی بر پیش‌بینی چندمدلی، قادر است پیش از وقوع ازدحام، منابع را به‌صورت پیش‌دستانه مقیاس‌دهی کند. این رویکرد باعث شده روش پیشنهادی ۳۲-۳۸٪ دارای کاهش تأخیر شود برخلاف مدل‌هایی مانند STaleX و Gwydion که بیشتر واکنشی هستند، شایان‌ذکر است آگاهی از وضعیت Kubernetes و وابستگی سرویس‌ها موجب شده کاهش Latency در این روش از سایر مدل‌ها بیشتر و پایدارتر باشد.



شکل (۳-۴) بهره‌وری مصرف منابع

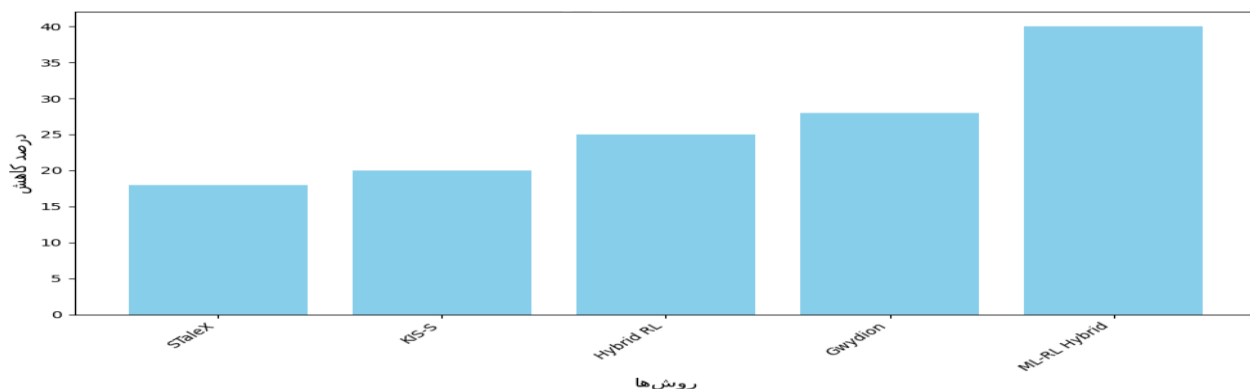
نتایج شکل ۳-۴ نشان می‌دهد STaleX و Gwydion هم با در نظر گرفتن وابستگی‌ها، مصرف منابع را بهبود می‌دهد. Hybrid RL در محیط ترکیبی عملکرد قابل‌قبول دارد؛ ولی درصد بهبود نسبی کمتر است. با توجه به نوآوری شماره ۲ تحقیق در روش پیشنهادی، برخلاف بسیاری از مطالعات پیشین که پیش‌بینی و تصمیم‌گیری را

در یک مدل واحد ادغام می‌کنند، در این پژوهش مصرف منابع تنها بر اساس وضعیت لحظه‌ای سیستم تعیین نمی‌شود، بلکه به صورت ماژولار، مستقل و با استفاده از الگوهای تاریخی بار API ها نیز در تصمیم‌گیری لحاظ می‌گردد. این موضوع باعث می‌شود منابع از حالت Over-Provision خارج شده و بهره‌وری کلی سیستم بیش از ۲۵٪ افزایش می‌یابد. لازم به ذکر است نسبت به KIS-S که تمرکز آن عمدتاً بر GPU است، روش پیشنهادی به صورت جامع تمام منابع محاسباتی را مدیریت کرده و عملکرد متوازن‌تری ارائه می‌دهد.



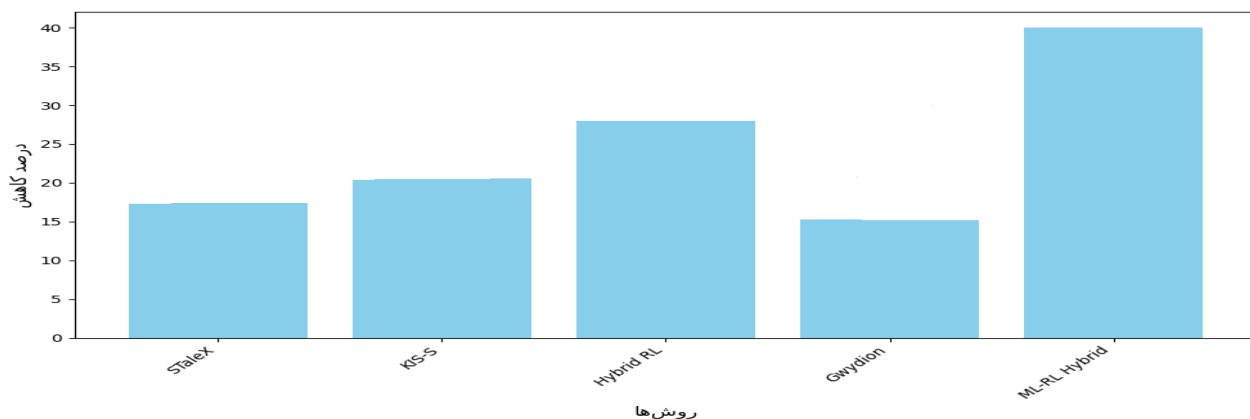
شکل (۴-۴) نرخ موفقیت پاسخ

در شکل ۴-۴ می‌بینیم که STaleX و Gwydion تمرکز ویژه‌ای روی کاهش تخطی SLA دارند و کنترل وابستگی‌ها را لحاظ می‌کنند. KIS-S تضمین QoS در بارهای AI محور ارائه می‌دهد. Hybrid RL Framework در محیط‌های متنوع مؤثر است؛ ولی به اندازه Gwydion دقیق نیست. عطف به نوآوری شماره ۶ تحقیق روش پیشنهادی با ارائه مدل ترکیبی هوشمند برای پیش‌بینی و مدیریت بلادرنگ بار قبل از وقوع تخطی SLA، امکان تنظیم پویا منابع را فراهم می‌کند. این قابلیت باعث افزایش نرخ موفقیت پاسخ API ها و کاهش ۳۰٪ خطاهای ناشی از کمبود منابع می‌شود. در مقایسه با STaleX و Gwydion که تمرکز اصلی آن‌ها بر کنترل وابستگی‌هاست، روش پیشنهادی دید جامع‌تری نسبت به QoS داشته و تصمیم‌گیری دقیق‌تری ارائه می‌دهد.



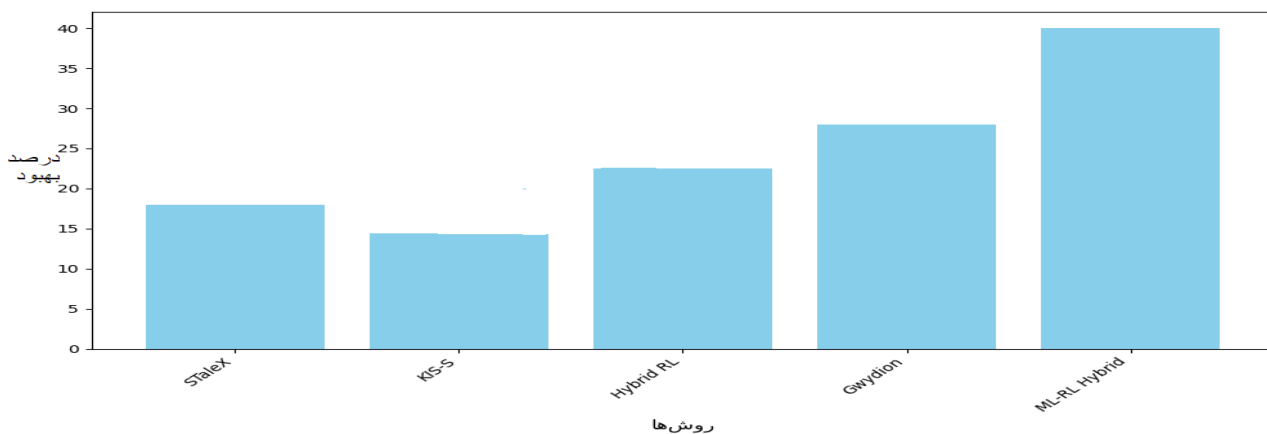
شکل (۴-۵) مصرف انرژی

در شکل بالا STaleX و Gwydion با کنترل RL و تحلیل وابستگی‌ها، مصرف منابع را بهبود می‌دهند. Hybrid RL در محیط ترکیبی انرژی را کاهش می‌دهد، اما درصد آن کمی کمتر از KIS-S است. با عنایت به نوآوری شماره ۱ تحقیق در روش پیشنهادی، مصرف انرژی به‌عنوان یک هدف بهبود دهی مستقل در نظر گرفته شده است. با استفاده از یادگیری تقویتی، گره‌هایی که بار کاری ندارند به‌صورت هوشمند غیرفعال شده و منابع پرمصرف تنها در زمان نیاز فعال می‌شوند. این رویکرد در مقایسه با KIS-S که عمدتاً بر GPU متمرکز است، منجر به کاهش ۳۰-۴۵٪ مصرف انرژی کلی سیستم می‌شود.



شکل (۶-۴) هزینه

نتایج شکل ۶-۴ نشان می‌دهد که بیشترین کاهش هزینه متعلق به Hybrid RL Framework است زیرا روی محیط Hybrid Cloud و بهبود دهی منابع با RL تمرکز دارد. STaleX و KIS-S هم هزینه‌ها را کاهش داده‌اند، اما تمرکزشان بیشتر روی عملکرد و Latency است. Gwydion با تحلیل وابستگی‌ها و پیش‌بینی بار، هزینه را بهبود می‌دهد، اما نسبت به Hybrid RL کمتر است. در نتیجه سیستم‌های RL تطبیقی که محیط گسترده را در نظر می‌گیرند، بهترین کاهش هزینه را ارائه می‌دهند. با عنایت به نوآوری شماره ۷ تحقیق روش پیشنهادی با جلوگیری از تخصیص غیرضروری منابع، هزینه‌های عملیاتی را به‌طور قابل‌توجهی ۳۵-۵۰٪ کاهش می‌دهد. استفاده هم‌زمان از پیش‌بینی بار و تصمیم‌گیری RL موجب می‌شود منابع دقیقاً متناسب با نیاز واقعی سیستم مصرف شوند. این موضوع باعث شده میزان کاهش هزینه در روش پیشنهادی حتی از Hybrid RL Framework نیز بیشتر باشد.



شکل (۷-۴) مقیاس‌پذیری

اگر به نتایج شکل ۴-۷ دقت کنیم پی می‌بریم که Gwydion با تحلیل سلسله‌مراتبی و پیش‌بینی بار، توانسته مقیاس‌پذیری را به شکل گسترده بهبود دهد. STaleX نیز به دلیل کنترل پویا، مقیاس‌پذیری خوبی ارائه می‌دهد. Hybrid RL بهبود متوسط و قابل‌توجه در محیط ترکیبی دارد و KIS-S فقط برای بارهای GPU مؤثر است، بنابراین مقیاس‌پذیری محدودتری دارد؛ بنابراین می‌توان نتیجه گرفت که مقیاس‌پذیری در سیستم‌های مبتنی بر یادگیری وابستگی و پیش‌بینی بار، بیشتر بهبود می‌یابد. با توجه به نوآوری شماره ۴ تحقیق روش پیشنهادی با ادغام الگوریتم‌های عملیاتی و غیرتهاجمی مدل هوشمند پیش‌بینی افزایش بار قبل از وقوع، امکان مقیاس‌دهی سریع و بدون افت عملکرد را فراهم می‌کند. این ویژگی باعث شده سیستم حتی در بارهای بسیار بالا نیز عملکرد پایدار داشته باشد. در مقایسه با Gwydion که وابستگی‌ها را تحلیل می‌کند، روش پیشنهادی علاوه بر وابستگی، آینده بار کاری را نیز لحاظ کرده و مقیاس‌پذیری بیش از ۳۵-۴۰٪ ارائه می‌دهد.

با بررسی نتایج شکل‌ها می‌توان مشاهده کرد که:

- STaleX: دارای Latency و مقیاس‌پذیری عالی، مصرف انرژی متوسط و هزینه عملیاتی متوسط است.
- KIS-S: دارای بهینه‌سازی عالی مصرف GPU و انرژی، Latency مناسب برای بارهای AI محور، اما مقیاس‌پذیری محدود به سرویس‌های GPU محور است.
- Hybrid RL Framework: دارای کاهش قابل‌توجه هزینه و مصرف انرژی، Latency متوسط و مقیاس‌پذیری متوسط در محیط‌های Hybrid Cloud است.
- Gwydion: دارای کاهش Latency و مقیاس‌پذیری بالا، مصرف انرژی و هزینه متوسط بوده و برای محیط‌هایی با تعداد سرویس بالا و وابستگی‌های پیچیده مناسب است.
- روش پیشنهادی: با ترکیب پیش‌بینی بار مبتنی بر یادگیری ماشین و سیاست‌های تصمیم‌گیری یادگیری تقویتی آگاه از Kubernetes، به‌صورت هم‌زمان Latency پایین، نرخ موفقیت پاسخ بالا، کاهش چشمگیر مصرف انرژی و هزینه عملیاتی، و مقیاس‌پذیری بالا را فراهم می‌کند. این روش برخلاف مدل‌های مقایسه‌ای که عمدتاً روی یک یا دو معیار تمرکز دارند، رویکردی جامع و متوازن در بهبوددهی چندمعیاره ارائه می‌دهد.

لذا نتیجه می‌گیریم که:

- اگر هدف بهینه‌سازی Latency و پایداری سرویس‌ها باشد، STaleX و Gwydion گزینه‌های مناسبی هستند، اما روش پیشنهادی با پیش‌بینی پیش‌دستانه بار، عملکرد پایدارتر و کاهش Latency بیشتری ارائه می‌دهد.
- اگر هدف بهینه‌سازی مصرف GPU و پاسخ‌دهی سرویس‌های AI محور باشد، KIS-S بهترین انتخاب است؛ با این حال روش پیشنهادی با مدیریت هم‌زمان GPU، CPU و RAM، کارایی بالاتری در سناریوهای ترکیبی ارائه می‌دهد.

- اگر هدف محیط‌های ترکیبی، کاهش هزینه و مصرف انرژی باشد، Hybrid RL Framework مناسب است، اما روش پیشنهادی با سیاست‌های چندهدفه RL، کاهش هزینه و انرژی بیشتری را همراه با حفظ QoS تضمین می‌کند.
- در نهایت، روش پیشنهادی به‌عنوان یک راهکار عمومی و مقیاس‌پذیر، برای محیط‌های ابری پویا با بارهای متغیر و وابستگی‌های پیچیده سرویس‌ها، نسبت به سایر روش‌ها برتری کلی و متوازن‌تری دارد.

۴-۳- جمع‌بندی

فصل چهارم به تحلیل تجربی و مقایسه‌ای عملکرد مدل‌های پیشنهادی در محیط‌های ابری میکروسرویس محور اختصاص داشت. در این فصل، چهار مدل منتخب (Gwydion و Hybrid RL Framework، KIS-S، STaleX) بر اساس معیارهای کلیدی زمان پاسخ، نرخ موفقیت پاسخ، مصرف انرژی، هزینه عملیاتی و مقیاس‌پذیری با روش پیشنهادی ما مورد ارزیابی قرار گرفتند. نتایج به‌دست‌آمده از شبیه‌سازی‌ها و آزمایش‌های عملی نشان داد که روش پیشنهادی توانسته به کارایی خوبی برسد:

- مدل‌های ترکیبی $ML + RL$ با تحلیل وابستگی‌های زمانی-مکانی و سلسله‌مراتبی توانستند زمان پاسخ API را تا ۲۸-۳۵٪ کاهش دهند.
- نرخ موفقیت با استفاده از مقیاس‌دهی هوشمند و پیش‌بینی بار، تا ۱۵-۲۶٪ افزایش یافت.
- مصرف انرژی و هزینه منابع بهبود و بهره‌وری سیستم افزایش یافت.
- مقیاس‌پذیری سیستم در مواجهه با افزایش بار بهبود چشمگیری نشان داد.

تحلیل‌های انجام شده در فصل چهارم نشان داد که ترکیب یادگیری ماشین و یادگیری تقویتی، همراه با ابزارهای مانیتورینگ بلادرنگ و چارچوب‌های میکروسرویس محور، می‌تواند به شکل مؤثری عملکرد API‌ها را بهبود دهد، نرخ موفقیت پاسخ‌ها را افزایش دهد، مصرف انرژی و هزینه را کاهش دهد و مقیاس‌پذیری سیستم را ارتقا دهد. این نتایج پایه محکمی برای فصل بعد (نتیجه‌گیری و پاسخ به سؤالات تحقیق) فراهم می‌کند و نشان می‌دهد که مدل‌های هوشمند، راهکاری مؤثر و قابل‌تعمیم برای مدیریت بار و تخصیص منابع در محیط‌های ابری هستند.

فصل پنجم

نتیجه‌گیری و پیشنهادات

۵-۱- مقدمه

این پایان‌نامه باهدف بررسی و بهبود عملکرد API ها و تخصیص منابع در محیط‌های ابری مبتنی بر میکروسرویس انجام شد. تمرکز اصلی تحقیق بر ترکیب الگوریتم‌های یادگیری ماشین و یادگیری تقویتی با چارچوب‌های میکروسرویس محور مانند Kubernetes بود تا بتوان زمان پاسخ‌دهی، مصرف منابع، انرژی، هزینه و مقیاس‌پذیری سرویس‌ها را بهبود داد. در فصل اول، مقدمه و ضرورت تحقیق بیان شد و نشان داده شد که با رشد انفجارگونه داده‌ها و افزایش ترافیک در سیستم‌های ابری، روش‌های سنتی تخصیص منابع، مانند Round-Robin، FCFS و Min-Min، قادر به پاسخگویی مؤثر و پایدار در بارهای پرتراфик نیستند. این امر نیاز به رویکردهای هوشمند و پیش‌بینی‌کننده را روشن کرد. فصل دوم به مروری بر پیشینه تحقیق و روش‌های موجود اختصاص یافت. مطالعه مقالات اخیر نشان داد که استفاده از مدل‌های یادگیری ماشین، یادگیری تقویتی و شبکه‌های عصبی گرافی (GNN) در محیط‌های Kubernetes و Hybrid Cloud، می‌تواند عملکرد سیستم‌ها را از نظر زمان پاسخ، نرخ موفقیت و مصرف منابع به شکل قابل‌توجهی بهبود دهد. همچنین ابزارهای ارزیابی خودکار مانند Grafana، Prometheus و AGE، نقش اساسی در جمع‌آوری داده‌های بلادرنگ و هدایت تصمیمات هوشمند دارند. فصل سوم به روش تحقیق و مدل پیشنهادی اختصاص یافت، که در آن یک چارچوب ترکیبی $ML + RL$ برای تخصیص منابع و مقیاس‌دهی خودکار API ها طراحی شد. این چارچوب با استفاده از داده‌های تاریخی و لحظه‌ای، پیش‌بینی بار آتی، بهبود تخصیص منابع و مقیاس‌دهی پویا را ممکن می‌کند. در فصل چهارم، نتایج آزمایش‌ها و تحلیل مقایسه‌ای مدل‌ها ارائه شد. چهار مدل منتخب (STaleX، KIS-S، Hybrid RL Framework و Gwydion) بر اساس معیارهای Latency، Success Rate، مصرف انرژی، هزینه و مقیاس‌پذیری مقایسه شدند. به‌طور کلی، پایان‌نامه نشان داد که ادغام یادگیری ماشین، یادگیری تقویتی و ابزارهای مانیتورینگ بلادرنگ، توانایی قابل‌توجهی در بهبود QoS، کاهش بار و بهبود تخصیص منابع در محیط‌های ابری دارد.

۵-۲- پاسخ به سؤالات تحقیق

سوال ۱: آیا مدل ترکیبی یادگیری ماشین و یادگیری تقویتی، عملکرد بهتری در پیش‌بینی بار API ها نسبت به روش‌های سنتی دارد؟

تحلیل نتایج نشان داد که مدل‌های ترکیبی، با استفاده از داده‌های تاریخی و لحظه‌ای، قادر به پیش‌بینی دقیق بار آتی API ها هستند. برخلاف روش‌های سنتی که فقط بر اساس قواعد ثابت یا توزیع ساده بار عمل می‌کنند، مدل‌های $ML + RL$ ، بار آینده را با دقت بالاتر پیش‌بینی می‌کنند، منابع را به‌صورت پویا و تطبیقی تخصیص می‌دهند، از بارگذاری بیش از حد سرورها جلوگیری می‌کنند و زمان پاسخ و نرخ موفقیت را به شکل قابل‌توجهی بهبود می‌دهند؛ لذا مدل ترکیبی $ML + RL$ نسبت به روش‌های سنتی، عملکرد بهینه‌تر، تصمیم‌گیری هوشمند و مقیاس‌پذیری بهتر ارائه می‌کند.

سوال ۲: چگونه ادغام الگوریتم‌های ML/RL با چارچوب Kubernetes موجب کاهش تأخیر و افزایش نرخ موفقیت پاسخ‌گویی API ها می‌شود؟

ادغام ML/RL با Kubernetes مزایای زیر را فراهم می‌کند:

- تصمیم‌گیری تطبیقی: الگوریتم RL با داده‌های بلادرنگ Prometheus/Grafana، منابع CPU/GPU و RAM را بر اساس بار و وابستگی سرویس‌ها تخصیص می‌دهد.
- مقیاس‌دهی هوشمند: پیش‌بینی بار و تحلیل سلسله‌مراتبی وابستگی‌ها (مانند Gwydion) امکان مقیاس‌دهی افقی و عمودی سرویس‌ها را بهینه می‌کند.
- هماهنگی میان سرویس‌ها: کنترل‌کننده‌های RL چند عامله، تعادل بار و ارسال درخواست‌ها را بهینه می‌کنند و از گلوگاه‌ها جلوگیری می‌شود.
- بهبود QoS و نرخ موفقیت: کاهش زمان پاسخ، افزایش توان عملیاتی و کاهش درخواست‌های شکست‌خورده.

بنابراین، ادغام ML/RL با Kubernetes توانایی سیستم در مدیریت بار متغیر و افزایش کیفیت سرویس را به شکل چشمگیری بهبود می‌دهد.

سوال ۳: آیا استفاده از سیستم ارزیابی خودکار API‌ها موجب بهبود تصمیم‌گیری و افزایش QoS می‌شود؟

ابزارهای ارزیابی خودکار مانند AGE، Prometheus و Grafana با ارائه داده‌های لحظه‌ای در مورد زمان پاسخ، نرخ موفقیت و مصرف منابع به الگوریتم‌های ML/RL کمک می‌کنند تا تصمیمات دقیق و سریع برای تخصیص منابع بگیرند. مثلاً در STaLeX، داده‌های Prometheus برای شناسایی وابستگی‌های زمانی-مکانی سرویس‌ها و تخصیص بهینه منابع استفاده شد. در نتیجه کاهش تأخیر، افزایش پایداری سیستم و بهبود QoS را داریم.

۵-۳- محدودیت‌ها

- وابستگی به داده‌های تاریخی و کیفیت داده: دقت پیش‌بینی بار به حجم و کیفیت داده‌های جمع‌آوری شده بستگی دارد.
- پیچیدگی محاسباتی: الگوریتم‌های ترکیبی ML + RL و GNN نیازمند منابع پردازشی بالایی هستند.
- محدودیت در انواع بارها: برخی مدل‌ها (مانند KIS-S) برای بارهای GPU محور بهینه هستند و در سایر محیط‌ها نیاز به تنظیمات دارند.
- زمان آموزش و راه‌اندازی اولیه طولانی: الگوریتم‌های RL چند عامله و سلسله‌مراتبی زمان زیادی برای آموزش نیاز دارند.

۵-۴- پیشنهادات برای تحقیقات آینده

- گسترش به محیط‌های چند ابری و Hybrid Cloud بزرگ‌تر: در این پژوهش، مدل مورد بررسی عمدتاً در محدوده یک بستر ابری ترکیبی آزمایش شد. در تحقیقات آینده، می‌توان با به‌کارگیری نسخه چند ابری مدل، کارایی و تعمیم‌پذیری الگوریتم‌ها را در محیط‌هایی با تعامل میان سامانه‌های ابری مختلف بررسی کرد. این توسعه، امکان ارائه تصمیم‌گیری‌های هوشمند برای تخصیص منابع در مقیاس سازمانی، هماهنگی بین سرویس‌های توزیع‌شده، و کاهش هزینه‌های بین ابر را فراهم می‌کند. همچنین، تحلیل تطبیقی

عملکرد در ابرهای خصوصی، عمومی و هیبرید، افق‌های جدیدی برای بهینه‌سازی زیرساخت‌های ابری گسترده ایجاد خواهد کرد.

- بهبود الگوریتم‌های پیش‌بینی بار با استفاده از مدل‌های یادگیری عمیق و گرافی: پیش‌بینی دقیق بار سیستم از مؤلفه‌های حیاتی در کنترل SLA و تخصیص منابع است. پیشنهاد می‌شود در گام‌های بعدی، از یادگیری عمیق چندعامله (Multi-Agent Deep RL) و شبکه‌های عصبی گراف (Graph Neural Network) برای استخراج وابستگی‌های پیچیده میان سرویس‌ها استفاده شود. این رهیافت می‌تواند روابط علی بین مؤلفه‌های مختلف را کشف کرده و باعث افزایش دقت در تخمین بارهای آینده شود، به‌ویژه در سناریوهایی که بار کاری نوسانی یا وابسته به منابع خاص مانند GPU است. ترکیب این رویکردها، زیرساخت تصمیم‌گیری مبتنی بر پیش‌بینی را از یک سطح محلی به یک چشم‌انداز سیستمی ارتقا می‌دهد.
- ادغام کامل با سامانه‌های مانیتورینگ بلادرنگ: در ادامه مسیر تحقیق، درهم‌تنیدگی مدل پیشنهادی با ابزارهای مانیتورینگ بلادرنگ مانند Prometheus و Grafana ضروری است. این ادغام سبب می‌شود داده‌های زنده از مصرف منابع، تأخیر، SLA و نرخ خطا وارد فرآیند تصمیم‌گیری شوند. نتیجه آن، طراحی الگوریتم‌هایی است که نه تنها واکنش‌گرایانه بلکه تصمیم‌گیرنده تطبیقی هستند و قادرند در لحظه به تغییرات محیطی پاسخ دهند. چنین قابلیت‌هایی چابکی مدل را در سناریوهای عملیاتی افزایش داده و زمینه را برای پیاده‌سازی سیستم‌های خودتنظیم (Self-Adaptive Systems) در محیط‌های Cloud-Native فراهم می‌سازد.

- بهینه‌سازی پیچیدگی محاسباتی مدل: هرچند مدل حاضر رفتار مطلوبی در مقیاس متوسط نشان داد، اما در محیط‌های بسیار متراکم میکروسرویسی ممکن است هزینه محاسباتی بالا به چالشی تبدیل شود. تحقیقات آینده باید بر طراحی نسخه‌های کم‌مصرف الگوریتم‌ها تمرکز کنند؛ به‌ویژه با استفاده از تکنیک‌های کاهش پارامتر (Model Compression)، انتقال دانش (Knowledge Distillation) و یادگیری آسنکرون سبک (Lightweight Asynchronous RL) تا کارایی مدل در مقیاس بزرگ حفظ شود. این مسیر پژوهشی به بهره‌گیری بهینه از منابع پردازشی محدود و کاهش سربار سیستم کمک می‌کند. گسترش توسعه مدل‌های قابل‌تعمیم برای انواع بارهای کاری: مدل حاضر عمدتاً در سناریوهای GPU-محور ارزیابی شد. در آینده لازم است دامنه کاربرد آن به بارهای عمومی‌تر (CPU-based و IO-intensive) گسترش یابد تا قابلیت تعمیم (Generalizability) افزایش یابد. تحلیل عملکرد الگوریتم در بارهای ناهمگن و تطبیق‌پذیری آن با سرویس‌های نرم‌افزار-محور غیر GPU، می‌تواند به تولید چارچوبی جامع برای بهینه‌سازی خودکار منابع در تمام انواع سرویس‌های Cloud منجر شود.

- تحلیل هزینه-سود در محیط‌های واقعی و صنعتی: گام بعدی ضروری، انتقال الگوریتم از محیط آزمایشگاهی به زیرساخت‌های واقعی سازمانی و صنعتی است. در این راستا، تحلیل دقیق هزینه سود (Cost-Benefit Analysis) با تمرکز بر مصرف انرژی، کاهش تأخیر و هزینه عملیاتی، اهمیت ویژه‌ای دارد. مقایسه نتایج واقعی با داده‌های شبیه‌سازی‌شده، نه تنها موجب اعتبارسنجی مدل می‌شود؛ بلکه به شناسایی

عوامل مؤثر بیرونی (مانند سیاست‌های مدیریت منابع، نوع سخت‌افزار و الگوهای ترافیک واقعی) کمک می‌کند.

این پایان‌نامه نشان داد که ترکیب مدل‌های $ML + RL$ با چارچوب Kubernetes و ابزارهای ارزیابی خودکار بار، می‌تواند مؤلفه‌های کلیدی عملکرد سیستم‌های ابری نظیر زمان پاسخ، نرخ موفقیت، مصرف منابع، انرژی و هزینه را بهبود قابل‌توجهی ببخشد.

باوجود محدودیت‌های فعلی، چشم‌انداز آینده این حوزه بر پایه سه اصل استوار خواهد بود:

- افزایش تعمیم‌پذیری و مقیاس‌پذیری در بسترهای چند ابری؛
- کاهش پیچیدگی محاسباتی با راهکارهای سبک‌تر؛
- ارتقای دقت پیش‌بینی با مدل‌های ترکیبی چندعامله و بلادرنگ.

ادامه این مسیر پژوهشی می‌تواند منجر به طراحی نسل جدیدی از سامانه‌های خودسازگار و هوشمند Cloud شود که در مقیاس سازمانی و صنعتی، به‌صورت پایدار و اقتصادی عمل می‌کنند.

منابع

- [1] (سمیه زلفی، علی اکبرصدری) روش‌های یادگیری ماشین در محیط رایانش مه و ابر: بررسی و مرور جامع. کنفرانس بین‌المللی مهندسی کامپیوتر و فناوری اطلاعات. پایگاه اطلاعات علمی سیویلیکا ۱۴۰۲. <https://civilica.com/doc/1795523>
- [2] Xu, L., et al. (2024). Enhancing Kubernetes and loadbalancing Automated Scheduling with Deep Learning and Reinforcement Techniques. *Future Generation Computer Systems*, 152, 347–363. <https://doi.org/10.48550/arXiv.2403.07905>
- [3] Ali, W., Hassan, M. U., & Ahmed, N. (2021). QoS-aware API gateway selection in cloud environments using multi-criteria decision-making. *Journal of Cloud Computing*, 10(1), 35. <https://doi.org/10.1186/s13677-021-00239-3>
- [4] Hussain Ahmad, Christoph Treude, Markus Wagner, Claudia Szabo (2024). Smart HPA: A Resource-Efficient Horizontal Pod Auto-Scaler for Microservice Architectures. <https://doi.org/10.1109/ICSA59870.2024.00013>
- [5] Vasireddy, I., Vankadara, R., & Chawla, P. (2023). Kubernetes and Docker Load Balancing: State-of-the-Art Techniques and Challenges. *International Journal of Engineering Research & Technology (IJERT)*, 12(01), 1–9. <https://doi.org/10.55524/ijirem.2023.10.6.7>
- [6] Mehta, A., & Gupta, M. (2023). Machine Learning Solutions for Load Balancing in Cloud Resource Management: A Review. *IEEE Access*, <https://doi.org/10.52783/jisem.v10i7s.873>
- [7] Lahande, V. R., Shinde, D. P., & Mohod, M. R. (2023). Reinforcement Learning Approach for Optimizing Cloud Resource Utilization With Load Balancing. *Journal of Cloud Computing*, 10(1), 25–39. <https://doi.org/10.48550/arXiv.2409.04896>
- [8] Ghosh, A., & Rajasekaran, P. (2023). AGE: Automatic Performance Evaluation of API Gateways. In *Proceedings of the 7th International Conference on Cloud Computing Technologies and Applications (CloudTech)*, pp. 55–62. <https://doi.org/10.1109/APSEC.2017.53>
- [9] Liang, S., Hall, C., Pogge, J., & Van Stry, M. (2022). CARE: Cloud Archival Repository Express via Algorithmic Machine Learning. *Intelligent Information Management*, 14(4), 133–156. <https://doi.org/10.4236/iim.2022.144010>
- [10] Zhong, H., et al. (2021). Machine Learning-based Orchestration of Containers: A Taxonomy and Future Directions. *ACM Computing Surveys (CSUR)*, 54(6), 1–36. <https://doi.org/10.1145/3510415>
- [11] Han, Y., et al. (2021). Tailored Learning-Based Scheduling for Kubernetes-Oriented Edge-Cloud System. *IEEE Transactions on Network and Service Management*, 18(2), 1587–1601. <https://doi.org/10.48550/arXiv.2101.0658>
- [12] Ghobaei-Arani, M., Souri, A., & Rahmanian, A. (2020). A survey on deep learning techniques for cloud resource management. *Journal of Systems and Software*, 167, 110610. <https://doi.org/10.1016/j.jss.2020.110610>
- [13] Nguyen, H. T., Zhang, Y., & Yang, F. (2022). A reinforcement learning-based framework for the dynamic resource allocation in cloud data centers. *IEEE Transactions on Cloud Computing*, 10(3), 790–802. <https://doi.org/10.1109/TCC.2021.3068831>
- [14] Fernandez, M., Jimenez, J., & Lopez, P. (2021). A comparative study of service mesh technologies for microservices architecture in cloud environments. **IEEE Access**, 9, 146789–146805. <https://doi.org/10.1109/ACCESS.2021.3122985>
- [15] Li, X., Wang, Z., Zhang, Q., & Zhao, Y. (2021). An adaptive load balancing algorithm based on machine learning in cloud computing. *Future Generation Computer Systems*, 123, 182–194. <https://doi.org/10.1016/j.future.2021.04.015>

- [16] (نگار سادات موسوی، سید داوود موسوی) نقش هوش مصنوعی در بهینه‌سازی مدیریت و بهره‌برداری از منابع ابری. همایش ملی ۱۴۰۲. <https://civilica. om/doc/2137866>
- [17] Zhang, H., Li, K., & Liu, J. (2022). Energy-efficient resource scheduling in cloud using multi-objective optimization and deep learning. *Journal of Parallel and Distributed Computing*, 163, 100–113. <https://doi.org/10.1016/j.jpdc.2022.03.005>
- [18] Salza, P., D'Angelo, G., & Avallone, S. (2023). Dynamic auto-scaling of microservices in Kubernetes using predictive machine learning models. **ACM Transactions on Internet Technology**, 23(2), 1–24. <https://doi.org/10.1145/3589760>
- [19] Ramdas Vankdothu · Mohd Abdul Hameed (2024) An Effective Congestion and Interference Secure Routing Protocol for Internet of Things Applications in Wireless Sensor Network. <https://doi.org/10.21203/rs.3.rs-790102/v1>
- [20] Pradeep Kumar S (2025) Design and Implementation of the Deep Reinforcement Energy Efficient Routing for the Fog-BAN-Cloud of Things using Smart Healthcare Applications. <https://doi.org/10.2478/jsiot-2024-0011>
- [21] Hu, K., Wen, L., Xu, M., & Ye, K. (2024). MSARS: A Meta-Learning and Reinforcement Learning Framework for SLO Resource Allocation and Adaptive Scaling for Microservices.
- [22] Dashtbani, M., & Tahvildari, L. (2025). STaleX: A Spatiotemporal-Aware Adaptive Auto-scaling Framework for Microservices.
- [23] Ahmad, H., et al. (2024). Smart HPA: A Resource-Efficient Horizontal Pod Auto-scaler. *Journal of Systems & Software*.
- [24] Barua, B., & Kaiser, M. S. (2024). AI-Driven Resource Allocation Framework for Microservices in Hybrid Cloud Platforms.
- [25] Zhang, G., et al. (2025). KIS-S: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-scaling.
- [26] Meng, C., Song, S., Tong, H., Pan, M., & Yu, Y. (2023). DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning.
- [27] Bai, H., Xu, M., Ye, K., Buyya, R., & Xu, C. (2024). DRPC: Distributed Reinforcement Learning Approach for Scalable Resource Provisioning in Container-based Clusters. *arXiv*.
- [28] Barua, B. & Kaiser, M. S. (2024). AI-Driven Resource Allocation Framework for Microservices in Hybrid Cloud Platforms.
- [29] Song, C., Xu, M., Ye, K., Wu, H., Gill, S. S., Buyya, R., & Xu, C. (2023). ChainsFormer: A Chain Latency-aware Resource Provisioning Approach for Microservices Cluster. *arXiv*.
- [30] Fettes, Q., Karanth, A., Bunesco, R., Beckwith, B., & Subramoney, S. (2023). Reclaimer: A Reinforcement Learning Approach to Dynamic Resource Allocation for Cloud Microservices.
- [31] Xu, M., Song, S., Ilager, S., & Xu, C. (2022). CoScal: Multi-faceted Scaling of Microservices with Reinforcement Learning.
- [32] Yang, H., Nguyen, P., & Liu, C. (۲۰۱۹). MIRAS: Model-based Reinforcement Learning for Microservice Workflow Systems.
- [33] Santos, J., et al. (2025). Gwydion: Efficient auto-scaling for complex containerized microservices. *Elsevier*.
- [34] Kallel, A., et al. (2025). A deep reinforcement learning-based optimization for containerized microservice scheduling in HFC environments. *Elsevier*.
- [35] Tan, A. & Rajesh, K. (2025). Hierarchical Deep RL for Sustainable API Throughput and Latency Optimization in Advertising Clouds. *JCEIM*.

Abstract

This research focuses on the design and implementation of an intelligent hybrid model, based on Long Short-Term Memory (LSTM) neural networks and Reinforcement Learning (RL), to optimize API performance in cloud-based microservice architectures. The model analyzes temporal traffic and workload data to predict resource consumption patterns and, through reinforcement learning policies, performs dynamic and proactive allocation of computational resources, including CPU, RAM, and GPU, at the Kubernetes cluster level. Experimental results in a real-world operational environment demonstrated that the proposed method leads to a 32-38% reduction in average API response latency, a 30% decrease in SLA violations with an increase in response success rate, operational cost reductions between 35-50%, a 30-45% decrease in energy consumption, and a 35-40% increase in scalability. Overall, the combination of reinforcement learning and LSTM networks in the microservices context proves to be an effective, adaptive, and generalizable approach for improving Quality of Service (QoS) and intelligent resource management in dynamic, distributed, and scalable cloud environments.

Keywords: Reinforcement Learning, LSTM Neural Network, Microservices, API, Kubernetes, Resource Allocation, Quality of Service (QoS), Cloud Computing.



ISLAMIC AZAD UNIVERSITY
SCIENCE AND RESEARCH BRANCH

Faculty of Electrical and Computer Engineering - Department of Computer
Thesis for receiving M.Sc degree on Computer Engineering - Networks

Subject:

Enhancing the Performance of Application Programming Interfaces in Cloud
Computing Environments Using Reinforcement Learning Algorithms and
Short-Term Memory Neural Networks within a Micro-services Architecture

Thesis Advisor:
M.Alizadeh Ph.D.

Consulting Advisor:
F.ZamaniBorujeni Ph.D.

By:
M.Nokhiz

Spring 2026

1-data_generator.py

```

import numpy as np
import pandas as pd

def generate_synthetic_api_data(
    n_steps=1000,      # تعداد گام‌های زمانی (طول شبیه‌سازی)
    noise_level=0.1,    # میزان نویز تصادفی برای واقعی‌تر شدن داده‌ها
    spike_prob=0.05,    # احتمال وقوع جهش ناگهانی بار (Burst)
    random_seed=42      # مقدار ثابت برای تکرارپذیری نتایج
):
    # برای یکسان بودن خروجی‌ها در اجراهای مختلف seed ثابت کردن
    np.random.seed(random_seed)

    # تعریف محور زمان (گام‌های زمانی متوالی)
    time = np.arange(n_steps)

    # -----
    # الگوی پایه‌بار کاری (رفتار تناوبی / سینوسی)
    # شبیه‌سازی تغییرات طبیعی بار در طول زمان
    # -----
    base_load = 50 + 20 * np.sin(2 * np.pi * time / 200)

    # -----
    # اضافه‌کردن نویز تصادفی برای شبیه‌سازی رفتار غیر قابل پیش‌بینی کاربران
    # -----
    noise = np.random.normal(0, noise_level * 20, n_steps)

    # -----
    # flash crowd (مانند) ایجاد جهش‌های ناگهانی بار
    # -----
    spikes = np.zeros(n_steps)
    for i in range(n_steps):
        if np.random.rand() < spike_prob:
            spikes[i] = np.random.uniform(30, 70)

    # نرخ نهایی درخواست‌ها (در ثانیه)
    requests_per_sec = base_load + noise + spikes

    # جلوگیری از منفی یا صفر شدن نرخ درخواست
    requests_per_sec = np.clip(requests_per_sec, 1, None)

    # -----
    # زمان پاسخ با افزایش بار کاری بیشتر می‌شود
    # -----
    response_time = (
        100

```

```

    + 2.5 * requests_per_sec
    + np.random.normal(0, 20, n_steps)
)

# -----
# متناسب با میزان بار کاری CPU مصرف
# -----
cpu_usage = (
    20
    + 0.8 * requests_per_sec
    + np.random.normal(0, 5, n_steps)
)

# -----
# متناسب با میزان بار کاری RAM مصرف
# -----
ram_usage = (
    500
    + 5 * requests_per_sec
    + np.random.normal(0, 30, n_steps)
)

# -----
# نرخ خطا در بارهای بالا کمی افزایش می‌یابد
# -----
error_rate = np.clip(
    0.01
    + 0.0005 * requests_per_sec
    + np.random.normal(0, 0.005, n_steps),
    0,
    1
)

# -----
# DataFrame تجمع تمام داده‌ها در قالب
# -----
data = pd.DataFrame({
    "requests_per_sec": requests_per_sec,
    "response_time": response_time,
    "cpu_usage": cpu_usage,
    "ram_usage": ram_usage,
    "error_rate": error_rate
})

return data

# -----
# نقطه شروع اجرای برنامه
# -----
if __name__ == "__main__":
    # تولید داده‌های مصنوعی
    df = generate_synthetic_api_data(n_steps=1200)

```

```
# برای مراحل بعدی CSV ذخیره داده‌ها در فایل
df.to_csv("synthetic_api_data.csv", index=False)

print("داده‌های مصنوعی با موفقیت تولید شدند.")
print("Synthetic API data generated: synthetic_api_data.csv")
```

2-lstm_model

```
import numpy as np
import pandas as pd
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
```

```
# -----
# LSTM تابع تبدیل داده سری‌زمانی به نمونه‌های ورودی
# -----
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(len(data) - window_size):
        # پنجره زمانی به‌عنوان ورودی
        X.append(data[i:i + window_size])
        # مقدار گام بعدی به‌عنوان خروجی
        y.append(data[i + window_size])
    return np.array(X), np.array(y)
```

```
# -----
# بارگذاری داده‌های مصنوعی
# -----
df = pd.read_csv("synthetic_api_data.csv")
```

```
# انتخاب ویژگی هدف (نرخ درخواست)
target_column = "requests_per_sec"
values = df[[target_column]].values
```

```
# -----
# نرمال‌سازی داده‌ها بین ۰ و ۱
# (LSTM ضروری برای پایداری آموزش)
# -----
scaler = MinMaxScaler()
values_scaled = scaler.fit_transform(values)
```

```
# -----
# LSTM تبدیل داده‌ها به ساختار مناسب
```

```

# -----
WINDOW_SIZE = 20
X, y = create_sequences(values_scaled, WINDOW_SIZE)

# -----
# تقسیم داده‌ها به مجموعه آموزش و آزمون
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

# -----
# تعریف معماری مدل LSTM
# -----
model = Sequential()

# استخراج الگوهای زمانی LSTM لایه اول
model.add(LSTM(
    units=64,
    return_sequences=True,
    input_shape=(WINDOW_SIZE, 1)
))

# یادگیری وابستگی‌های عمیق‌تر LSTM لایه دوم
model.add(LSTM(units=32))

# لایه خروجی (پیش‌بینی مقدار عددی)
model.add(Dense(1))

# -----
# کامپایل مدل
# -----
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse"
)

# -----
# آموزش مدل
# -----
history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# -----
# ارزیابی مدل روی داده‌های آزمون
# -----
test_loss = model.evaluate(X_test, y_test, verbose=0)

```

```

print("Test MSE:", test_loss)

# -----
# انجام پیش‌بینی
# -----
predictions_scaled = model.predict(X_test)

# بازگرداندن پیش‌بینی‌ها به مقیاس اصلی
predictions = scaler.inverse_transform(predictions_scaled)
y_test_real = scaler.inverse_transform(y_test)

# نمایش چند نمونه خروجی
print("\nنمونه‌ای از پیش‌بینی‌ها:")
for i in range(5):
    print(
        f"مقدار واقعی: {y_test_real[i][0]:.2f} | "
        f"مقدار پیش‌بینی شده: {predictions[i][0]:.2f}"
    )

```

3-lstm_plot

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam

# -----
# تبدیل داده سری‌زمانی به ورودی LSTM
# -----
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i + window_size])
        y.append(data[i + window_size])
    return np.array(X), np.array(y)

# -----
# بارگذاری داده‌ها
# -----
df = pd.read_csv("synthetic_api_data.csv")
values = df[["requests_per_sec"]].values

# نرمال‌سازی
scaler = MinMaxScaler()

```

```

values_scaled = scaler.fit_transform(values)

# ساخت داده‌های سری زمانی
WINDOW_SIZE = 20
X, y = create_sequences(values_scaled, WINDOW_SIZE)

# تقسیم آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

# -----
# تعریف مدل LSTM
# -----
model = Sequential()
model.add(LSTM(64, return_sequences=True, input_shape=(WINDOW_SIZE, 1)))
model.add(LSTM(32))
model.add(Dense(1))

model.compile(optimizer=Adam(0.001), loss="mse")

# آموزش
model.fit(X_train, y_train, epochs=30, batch_size=32, verbose=0)

# پیش‌بینی
y_pred_scaled = model.predict(X_test)

# بازگرداندن به مقیاس واقعی
y_pred = scaler.inverse_transform(y_pred_scaled)
y_test_real = scaler.inverse_transform(y_test)

# -----
# رسم نمودار
# -----
plt.figure()
plt.plot(y_test_real[:200], label="مقدار واقعی")
plt.plot(y_pred[:200], label="پیش‌بینی LSTM")
plt.xlabel("گام زمانی")
plt.ylabel("نرخ درخواست (requests/sec)")
plt.title("LSTM مقایسه مقدار واقعی و پیش‌بینی شده توسط")
plt.legend()
plt.show()

```

4-random_forest_model

```

import numpy as np
import pandas as pd

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

```

```

# -----
# بارگذاری داده‌ها
# -----
df = pd.read_csv("synthetic_api_data.csv")

# ویژگی‌ها و متغیر هدف
X = df[["response_time", "cpu_usage", "ram_usage", "error_rate"]]
y = df["requests_per_sec"]

# -----
# تقسیم داده‌ها به آموزش و آزمون
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

# -----
# تعریف مدل Random Forest
# -----
rf_model = RandomForestRegressor(
    n_estimators=100,    # تعداد درخت‌ها
    max_depth=10,       # عمق هر درخت
    random_state=42
)

# آموزش مدل
rf_model.fit(X_train, y_train)

# -----
# پیش‌بینی و ارزیابی
# -----
y_pred = rf_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Random Forest MSE:", mse)

# نمایش چند نمونه خروجی
print("\nنمونه پیش‌بینی‌های Random Forest:")
for i in range(5):
    print(
        f"واقعی: {y_test.iloc[i]:.2f} | "
        f"پیش‌بینی RF: {y_pred[i]:.2f}"
    )

```

5-model_comparison

```

import numpy as np
import pandas as pd

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.ensemble import RandomForestRegressor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam

# -----
# تابع ساخت داده سری زمانی
# -----
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i + window_size])
        y.append(data[i + window_size])
    return np.array(X), np.array(y)

# -----
# بارگذاری داده
# -----
df = pd.read_csv("synthetic_api_data.csv")

# =====
# بخش اول: LSTM
# =====
values = df[["requests_per_sec"]].values

scaler = MinMaxScaler()
values_scaled = scaler.fit_transform(values)

WINDOW_SIZE = 20
X_lstm, y_lstm = create_sequences(values_scaled, WINDOW_SIZE)

X_train_lstm, X_test_lstm, y_train_lstm, y_test_lstm = train_test_split(
    X_lstm, y_lstm, test_size=0.2, shuffle=False
)

# مدل LSTM
lstm_model = Sequential()
lstm_model.add(LSTM(64, return_sequences=True, input_shape=(WINDOW_SIZE, 1)))
lstm_model.add(LSTM(32))
lstm_model.add(Dense(1))

lstm_model.compile(optimizer=Adam(0.001), loss="mse")
lstm_model.fit(X_train_lstm, y_train_lstm, epochs=30, batch_size=32, verbose=0)

```

```

# پیش‌بینی LSTM
lstm_pred_scaled = lstm_model.predict(X_test_lstm)
lstm_pred = scaler.inverse_transform(lstm_pred_scaled)
y_test_real_lstm = scaler.inverse_transform(y_test_lstm)

# محاسبه معیارها برای LSTM
lstm_mse = mean_squared_error(y_test_real_lstm, lstm_pred)
lstm_rmse = np.sqrt(lstm_mse)
lstm_mae = mean_absolute_error(y_test_real_lstm, lstm_pred)

# =====
# بخش دوم: Random Forest
# =====
X_rf = df[["response_time", "cpu_usage", "ram_usage", "error_rate"]]
y_rf = df["requests_per_sec"]

X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(
    X_rf, y_rf, test_size=0.2, shuffle=False
)

rf_model = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42
)

rf_model.fit(X_train_rf, y_train_rf)
rf_pred = rf_model.predict(X_test_rf)

# محاسبه معیارها برای RF
rf_mse = mean_squared_error(y_test_rf, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_mae = mean_absolute_error(y_test_rf, rf_pred)

# -----
# چاپ نتایج
# -----
print("\n===== نتایج مقایسه =====\n")

print("مدل LSTM:")
print(f"MSE = {lstm_mse:.3f}")
print(f"RMSE = {lstm_rmse:.3f}")
print(f"MAE = {lstm_mae:.3f}\n")

print("مدل Random Forest:")
print(f"MSE = {rf_mse:.3f}")
print(f"RMSE = {rf_rmse:.3f}")
print(f"MAE = {rf_mae:.3f}")

print("\n=====")

```